Spring Boot 4 & Spring Framework 7 Annotations Interview Guide

Table of Contents

1. [Core Spring Framework Annotations]
2. [Spring Boot 4 Core Annotations]
3. [Web & REST Annotations]
4. [Validation Annotations]
5. [Conditional Annotations]
6. [Data Access Annotations]
7. [AOP & Aspect Annotations]
8. [Spring Security Annotations]
9. [Testing Annotations]
10. [Caching Annotations]
11. [Scheduling Annotations]
12. [Actuator & Monitoring Annotations]
13. [Spring Boot 4 New Features]
14. [Common Annotation Combinations]
15. [Annotation Best Practices]
16. [Spring Context Annotations]
17. [AspectJ Annotations]
18. [JMS, Redis, RocksDB, and Kafka Annotations]
19. [Spring Cloud Annotations]
20. [Advanced Spring Data Annotations]
21. [Spring WebFlux & Reactive Annotations]
22. [GraphQL Annotations]
23. [REST Microservices Annotations]
24. [gRPC Annotations]
25. [Resilience4j Annotations]
26. [Microservices Design Patterns]
27. [System Design Patterns & Principles]
28. [Domain-Driven Design (DDD) Annotations]
29. [Object-Oriented Modeling & Design (OOMD)]
30. [Hibernate Annotations]
31. [MyBatis Annotations]
32. [Java JSR Annotations (Up to Java 26)]
33. [Guava Annotations]
34. [Apache Commons Annotations]
35. [Logging Framework Annotations]
36. [Testing Annotations]

---

1. Core Spring Framework Annotations

@Component & Stereotype Annotations

```
@Component              // Generic Spring-managed bean
@Service               // Service layer component
```

```
@Repository            // DAO layer component (enables exception transl
@Controller            // MVC controller component
@RestController         // REST controller (combines @Controller + @Res
```

- **@Component**: Base annotation for any Spring-managed bean
- **@Service**: Indicates service layer business logic
- **@Repository**: Enables DAO exception translation to Spring's DataAccessException
- **@Controller**: Traditional MVC controllers returning views
- **@RestController**: Returns JSON/XML directly, no view resolution

Dependency Injection Annotations

```
@Autowired             // Field, constructor, or setter injection
@Qualifier("beanName") // Disambiguate when multiple beans of same type
@Primary               // Mark as primary candidate when multiple beans
@Resource(name = "beanName") // JSR-250 injection (by name)
@Inject                // JSR-330 injection (equivalent to @Autowired)
```

- **@Autowired**: Preferred on constructors (required = true by default)
- **@Qualifier**: Used with @Autowired to specify bean name
- **@Primary**: Marks default bean when multiple candidates exist
- **@Resource**: Name-based injection, falls back to type if name not found

Configuration Annotations

```
@Configuration         // Indicates configuration class
@Bean                  // Declares a bean method
@ComponentScan         // Scans for components (basePackages)
@Import                // Import other configuration classes
@PropertySource        // Load properties file
@Profile               // Bean activation based on active profile
```

---

2. Spring Boot 4 Core Annotations

Application Configuration

```
@SpringBootApplication    // Combines @Configuration + @EnableAutoConfi
@EnableAutoConfiguration // Enables Spring Boot's auto-configuration me
@SpringBootConfiguration // Alternative to @Configuration for Boot apps
```

Properties & Configuration

```
@ConfigurationProperties // Bind external properties to fields
@Value("${property.key}") // Inject single property value
@EmbeddedId               // Embedded primary key in JPA entities
@Validated                // Enable validation on configuration properti
```

```
@ConfigurationProperties(prefix = "app.datasource")
@Validated
public class DataSourceProperties {
```

```
    @NotNull
    private String url;

    @Min(1)
    @Max(100)
    private int maxConnections = 10;

    // Getters and setters
}
```

Conditional Annotations

```
@ConditionalOnClass        // Bean exists if class is on classpath
@ConditionalOnMissingClass // Bean exists if class is NOT on classpath
@ConditionalOnBean         // Bean exists if another bean exists
@ConditionalOnMissingBean  // Bean exists if another bean is missing
@ConditionalOnProperty     // Bean exists based on property value
@ConditionalOnResource     // Bean exists if resource is available
@ConditionalOnWebApplication // Bean exists in web context
@ConditionalOnNotWebApplication // Bean exists in non-web context

@Bean
@ConditionalOnProperty(name = "cache.enabled", havingValue = "true")
public CacheManager cacheManager() {
    return new ConcurrentMapCacheManager("users", "products");
}
```

---

3. Web & REST Annotations

Request Mapping

```
@RequestMapping          // Base URL mapping (class/method level)
@GetMapping             // HTTP GET mapping
@PostMapping            // HTTP POST mapping
@PutMapping             // HTTP PUT mapping
@DeleteMapping          // HTTP DELETE mapping
@PatchMapping           // HTTP PATCH mapping

@RestController
@RequestMapping("/api/v1/users")
public class UserController {

    @GetMapping("/{id}")
    public User getUser(@PathVariable Long id) {
        return userService.findById(id);
    }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public User createUser(@Valid @RequestBody User user) {
        return userService.create(user);
```

```
    }
}
```

Request Parameter Binding

```
@PathVariable          // Bind URL template variable
@RequestParam          // Bind query parameter
@RequestBody          // Bind HTTP request body
@RequestHeader         // Bind HTTP header
@CookieValue           // Bind HTTP cookie
@MatrixVariable       // Bind matrix variable in URL
@RequestPart          // Bind multipart/form-data part
```

```
@GetMapping("/search")
public List<User> searchUsers(
    @RequestParam(required = false) String name,
    @RequestParam(defaultValue = "0") int page,
    @RequestParam("size") int pageSize,
    @RequestHeader("User-Agent") String userAgent) {
    return userService.search(name, page, pageSize);
}
```

Response Handling

```
@ResponseBody         // Return value as response body
@ResponseStatus        // Set HTTP status code
 ResponseEntity<T>    // Full response control (status, headers, body)
```

```
@GetMapping("/users/{id}")
public ResponseEntity<User> getUser(@PathVariable Long id) {
    return userService.findById(id)
        .map(user -> ResponseEntity.ok().header("X-Custom-Header", "val
        .orElse(ResponseEntity.notFound().build());
}
```

Cross-Origin & CORS

```
@CrossOrigin           // Enable CORS for controller/method
@CrossOrigin(origins = "http://localhost:3000", methods = {GET, POST})
```

---

5. Conditional Annotations

Spring Boot Conditional Annotations

```
@ConditionalOnClass        // Bean exists if class is on classpath
@ConditionalOnMissingClass // Bean exists if class is NOT on classpath
@ConditionalOnBean         // Bean exists if another bean exists
@ConditionalOnMissingBean  // Bean exists if another bean is missing
@ConditionalOnProperty     // Bean exists based on property value
@ConditionalOnResource     // Bean exists if resource is available
@ConditionalOnWebApplication // Bean exists in web context
```

```
@ConditionalOnNotWebApplication // Bean exists in non-web context
@ConditionalOnJava         // Bean exists based on Java version
@ConditionalOnJndi         // Bean exists if JNDI location exists
@ConditionalOnExpression   // Bean exists based on SpEL expression
@ConditionalOnCloudPlatform // Bean exists on specific cloud platform
```

Conditional Examples

```
@Bean
@ConditionalOnProperty(name = "cache.enabled", havingValue = "true")
public CacheManager cacheManager() {
    return new ConcurrentMapCacheManager("users", "products");
}

@Bean
@ConditionalOnClass(name = "redis.clients.jedis.Jedis")
public RedisTemplate<String, Object> redisTemplate() {
    return new RedisTemplate<>();
}

@Bean
@ConditionalOnMissingBean(DataSource.class)
public DataSource defaultDataSource() {
    return new HikariDataSource();
}

@Configuration
@ConditionalOnWebApplication
public class WebConfiguration {

    @Bean
    @ConditionalOnProperty(name = "cors.enabled", havingValue = "true")
    public CorsConfigurationSource corsConfigurationSource() {
        CorsConfiguration configuration = new CorsConfiguration();
        configuration.setAllowedOrigins(Arrays.asList("*"));
        configuration.setAllowedMethods(Arrays.asList("GET", "POST", "P
        configuration.setAllowedHeaders(Arrays.asList("*"));
        UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfig
        source.registerCorsConfiguration("/**", configuration);
        return source;
    }
}

@Configuration
@ConditionalOnNotWebApplication
public class StandaloneConfiguration {

    @Bean
    public CommandLineRunner commandLineRunner() {
        return args -> System.out.println("Running in standalone mode")
    }
}
```

Custom Conditional Annotations

```
@Target({ElementType.TYPE, ElementType.METHOD})
@Retention(RetentionPolicy.RUNTIME)
@Conditional(OnDatabaseCondition.class)
public @interface ConditionalOnDatabase {
    String value() default "mysql";
}

public class OnDatabaseCondition implements Condition {

    @Override
    public boolean matches(ConditionContext context, AnnotatedTypeMetad
        Map<String, Object> attributes = metadata.getAnnotationAttribut
            ConditionalOnDatabase.class.getName());
        String database = (String) attributes.get("value");

        String property = context.getEnvironment().getProperty("app.dat
        return database.equals(property);
    }
}

@Configuration
public class DatabaseConfiguration {

    @Bean
    @ConditionalOnDatabase("mysql")
    public DataSource mysqlDataSource() {
        return new HikariDataSource();
    }

    @Bean
    @ConditionalOnDatabase("postgresql")
    public DataSource postgresqlDataSource() {
        return new HikariDataSource();
    }
}
```

---

6. Data Access Annotations

JPA Entity Mapping

```
@Entity                     // JPA entity class
@Table(name = "users")      // Specify table name
@Id                         // Primary key
@GeneratedValue             // Auto-generate primary key
@Column                     // Column specification
@Transient                  // Non-persistent field
@Embedded                   // Embedded object
@Enumerated                 // Enum mapping
@Lob                        // Large object (BLOB/CLOB)
```

```
@Temporal                   // Date/time mapping
@Basic                      // Basic property mapping
@Access                     // Property vs field access
@Version                    // Optimistic locking version
```

Comprehensive Entity Example

```
@Entity
@Table(name = "employees",
       indexes = {
           @Index(name = "idx_employee_email", columnList = "email"),
           @Index(name = "idx_employee_department", columnList = "depar
       },
       uniqueConstraints = {
           @UniqueConstraint(columnNames = "email")
       })
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "first_name", nullable = false, length = 50)
    private String firstName;

    @Column(name = "last_name", nullable = false, length = 50)
    private String lastName;

    @Column(name = "email", nullable = false, unique = true, length = 1
    private String email;

    @Enumerated(EnumType.STRING)
    @Column(name = "department", nullable = false)
    private Department department;

    @Column(name = "salary", precision = 10, scale = 2)
    private BigDecimal salary;

    @Lob
    @Column(name = "profile_picture")
    private byte[] profilePicture;

    @Temporal(TemporalType.TIMESTAMP)
    @Column(name = "hire_date", nullable = false)
    private Date hireDate;

    @Temporal(TemporalType.TIMESTAMP)
    @Column(name = "last_login")
    private Date lastLogin;

    @Version
    private Long version;
```

```
    @Transient
    private String fullName; // Not persisted

    @Embedded
    private Address address;

    // Getters and setters
}

@Embeddable
public class Address {
    @Column(name = "street", length = 200)
    private String street;

    @Column(name = "city", length = 100)
    private String city;

    @Column(name = "state", length = 50)
    private String state;

    @Column(name = "zip_code", length = 10)
    private String zipCode;
}
```

Relationship Mapping

```
@Entity
public class Department {

    @Id
    @GeneratedValue
    private Long id;

    @Column(nullable = false, unique = true)
    private String name;

    @OneToMany(mappedBy = "department", cascade = CascadeType.ALL, fetc
    private List<Employee> employees = new ArrayList<>();

    @OneToOne(mappedBy = "department")
    private DepartmentHead head;
}

@Entity
public class Employee {

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "department_id", nullable = false)
    private Department department;

    @OneToOne(fetch = FetchType.LAZY, cascade = CascadeType.ALL)
    @JoinColumn(name = "employee_details_id")
    private EmployeeDetails details;
```

```
    @ManyToMany(fetch = FetchType.LAZY)
    @JoinTable(
        name = "employee_projects",
        joinColumns = @JoinColumn(name = "employee_id"),
        inverseJoinColumns = @JoinColumn(name = "project_id")
    )
    private Set<Project> projects = new HashSet<>();
}
```

Spring Data JPA Repository Annotations

```
@Repository                  // Spring Data repository
@EnableJpaRepositories        // Enable JPA repositories
@Transactional               // Transaction management
@Query                      // Custom JPQL query
@Modifying                  // Query modifies data
@Param                      // Named parameter in query
@Lock                       // Pessimistic locking
@QueryHints                 // JPA query hints
@Procedure                  // Stored procedure call
@QueryRewriter              // Custom query rewriting
```

Repository Interface Examples

```
@Repository
public interface UserRepository extends JpaRepository<User, Long>, JpaS

    // Derived query methods
    Optional<User> findByEmail(String email);
    List<User> findByFirstNameAndLastName(String firstName, String last
    List<User> findByDepartmentAndSalaryGreaterThan(Department departme

    // Custom JPQL queries
    @Query("SELECT u FROM User u WHERE u.email = :email AND u.active =
    Optional<User> findByEmailAndActive(@Param("email") String email);

    @Query("SELECT u FROM User u WHERE u.department = :dept ORDER BY u.
    List<User> findByDepartmentOrderBySalary(@Param("dept") Department

    // Native SQL queries
    @Query(value = "SELECT * FROM users u WHERE u.email LIKE %?1%", nat
    List<User> findByEmailContaining(String emailPattern);

    // Modifying queries
    @Modifying
    @Transactional
    @Query("UPDATE User u SET u.lastLogin = CURRENT_TIMESTAMP WHERE u.i
    int updateLastLogin(@Param("id") Long userId);

    @Modifying
    @Transactional
    @Query("DELETE FROM User u WHERE u.active = false AND u.lastLogin <
```

```
    int deleteInactiveUsers(@Param("cutoffDate") LocalDateTime cutoffDa

    // Count queries
    @Query("SELECT COUNT(u) FROM User u WHERE u.department = :dept")
    long countByDepartment(@Param("dept") Department department);

    // Pagination
    @Query("SELECT u FROM User u WHERE u.department = :dept")
    Page<User> findByDepartment(@Param("dept") Department department, P

    // Locking
    @Lock(LockModeType.PESSIMISTIC_WRITE)
    @Query("SELECT u FROM User u WHERE u.id = :id")
    Optional<User> findByIdForUpdate(@Param("id") Long id);

    // Query hints
    @QueryHints(@QueryHint(name = org.hibernate.jpa.QueryHints.HINT_CAC
    @Query("SELECT u FROM User u WHERE u.department = :dept")
    List<User> findByDepartmentWithCache(@Param("dept") Department depa

    // Stored procedure
    @Procedure(name = "calculate_user_bonus")
    BigDecimal calculateUserBonus(@Param("user_id") Long userId, @Param
}
```

Custom Repository Implementation

```
public interface UserRepositoryCustom {
    List<User> findUsersWithComplexCriteria(String searchTerm, Departme
    void bulkUpdateDepartment(Long oldDeptId, Long newDeptId);
}

@Repository
public class UserRepositoryImpl implements UserRepositoryCustom {

    @PersistenceContext
    private EntityManager entityManager;

    @Override
    public List<User> findUsersWithComplexCriteria(String searchTerm, D
        CriteriaBuilder cb = entityManager.getCriteriaBuilder();
        CriteriaQuery<User> query = cb.createQuery(User.class);
        Root<User> root = query.from(User.class);

        List<Predicate> predicates = new ArrayList<>();

        if (searchTerm != null) {
            predicates.add(cb.or(
                cb.like(cb.lower(root.get("firstName")), "%" + searchTe
                cb.like(cb.lower(root.get("lastName")), "%" + searchTer
                cb.like(cb.lower(root.get("email")), "%" + searchTerm.t
            ));
        }
```

```
        if (department != null) {
            predicates.add(cb.equal(root.get("department"), department)
        }

        if (minSalary != null) {
            predicates.add(cb.greaterThanOrEqualTo(root.get("salary"),
        }

        query.where(predicates.toArray(new Predicate[0]));
        return entityManager.createQuery(query).getResultList();
    }

    @Override
    @Transactional
    public void bulkUpdateDepartment(Long oldDeptId, Long newDeptId) {
        String jpql = "UPDATE User u SET u.department.id = :newDeptId W
        entityManager.createQuery(jpql)
            .setParameter("newDeptId", newDeptId)
            .setParameter("oldDeptId", oldDeptId)
            .executeUpdate();
    }
}
```

Transaction Management Annotations

```
@Transactional                // Method/class-level transaction
@Transactional(propagation = Propagation.REQUIRED)
@Transactional(isolation = Isolation.READ_COMMITTED)
@Transactional(timeout = 30)
@Transactional(readOnly = true)
@Transactional(rollbackFor = Exception.class)
@Transactional(noRollbackFor = BusinessException.class)
@Transactional(value = "transactionManager")
```

Transaction Examples

```
@Service
public class PaymentService {

    @Transactional(
        propagation = Propagation.REQUIRED,
        isolation = Isolation.READ_COMMITTED,
        timeout = 30,
        rollbackFor = {PaymentException.class, InsufficientFundsExcepti
    )
    public Payment processPayment(PaymentRequest request) {
        // Payment processing logic
        Account fromAccount = accountRepository.findById(request.getFro
            .orElseThrow(() -> new AccountNotFoundException(request.get

        Account toAccount = accountRepository.findById(request.getToAcc
            .orElseThrow(() -> new AccountNotFoundException(request.get
```

```
        // Validate sufficient funds
        if (fromAccount.getBalance().compareTo(request.getAmount()) < 0
            throw new InsufficientFundsException("Insufficient funds");
        }

        // Perform transfer
        fromAccount.setBalance(fromAccount.getBalance().subtract(reques
        toAccount.setBalance(toAccount.getBalance().add(request.getAmou

        accountRepository.save(fromAccount);
        accountRepository.save(toAccount);

        return createPaymentRecord(request);
    }

    @Transactional(readOnly = true)
    public List<Payment> getPaymentHistory(Long userId) {
        return paymentRepository.findByUserId(userId);
    }

    @Transactional(propagation = Propagation.REQUIRES_NEW)
    public void auditPayment(Payment payment) {
        // Always runs in a new transaction
        auditRepository.logPayment(payment);
    }

    @Transactional(propagation = Propagation.NOT_SUPPORTED)
    public List<Payment> generatePaymentReport(LocalDate startDate, Loc
        // Runs without transaction (for large data reads)
        return paymentRepository.findByDateRange(startDate, endDate);
    }
}
```

JDBC Template Annotations

```
@Repository
public class JdbcUserRepository {

    private final JdbcTemplate jdbcTemplate;
    private final SimpleJdbcInsert userInsert;
    private final SimpleJdbcCall getUserProcedure;

    public JdbcUserRepository(JdbcTemplate jdbcTemplate) {
        this.jdbcTemplate = jdbcTemplate;
        this.userInsert = new SimpleJdbcInsert(jdbcTemplate)
            .withTableName("users")
            .usingGeneratedKeyColumns("id");

        this.getUserProcedure = new SimpleJdbcCall(jdbcTemplate)
            .withProcedureName("get_user_details")
            .declareParameters(
                new SqlParameter("p_user_id", Types.BIGINT),
```

```
            new SqlOutParameter("p_user_data", Types.REF_CURSOR, ne
        );
}

public User save(User user) {
    Map<String, Object> parameters = new HashMap<>();
    parameters.put("first_name", user.getFirstName());
    parameters.put("last_name", user.getLastName());
    parameters.put("email", user.getEmail());
    parameters.put("department_id", user.getDepartment().getId());

    Number id = userInsert.executeAndReturnKey(parameters);
    user.setId(id.longValue());
    return user;
}

public Optional<User> findById(Long id) {
    String sql = "SELECT * FROM users WHERE id = ?";
    try {
        User user = jdbcTemplate.queryForObject(sql, new UserRowMap
        return Optional.of(user);
    } catch (EmptyResultDataAccessException e) {
        return Optional.empty();
    }
}

public List<User> findByDepartment(Long departmentId) {
    String sql = "SELECT * FROM users WHERE department_id = ?";
    return jdbcTemplate.query(sql, new UserRowMapper(), departmentI
}

@Transactional
public int update(User user) {
    String sql = "UPDATE users SET first_name = ?, last_name = ?, e
    return jdbcTemplate.update(sql,
        user.getFirstName(),
        user.getLastName(),
        user.getEmail(),
        user.getDepartment().getId(),
        user.getId());
}

@Transactional
public int deleteById(Long id) {
    String sql = "DELETE FROM users WHERE id = ?";
    return jdbcTemplate.update(sql, id);
}

public User callGetUserProcedure(Long userId) {
    SqlParameterSource in = new MapSqlParameterSource()
        .addValue("p_user_id", userId);

    Map<String, Object> out = getUserProcedure.execute(in);
```

```
        return (User) out.get("p_user_data");
    }

    private static class UserRowMapper implements RowMapper<User> {
        @Override
        public User mapRow(ResultSet rs, int rowNum) throws SQLExceptio
            User user = new User();
            user.setId(rs.getLong("id"));
            user.setFirstName(rs.getString("first_name"));
            user.setLastName(rs.getString("last_name"));
            user.setEmail(rs.getString("email"));
            // Map other fields
            return user;
        }
    }
}
```

Database Initialization

```
@Profile("dev")
@Configuration
public class DatabaseInitializer {

    @Bean
    public DataSourceInitializer dataSourceInitializer(DataSource dataS
        DataSourceInitializer initializer = new DataSourceInitializer()
        initializer.setDataSource(dataSource);
        initializer.setDatabasePopulator(databasePopulator());
        return initializer;
    }

    private DatabasePopulator databasePopulator() {
        ResourceDatabasePopulator populator = new ResourceDatabasePopul
        populator.addScript(new ClassPathResource("schema.sql"));
        populator.addScript(new ClassPathResource("data.sql"));
        populator.setSeparator(";");
        return populator;
    }
}

// Using Flyway for database migrations
@Configuration
public class FlywayConfig {

    @Bean
    @ConfigurationProperties(prefix = "spring.flyway")
    public Flyway flyway(DataSource dataSource) {
        return Flyway.configure()
            .dataSource(dataSource)
            .baselineOnMigrate(true)
            .validateOnMigrate(true)
            .load();
    }
```

```
    @Bean
    public FlywayMigrationStrategy flywayMigrationStrategy() {
        return flyway -> {
            // Custom migration logic
            flyway.migrate();
        };
    }
}
```

JPA Event Listeners

```
@EntityListeners(AuditListener.class)
@Entity
public class AuditableEntity {

    @CreatedDate
    @Column(name = "created_at", updatable = false)
    private LocalDateTime createdAt;

    @LastModifiedDate
    @Column(name = "updated_at")
    private LocalDateTime updatedAt;

    @CreatedBy
    @Column(name = "created_by", updatable = false)
    private String createdBy;

    @LastModifiedBy
    @Column(name = "updated_by")
    private String updatedBy;
}

@Component
public class AuditListener {

    @PrePersist
    public void prePersist(Object entity) {
        if (entity instanceof AuditableEntity) {
            AuditableEntity auditable = (AuditableEntity) entity;
            auditable.setCreatedAt(LocalDateTime.now());
            auditable.setCreatedBy(getCurrentUser());
        }
    }

    @PreUpdate
    public void preUpdate(Object entity) {
        if (entity instanceof AuditableEntity) {
            AuditableEntity auditable = (AuditableEntity) entity;
            auditable.setUpdatedAt(LocalDateTime.now());
            auditable.setUpdatedBy(getCurrentUser());
        }
    }
```

```
    private String getCurrentUser() {
        return SecurityContextHolder.getContext().getAuthentication().g
    }
}
```

Data Access Best Practices

- **Use appropriate fetch types** - LAZY for associations, EAGER for essential data
- **Implement proper transaction boundaries** - Keep transactions short and focused
- **Use connection pooling** - Configure HikariCP for optimal performance
- **Batch operations for bulk updates** - Use `@BatchSize` and JDBC batch updates
- **Optimize queries** - Use proper indexing and avoid N+1 problems
- **Handle exceptions properly** - Use Spring's DataAccessException hierarchy
- **Use DTOs for API responses** - Avoid exposing entities directly
- **Implement proper caching** - Use `@Cacheable` for frequently accessed data

---

4. Validation Annotations

Bean Validation (JSR-380)

```
@Valid                  // Validate nested object
@NotNull               // Field must not be null
@NotEmpty              // String/collection must not be empty
@NotBlank              // String must not be blank
@Size(min=, max=)      // Size constraints
@Min(value) / @Max     // Numeric bounds
@Email                 // Valid email format
@Pattern(regexp)       // Regular expression
@Future / @Past        // Date constraints
@Positive / @Negative  // Positive/negative numbers
@AssertTrue            // Must be true

public class UserRegistrationRequest {

    @NotBlank(message = "Username is required")
    @Size(min = 3, max = 20, message = "Username must be between 3 and
    private String username;

    @Email(message = "Invalid email format")
    @NotBlank(message = "Email is required")
    private String email;

    @NotNull(message = "Age is required")
    @Min(value = 18, message = "Must be at least 18 years old")
    private Integer age;

    @Valid
    @NotNull(message = "Address is required")
    private Address address;
}
```

Custom Validation

```
@Target({ElementType.FIELD, ElementType.PARAMETER})
@Retention(RetentionPolicy.RUNTIME)
@Constraint(validatedBy = UniqueUsernameValidator.class)
public @interface UniqueUsername {
    String message() default "Username must be unique";
    Class<?>[] groups() default {};
    Class<? extends Payload>[] payload() default {};
}
```

---

9. Testing Annotations

Spring Boot Testing Annotations

```
@SpringBootTest         // Full Spring Boot test context
@WebMvcTest            // Web layer test (controllers only)
@DataJpaTest           // JPA repository test
@JdbcTest              // JDBC repository test
@DataMongoTest         // MongoDB test
@RestClientTest        // REST client test
@JsonTest              // JSON serialization test
@MockBean              // Mock a bean in test context
@MockitoBean           // Mockito mock bean (Spring Boot 3+)
@SpyBean               // Spy on a bean in test context
@TestConfiguration    // Test-specific configuration
@TestPropertySource     // Test-specific properties
@AutoConfigureTestDatabase // Auto-configure test database
@AutoConfigureMockMvc   // Auto-configure MockMvc
@AutoConfigureWebTestClient // Auto-configure WebTestClient
```

JUnit 5 Testing Annotations

```
@Test                  // Test method
@DisplayName            // Test display name
@ParameterizedTest      // Parameterized test
@RepeatedTest           // Repeated test execution
@Disabled               // Disabled test
@Tag                    // Test tag for filtering
@ExtendWith             // Register extension
@TempDir               // Temporary directory
@TestInstance         // Test instance lifecycle
@BeforeAll             // Before all tests
@AfterAll              // After all tests
@BeforeEach            // Before each test
@AfterEach             // After each test
```

Testing Examples

```
@SpringBootTest
@AutoConfigureMockMvc
```

```
@TestPropertySource(properties = {
    "spring.datasource.url=jdbc:h2:mem:testdb",
    "spring.jpa.hibernate.ddl-auto=create-drop"
})
public class UserControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @MockBean
    private UserService userService;

    @Test
    @DisplayName("Should create user successfully")
    void shouldCreateUser() throws Exception {
        // Given
        UserRequest request = new UserRequest("testuser", "test@example
        User createdUser = new User(1L, "testuser", "test@example.com")

        when(userService.createUser(any(UserRequest.class))).thenReturn

        // When & Then
        mockMvc.perform(post("/api/users")
                .contentType(MediaType.APPLICATION_JSON)
                .content(objectMapper.writeValueAsString(request)))
                .andExpect(status().isCreated())
                .andExpect(jsonPath("$.id", is(1)))
                .andExpect(jsonPath("$.username", is("testuser")));

        verify(userService).createUser(any(UserRequest.class));
    }

    @ParameterizedTest
    @ValueSource(strings = {"user1", "user2", "user3"})
    @DisplayName("Should find user by username")
    void shouldFindUserByUsername(String username) {
        // Given
        User user = new User(1L, username, username + "@example.com");
        when(userService.findByUsername(username)).thenReturn(Optional.

        // When
        Optional<User> result = userService.findByUsername(username);

        // Then
        assertTrue(result.isPresent());
        assertEquals(username, result.get().getUsername());
    }
}

@DataJpaTest
@AutoConfigureTestDatabase(replace = AutoConfigureTestDatabase.Replace.
public class UserRepositoryTest {
```

```
    @Autowired
    private TestEntityManager entityManager;

    @Autowired
    private UserRepository userRepository;

    @Test
    @DisplayName("Should find user by email")
    void shouldFindByEmail() {
        // Given
        User user = new User("testuser", "test@example.com");
        entityManager.persistAndFlush(user);

        // When
        Optional<User> result = userRepository.findByEmail("test@exampl

        // Then
        assertTrue(result.isPresent());
        assertEquals("test@example.com", result.get().getEmail());
    }

    @Test
    @DisplayName("Should count users by status")
    void shouldCountByStatus() {
        // Given
        User user1 = new User("user1", "user1@example.com");
        user1.setStatus(UserStatus.ACTIVE);

        User user2 = new User("user2", "user2@example.com");
        user2.setStatus(UserStatus.ACTIVE);

        User user3 = new User("user3", "user3@example.com");
        user3.setStatus(UserStatus.INACTIVE);

        entityManager.persist(user1);
        entityManager.persist(user2);
        entityManager.persist(user3);
        entityManager.flush();

        // When
        Long activeCount = userRepository.countByStatus(UserStatus.ACTI

        // Then
        assertEquals(2L, activeCount);
    }
}
```

Integration Testing

```
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_P
@TestPropertySource(properties = {
    "spring.cloud.config.enabled=false",
    "spring.cloud.discovery.enabled=false"
```

```
})
public class UserIntegrationTest {

    @Autowired
    private TestRestTemplate restTemplate;

    @Autowired
    private UserRepository userRepository;

    @Test
    @DisplayName("Should create and retrieve user via REST API")
    void shouldCreateAndRetrieveUser() {
        // Given
        UserRequest request = new UserRequest("integrationuser", "integ

        // When - Create user
        ResponseEntity<User> createResponse = restTemplate.postForEntit
            "/api/users", request, User.class);

        // Then - Verify creation
        assertEquals(HttpStatus.CREATED, createResponse.getStatusCode()
        User createdUser = createResponse.getBody();
        assertNotNull(createdUser);
        assertEquals("integrationuser", createdUser.getUsername());

        // When - Retrieve user
        ResponseEntity<User> getResponse = restTemplate.getForEntity(
            "/api/users/" + createdUser.getId(), User.class);

        // Then - Verify retrieval
        assertEquals(HttpStatus.OK, getResponse.getStatusCode());
        User retrievedUser = getResponse.getBody();
        assertNotNull(retrievedUser);
        assertEquals(createdUser.getId(), retrievedUser.getId());
    }
}

---

## 10. Caching Annotations

### Cache Abstraction
```java
@EnableCaching          // Enable caching support
@Cacheable             // Cache method result
@CachePut              // Update cache with method result
@CacheEvict            // Remove entry from cache
@Caching               // Multiple cache operations
@CacheConfig           // Class-level cache configuration
```

Caching Examples

```
@Service
```
```

```
@CacheConfig(cacheNames = "users")
public class UserService {

    @Cacheable(key = "#id")
    public User findById(Long id) {
        // Database lookup
        return userRepository.findById(id);
    }

    @CachePut(key = "#result.id")
    public User updateUser(User user) {
        // Update in database
        return userRepository.save(user);
    }

    @CacheEvict(key = "#id")
    public void deleteUser(Long id) {
        userRepository.deleteById(id);
    }

    @CacheEvict(allEntries = true)
    public void clearCache() {
        // Clear all cache entries
    }

    @Caching(
        cacheable = @Cacheable(key = "#username"),
        evict = @CacheEvict(key = "#result.id")
    )
    public User createOrUpdateUser(String username, User user) {
        // Complex caching logic
        return userRepository.save(user);
    }
}

@Configuration
@EnableCaching
public class CacheConfig {

    @Bean
    public CacheManager cacheManager() {
        SimpleCacheManager cacheManager = new SimpleCacheManager();
        cacheManager.setCaches(Arrays.asList(
            new ConcurrentMapCache("users"),
            new ConcurrentMapCache("products"),
            new ConcurrentMapCache("orders")
        ));
        return cacheManager;
    }

    @Bean
    @ConditionalOnProperty(name = "cache.type", havingValue = "redis")
    public RedisCacheManager redisCacheManager(RedisConnectionFactory c
```

```
        return RedisCacheManager.builder(connectionFactory)
            .cacheDefaults(RedisCacheConfiguration.defaultCacheConfig()
                .entryTtl(Duration.ofMinutes(10)))
            .build();
    }
}

---

## 11. Scheduling & Asynchronous Annotations

### Scheduling
```java
@EnableScheduling        // Enable scheduled tasks
@EnableAsync             // Enable async support
@Scheduled              // Schedule method execution
@Async                  // Async method execution
@Timed                  // Timed execution
@EventListener          // Event listener
@AsyncEventListener      // Async event listener
```

Scheduling Examples

```
@Configuration
@EnableScheduling
@EnableAsync
public class SchedulingConfig {

    @Bean
    public TaskExecutor taskExecutor() {
        ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
        executor.setCorePoolSize(5);
        executor.setMaxPoolSize(10);
        executor.setQueueCapacity(25);
        executor.setThreadNamePrefix("async-");
        executor.initialize();
        return executor;
    }

    @Bean
    public ScheduledExecutorService scheduledExecutorService() {
        return Executors.newScheduledThreadPool(5);
    }
}

@Service
public class ScheduledTasks {

    private final Logger log = LoggerFactory.getLogger(ScheduledTasks.c

    @Scheduled(fixedRate = 5000) // Execute every 5 seconds
    public void reportCurrentTime() {
        log.info("The time is now {}", LocalDateTime.now());
```
```

```
    }

    @Scheduled(fixedDelay = 10000) // Execute 10 seconds after previous
    public void processBatch() {
        log.info("Processing batch job");
        // Batch processing logic
    }

    @Scheduled(cron = "0 0 12 * * MON-FRI") // Weekdays at noon
    public void weekdayNoonTask() {
        log.info("Running weekday noon task");
        // Business logic
    }

    @Scheduled(cron = "${app.backup.cron:0 0 2 * * *}") // Configurable
    public void backupDatabase() {
        log.info("Starting database backup");
        // Backup logic
    }

    @Scheduled(initialDelay = 1000, fixedRate = 5000)
    public void delayedAndRepeatedTask() {
        log.info("Delayed and repeated task");
    }
}

@Service
public class AsyncService {

    private final Logger log = LoggerFactory.getLogger(AsyncService.cla

    @Async("taskExecutor")
    public CompletableFuture<String> processAsync(String input) {
        log.info("Processing {} asynchronously", input);

        try {
            Thread.sleep(2000); // Simulate long-running task
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }

        return CompletableFuture.completedFuture("Processed: " + input)
    }

    @Async
    public void sendEmailAsync(String to, String subject, String body)
        log.info("Sending email to {}", to);
        // Email sending logic
    }

    @Async
    @Retryable(value = {Exception.class}, maxAttempts = 3, backoff = @B
    public void unreliableOperation() {
```

```
        // Operation that might fail
        if (Math.random() > 0.7) {
            throw new RuntimeException("Random failure");
        }
        log.info("Operation succeeded");
    }
}

@Component
public class EventListeners {

    @EventListener
    @Async
    public void handleUserCreated(UserCreatedEvent event) {
        log.info("Handling user created event asynchronously: {}", even
        // Send welcome email, update statistics, etc.
    }

    @EventListener(condition = "#event.priority == 'HIGH'")
    public void handleHighPriorityEvent(PriorityEvent event) {
        log.info("Handling high priority event: {}", event.getMessage()
        // Handle high priority events immediately
    }

    @EventListener
    @TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
    public void handleAfterCommit(DomainEvent event) {
        log.info("Handling event after transaction commit: {}", event.g
        // Handle events only after successful transaction commit
    }
}

---

## 12. Spring Context Annotations

### Application Context Management
```java
@Configuration          // Configuration class
@ComponentScan         // Component scanning
@Import                // Import configuration classes
@PropertySource        // Load properties files
@Profile               // Profile-based bean activation
@ActiveProfiles        // Test profile activation
@ContextConfiguration  // Test context configuration
@TestPropertySource     // Test properties
```

Context Examples

```
@Configuration
@ComponentScan(basePackages = "com.example")
@PropertySource("classpath:application.properties")
@Import({DatabaseConfig.class, SecurityConfig.class})
```
```

```
public class ApplicationConfig {

    @Bean
    @Profile("dev")
    public DataSource devDataSource() {
        return new HikariDataSource();
    }

    @Bean
    @Profile("prod")
    public DataSource prodDataSource() {
        return new HikariDataSource();
    }

    @Bean
    public static PropertySourcesPlaceholderConfigurer propertySourcesP
        return new PropertySourcesPlaceholderConfigurer();
    }
}

@TestConfiguration
@ActiveProfiles("test")
@TestPropertySource(properties = {
    "spring.datasource.url=jdbc:h2:mem:testdb",
    "spring.jpa.hibernate.ddl-auto=create-drop"
})
public class TestConfig {

    @Bean
    @Primary
    public DataSource testDataSource() {
        return new EmbeddedDatabaseBuilder()
            .setType(EmbeddedDatabaseType.H2)
            .build();
    }
}

---

## 13. Spring Boot 4 New Features

### Native Image Support
```java
@NativeHint              // GraalVM native image hints
@RegisterReflectionForBinding // Register for reflection
@RegisterForReflection   // Register class for reflection
```

Observability

```
@Observed               // Micrometer observation
@Timed                  // Timer metric
@Counted                // Counter metric
@Gauge                  // Gauge metric
```
```

Spring Boot 4 Examples

```
@RestController
public class UserController {

    @Observed(name = "user.find", contextualName = "find-user")
    @GetMapping("/api/users/{id}")
    public User getUser(@PathVariable Long id) {
        return userService.findById(id);
    }

    @Timed(name = "user.create", description = "Time taken to create us
    @Counted(name = "user.create.calls", description = "Number of user
    @PostMapping("/api/users")
    public User createUser(@RequestBody UserRequest request) {
        return userService.createUser(request);
    }
}

@Configuration
@RegisterReflectionForBinding({User.class, UserRequest.class})
public class NativeConfiguration {

    @Bean
    @RegisterForReflection
    public ObjectMapper objectMapper() {
        return new ObjectMapper();
    }
}

@Component
public class MetricsComponent {

    private final MeterRegistry meterRegistry;
    private final AtomicLong activeUsers = new AtomicLong(0);

    public MetricsComponent(MeterRegistry meterRegistry) {
        this.meterRegistry = meterRegistry;
        Gauge.builder("users.active", activeUsers, AtomicLong::get)
            .description("Number of active users")
            .register(meterRegistry);
    }

    public void incrementActiveUsers() {
        activeUsers.incrementAndGet();
    }

    public void decrementActiveUsers() {
        activeUsers.decrementAndGet();
    }
}

---
```

14. Common Annotation Combinations

REST Controller Pattern
```java
@RestController
@RequestMapping("/api/users")
@Validated
public class UserController {

    @GetMapping
    @ResponseStatus(HttpStatus.OK)
    public Page<User> getUsers(Pageable pageable) {
        return userService.findAll(pageable);
    }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public User createUser(@Valid @RequestBody UserRequest request) {
        return userService.createUser(request);
    }

    @PutMapping("/{id}")
    @ResponseStatus(HttpStatus.OK)
    public User updateUser(@PathVariable Long id, @Valid @RequestBody U
        return userService.updateUser(id, request);
    }

    @DeleteMapping("/{id}")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    public void deleteUser(@PathVariable Long id) {
        userService.deleteUser(id);
    }
}
```

---

15. Annotation Best Practices

Configuration Organization
- Use `@Configuration` classes for related beans
- Group by functionality (database, security, web, etc.)
- Use `@Profile` for environment-specific configurations
- Apply `@ConditionalOn*` for optional features

Component Stereotypes
- Use specific stereotypes (`@Service`, `@Repository`, `@Controller`) o
- Keep component names descriptive and consistent
- Use `@Qualifier` only when necessary for disambiguation

Dependency Injection
- Prefer constructor injection over field injection
- Use `@Autowired` on constructors (optional in Spring 4.3+)

- Use `@Lazy` for circular dependencies or expensive beans

Transaction Management
- Apply `@Transactional` at service layer, not repository layer
- Specify appropriate propagation and isolation levels
- Use `@ReadOnly` for read-only operations

Security
- Apply method security annotations at service layer
- Use `@PreAuthorize` for complex authorization rules
- Combine with `@Secured` for simple role-based access

Performance
- Use `@Cacheable` for expensive, frequently called methods
- Apply `@Async` for long-running, non-blocking operations
- Use `@Scheduled` for periodic maintenance tasks

Testing
- Use appropriate test slices (`@WebMvcTest`, `@DataJpaTest`)
- Mock external dependencies with `@MockBean`
- Use `@TestPropertySource` for test-specific configuration

Documentation
- Use `@ApiModel` and `@ApiModelProperty` for API documentation
- Apply `@Description` for custom annotations
- Document complex conditional logic in Javadoc

---

7. AOP & Aspect Annotations

Core Aspect Annotations
```java
@Aspect                  # Define aspect class
@Component               # Make aspect a Spring bean
@EnableAspectJAutoProxy # Enable AOP support
@Pointcut               # Define reusable pointcut expression
```

Advice Types

```
@Before                 # Before advice (executes before join point)
@After                  # After advice (executes after join point, final
@AfterReturning         # After successful return advice
@AfterThrowing          # After exception advice
@Around                 # Around advice (wraps join point)
```

Basic Aspect Example

```
@Aspect
@Component
@Slf4j
public class LoggingAspect {
```

```
    @Pointcut("execution(* com.example.service.*.*(..))")
    public void serviceLayer() {}

    @Pointcut("within(com.example.repository..*)")
    public void repositoryLayer() {}

    @Before("serviceLayer()")
    public void logBeforeService(JoinPoint joinPoint) {
        log.info("Executing service method: {}", joinPoint.getSignature
        log.info("Arguments: {}", Arrays.toString(joinPoint.getArgs()))
    }

    @AfterReturning(pointcut = "serviceLayer()", returning = "result")
    public void logAfterService(JoinPoint joinPoint, Object result) {
        log.info("Service method {} returned: {}",
                joinPoint.getSignature().toShortString(), result);
    }

    @AfterThrowing(pointcut = "serviceLayer()", throwing = "exception")
    public void logServiceException(JoinPoint joinPoint, Exception exce
        log.error("Exception in {}: {}",
                joinPoint.getSignature().toShortString(), exception.get
    }
}
```

Pointcut Expressions

```
// Execution patterns
@Pointcut("execution(public * com.example.service.*.*(..))")
public void publicServiceMethods() {}

@Pointcut("execution(* com.example.repository.*.*(..))")
public void repositoryMethods() {}

@Pointcut("execution(* com.example..*.*(..))")
public void allMethodsInPackage() {}

// Within patterns
@Pointcut("within(com.example.service.*)")
public void withinServicePackage() {}

@Pointcut("within(@com.example.annotation.Cacheable *)")
public void cacheableClasses() {}

// Annotation patterns
@Pointcut("@annotation(com.example.annotation.Loggable)")
public void loggableMethods() {}

@Pointcut("@target(com.example.annotation.Repository)")
public void repositoryClasses() {}

@Pointcut("@args(com.example.annotation.Validated)")
public void methodsWithValidatedArgs() {}
```

```
// Bean patterns
@Pointcut("bean(*Service)")
public void serviceBeans() {}

@Pointcut("!bean(*Controller)")
public void nonControllerBeans() {}

// Combined pointcuts
@Pointcut("serviceLayer() && @annotation(org.springframework.transactio
public void transactionalServiceMethods() {}

@Pointcut("repositoryLayer() || serviceLayer())")
public void dataAccessLayer() {}
```

Around Advice Examples

```
@Aspect
@Component
@Slf4j
public class PerformanceAspect {

    @Around("serviceLayer()")
    public Object measureExecutionTime(ProceedingJoinPoint joinPoint) t
        long startTime = System.currentTimeMillis();
        String methodName = joinPoint.getSignature().toShortString();

        try {
            Object result = joinPoint.proceed();
            long duration = System.currentTimeMillis() - startTime;
            log.info("Method {} executed in {} ms", methodName, duratio

            // Alert for slow methods
            if (duration > 1000) {
                log.warn("SLOW METHOD DETECTED: {} took {} ms", methodN
            }

            return result;
        } catch (Exception e) {
            long duration = System.currentTimeMillis() - startTime;
            log.error("Method {} failed after {} ms: {}", methodName, d
            throw e;
        }
    }

    @Around("execution(* com.example.service.*.*(..)) && args(userId, .
    public Object validateUserAccess(ProceedingJoinPoint joinPoint, Lon
        // Validate user access before proceeding
        if (!securityService.hasAccess(getCurrentUser(), userId)) {
            throw new AccessDeniedException("Access denied for user " +
        }

        return joinPoint.proceed();
```

```
    }
}

---

## 16. Configuration for AOP
        log.info("Cached result for key: {}", key);

        return result;
    }

    private String generateKey(ProceedingJoinPoint joinPoint, String cu
        if (!customKey.isEmpty()) {
            return customKey;
        }
        return joinPoint.getSignature().toShortString() + Arrays.toStri
    }

    private static class CacheEntry {
        private final Object value;
        private final long expiryTime;

        CacheEntry(Object value, long expiryTime) {
            this.value = value;
            this.expiryTime = expiryTime;
        }

        Object getValue() { return value; }
        boolean isExpired() { return System.currentTimeMillis() > expir
    }
}
```

Security Aspect

```
@Aspect
@Component
public class SecurityAspect {

    @Around("@annotation(requiresRole)")
    public Object checkRole(ProceedingJoinPoint joinPoint, RequiresRole
        Authentication auth = SecurityContextHolder.getContext().getAut

        if (auth == null || !auth.isAuthenticated()) {
            throw new AuthenticationException("User not authenticated")
        }

        boolean hasRequiredRole = auth.getAuthorities().stream()
            .anyMatch(authority -> authority.getAuthority().equals("ROL

        if (!hasRequiredRole) {
            throw new AccessDeniedException("Required role: " + require
        }
```

```
        return joinPoint.proceed();
    }

    @Around("execution(* com.example.service.admin.*.*(..))")
    public Object requireAdminAccess(ProceedingJoinPoint joinPoint) thr
        if (!securityService.isAdmin(getCurrentUser())) {
            throw new AccessDeniedException("Admin access required");
        }
        return joinPoint.proceed();
    }
}
```

Transaction Aspect

```
@Aspect
@Component
public class TransactionAspect {

    @Around("@annotation(transactional)")
    public Object manageTransaction(ProceedingJoinPoint joinPoint, Tran
        TransactionStatus status = transactionManager.getTransaction(ne

        try {
            Object result = joinPoint.proceed();
            transactionManager.commit(status);
            return result;
        } catch (Exception e) {
            if (transactional.rollbackFor().length > 0) {
                for (Class<? extends Exception> rollbackFor : transacti
                    if (rollbackFor.isInstance(e)) {
                        transactionManager.rollback(status);
                        throw e;
                    }
                }
            }
            transactionManager.commit(status);
            throw e;
        }
    }
}
```

Retry Aspect

```
@Aspect
@Component
public class RetryAspect {

    @Around("@annotation(retry)")
    public Object retryMethod(ProceedingJoinPoint joinPoint, Retry retr
        int attempts = 0;
        Exception lastException = null;

        while (attempts < retry.maxAttempts()) {
```

```
            try {
                return joinPoint.proceed();
            } catch (Exception e) {
                attempts++;
                lastException = e;

                if (attempts >= retry.maxAttempts()) {
                    break;
                }

                log.warn("Attempt {} failed for method {}, retrying..."
                        attempts, joinPoint.getSignature().toShortStrin

                Thread.sleep(retry.delayMs());
            }
        }

        log.error("Method {} failed after {} attempts",
                joinPoint.getSignature().toShortString(), attempts);
        throw lastException;
    }
}
```

Audit Aspect

```
@Aspect
@Component
public class AuditAspect {

    @AfterReturning(pointcut = "@annotation(auditable)", returning = "r
    public void auditSuccessfulOperation(JoinPoint joinPoint, Auditable
        AuditEvent event = AuditEvent.builder()
            .operation(auditable.operation())
            .target(getTarget(joinPoint))
            .user(getCurrentUser())
            .timestamp(Instant.now())
            .success(true)
            .result(result)
            .build();

        auditService.logEvent(event);
    }

    @AfterThrowing(pointcut = "@annotation(auditable)", throwing = "exc
    public void auditFailedOperation(JoinPoint joinPoint, Auditable aud
        AuditEvent event = AuditEvent.builder()
            .operation(auditable.operation())
            .target(getTarget(joinPoint))
            .user(getCurrentUser())
            .timestamp(Instant.now())
            .success(false)
            .error(exception.getMessage())
            .build();
```

```
        auditService.logEvent(event);
    }

    private String getTarget(JoinPoint joinPoint) {
        Object[] args = joinPoint.getArgs();
        return args.length > 0 ? args[0].toString() : "unknown";
    }

    private String getCurrentUser() {
        return SecurityContextHolder.getContext().getAuthentication().g
    }
}
```

Configuration for AOP

```
@Configuration
@EnableAspectJAutoProxy(
    proxyTargetClass = true,  // Use CGLIB proxies
    exposeProxy = true        // Expose proxy to target
)
public class AopConfig {

    @Bean
    public LoggingAspect loggingAspect() {
        return new LoggingAspect();
    }

    @Bean
    public PerformanceAspect performanceAspect() {
        return new PerformanceAspect();
    }
}
```

Advanced Pointcut Combinations

```
@Aspect
@Component
public class ComplexPointcutsAspect {

    // Complex combinations
    @Pointcut("serviceLayer() && @annotation(org.springframework.transa
    public void transactionalServices() {}

    @Pointcut("repositoryLayer() && !@annotation(org.springframework.tr
    public void nonTransactionalRepositories() {}

    @Pointcut("execution(public * *(..)) && @annotation(com.example.ann
    public void publicSecuredMethods() {}

    @Pointcut("bean(*Service) && execution(* save*(..))")
    public void serviceSaveMethods() {}
```

```
    // Parameter-based pointcuts
    @Pointcut("execution(* com.example.service.*.*(Long, ..)) && args(u
    public void methodsWithUserIdParameter(Long userId) {}

    @Pointcut("@args(com.example.annotation.Validated)")
    public void methodsWithValidatedParameters() {}

    // Annotation on target
    @Pointcut("@target(org.springframework.stereotype.Repository)")
    public void repositoryClasses() {}

    // Annotation on parameter
    @Pointcut("execution(* com.example.service.*.*(@com.example.annotat
    public void methodsWithValidatedFirstParameter() {}
}
```

AOP Best Practices

- **Use @Around sparingly** - Prefer specific advice types when possible
- **Keep aspects focused** - Single responsibility per aspect
- **Use descriptive pointcut names** - Improve readability
- **Consider performance impact** - AOP adds overhead
- **Test aspects thoroughly** - Complex interactions can be hard to debug
- **Use appropriate proxy settings** - CGLIB vs JDK proxies
- **Handle exceptions properly** - Don't swallow exceptions in advice
- **Log appropriate level** - Avoid excessive logging in production

---

17. AspectJ Annotations

AspectJ Core Annotations

```
@Aspect                  // Define aspect class (Spring AOP)
@DeclareParents          // Introduce new interface to target types
@DeclareMixin            // Introduce new mixin implementation
@Around                  // Around advice
@Before                  // Before advice
@After                   // After advice (finally)
@AfterReturning          // After successful return advice
@AfterThrowing           // After exception advice
@Pointcut               // Define pointcut expression
@SuppressWarnings        // Suppress AspectJ warnings
```

Native AspectJ Syntax vs Annotations

```
// Native AspectJ Syntax (.aj file)
public aspect LoggingAspect {

    pointcut serviceExecution(): execution(* com.example.service.*.*(..

    before(): serviceExecution() {
        System.out.println("Before: " + thisJoinPoint.getSignature());
```

```
    }

    after() returning(Object result): serviceExecution() {
        System.out.println("After returning: " + result);
    }

    after() throwing(Exception ex): serviceExecution() {
        System.out.println("After throwing: " + ex.getMessage());
    }

    Object around(): serviceExecution() {
        long start = System.currentTimeMillis();
        try {
            return proceed();
        } finally {
            long duration = System.currentTimeMillis() - start;
            System.out.println("Execution time: " + duration + "ms");
        }
    }
}

// Equivalent using Spring AOP Annotations
@Aspect
@Component
public class LoggingAspect {

    @Pointcut("execution(* com.example.service.*.*(..))")
    public void serviceExecution() {}

    @Before("serviceExecution()")
    public void beforeService(JoinPoint joinPoint) {
        System.out.println("Before: " + joinPoint.getSignature());
    }

    @AfterReturning(pointcut = "serviceExecution()", returning = "resul
    public void afterReturningService(JoinPoint joinPoint, Object resul
        System.out.println("After returning: " + result);
    }

    @AfterThrowing(pointcut = "serviceExecution()", throwing = "ex")
    public void afterThrowingService(JoinPoint joinPoint, Exception ex)
        System.out.println("After throwing: " + ex.getMessage());
    }

    @Around("serviceExecution()")
    public Object aroundService(ProceedingJoinPoint joinPoint) throws T
        long start = System.currentTimeMillis();
        try {
            return joinPoint.proceed();
        } finally {
            long duration = System.currentTimeMillis() - start;
            System.out.println("Execution time: " + duration + "ms");
        }
```

```
    }
}
```

AspectJ Pointcut Designators

```
// Execution pointcuts
execution(* com.example.service.*.*(..))                        // All ser
execution(public * com.example.controller.*.*(..))               // Public
execution(* com.example.repository.*.find*(..))                 // Reposit

// Within pointcuts
within(com.example.service.*)                                   // All cla
within(@org.springframework.stereotype.Repository *)            // All rep

// Target pointcuts
target(com.example.service.UserService)                          // Target
target(java.io.Serializable)                                    // Target

// This pointcuts
this(com.example.service.UserService)                           // Proxy i
this(java.io.Serializable)                                     // Proxy im

// Args pointcuts
args(java.lang.String)                                          // Single
args(java.lang.String, java.lang.Integer)                      // String a
(@org.springframework.validation.Valid *)                      // Any @Val

// @annotation pointcuts
@annotation(org.springframework.transaction.annotation.Transactional) /

// @within pointcuts
@within(org.springframework.stereotype.Service)                 // Classes

// @target pointcuts
@target(org.springframework.stereotype.Repository)               // Target

// @args pointcuts
@args(com.example.annotation.Validated)                         // Argumen

// Bean pointcuts (Spring-specific)
bean(*Service)                                                  // Beans e
bean(userRepository)                                            // Specifi

// Reference pointcuts
cflow(execution(* com.example.service.*.*(..)))                // Control
cflowbelow(execution(* com.example.service.*.*(..)))           // Control

// Handler pointcuts
handler(java.lang.Exception)                                    // Excepti

// Initialization pointcuts
initialization(com.example.service.*.new(..))                  // Construc
preinitialization(com.example.service.*.new(..))               // Pre-init
```

```
// Static initialization pointcuts
staticinitialization(com.example.service.*)                    // Static
```

AspectJ Introduction (DeclareParents)

```
@Aspect
@Component
public class IntroductionAspect {

    // Introduce Auditable interface to all repository classes
    @DeclareParents(
        value = "com.example.repository.*+",
        defaultImpl = DefaultAuditable.class
    )
    public static Auditable auditable;

    // Introduce Cacheable interface to all service classes
    @DeclareParents(
        value = "com.example.service.*+",
        defaultImpl = DefaultCacheable.class
    )
    public static Cacheable cacheable;
}

// Interface to introduce
public interface Auditable {
    void setCreatedBy(String createdBy);
    String getCreatedBy();
    void setCreatedDate(Date createdDate);
    Date getCreatedDate();
}

// Default implementation
public class DefaultAuditable implements Auditable {
    private String createdBy;
    private Date createdDate;

    @Override
    public void setCreatedBy(String createdBy) {
        this.createdBy = createdBy;
    }

    @Override
    public String getCreatedBy() {
        return createdBy;
    }

    @Override
    public void setCreatedDate(Date createdDate) {
        this.createdDate = createdDate;
    }
```

```
    @Override
    public Date getCreatedDate() {
        return createdDate;
    }
}

// Usage
@Service
public class UserService implements UserServiceInterface {
    // Now has Auditable methods available through introduction
}
```

AspectJ Mixin (DeclareMixin)

```
@Aspect
@Component
public class MixinAspect {

    // Introduce timestamp mixin to all entities
    @DeclareMixin(
        value = "com.example.entity.*",
        interfaces = TimestampMixin.class
    )
    public static TimestampMixin createTimestampMixin() {
        return new TimestampMixinImpl();
    }
}

// Mixin interface
public interface TimestampMixin {
    void setCreatedAt(LocalDateTime createdAt);
    LocalDateTime getCreatedAt();
    void setUpdatedAt(LocalDateTime updatedAt);
    LocalDateTime getUpdatedAt();
}

// Mixin implementation
public class TimestampMixinImpl implements TimestampMixin {
    private LocalDateTime createdAt;
    private LocalDateTime updatedAt;

    @Override
    public void setCreatedAt(LocalDateTime createdAt) {
        this.createdAt = createdAt;
    }

    @Override
    public LocalDateTime getCreatedAt() {
        return createdAt;
    }

    @Override
```

```
    public void setUpdatedAt(LocalDateTime updatedAt) {
        this.updatedAt = updatedAt;
    }

    @Override
    public LocalDateTime getUpdatedAt() {
        return updatedAt;
    }
}
```

AspectJ Compile-Time Weaving Configuration

```
<!-- Maven configuration for AspectJ compile-time weaving -->
<dependencies>
    <dependency>
        <groupId>org.aspectj</groupId>
        <artifactId>aspectjrt</artifactId>
        <version>1.9.19</version>
    </dependency>
    <dependency>
        <groupId>org.aspectj</groupId>
        <artifactId>aspectjweaver</artifactId>
        <version>1.9.19</version>
    </dependency>
    <dependency>
        <groupId>org.aspectj</groupId>
        <artifactId>aspectjtools</artifactId>
        <version>1.9.19</version>
    </dependency>
</dependencies>

<build>
    <plugins>
        <plugin>
            <groupId>org.codehaus.mojo</groupId>
            <artifactId>aspectj-maven-plugin</artifactId>
            <version>1.14.0</version>
            <configuration>
                <complianceLevel>11</complianceLevel>
                <source>11</source>
                <target>11</target>
                <showWeaveInfo>true</showWeaveInfo>
                <verbose>true</verbose>
                <Xlint>ignore</Xlint>
                <encoding>UTF-8</encoding>
            </configuration>
            <executions>
                <execution>
                    <goals>
                        <goal>compile</goal>
                        <goal>test-compile</goal>
                    </goals>
                </execution>
```

```
            </executions>
        </plugin>
    </plugins>
</build>
```

AspectJ Load-Time Weaving Configuration

```
@Configuration
@EnableLoadTimeWeaving
public class AspectJConfig {

    @Bean
    public LoadTimeWeaver loadTimeWeaver() {
        return new InstrumentationLoadTimeWeaver();
    }

    @Bean
    public DefaultPointcutAdvisor defaultPointcutAdvisor() {
        AspectJExpressionPointcut pointcut = new AspectJExpressionPoint
        pointcut.setExpression("execution(* com.example.service.*.*(..)

        DefaultPointcutAdvisor advisor = new DefaultPointcutAdvisor();
        advisor.setPointcut(pointcut);
        advisor.setAdvice(new MethodInterceptor() {
            @Override
            public Object invoke(MethodInvocation invocation) throws Th
                // Advice logic
                return invocation.proceed();
            }
        });

        return advisor;
    }
}

// JVM argument for load-time weaving
// -javaagent:path/to/aspectjweaver.jar
```

Advanced AspectJ Examples

```
@Aspect
@Component
public class AdvancedAspectJExample {

    // Complex pointcut with multiple designators
    @Pointcut(
        "execution(public * com.example.service.*.*(..)) && " +
        "@annotation(org.springframework.transaction.annotation.Transac
        "args(com.example.dto.RequestDTO) && " +
        "target(com.example.service.BaseService)"
    )
    public void transactionalServiceMethods() {}
```

```
// Pointcut with annotation on parameter
@Pointcut("execution(* com.example.controller.*.*(..)) && @args(com
public void controllerMethodsWithValidatedArgs() {}

// Pointcut with control flow
@Pointcut("cflow(execution(* com.example.service.*.*(..))) && execu
public void repositoryMethodsCalledFromService() {}

// Pointcut with within and execution
@Pointcut("within(com.example.service.*) && execution(* save*(..))"
public void saveMethodsInServicePackage() {}

// Around advice with context information
@Around("transactionalServiceMethods()")
public Object aroundTransactionalService(ProceedingJoinPoint joinPo
    String methodName = joinPoint.getSignature().getName();
    Object[] args = joinPoint.getArgs();
    Object target = joinPoint.getTarget();

    log.info("Starting transactional method: {} on target: {} with
            methodName, target.getClass().getSimpleName(), Arrays.t

    try {
        Object result = joinPoint.proceed();
        log.info("Successfully completed transactional method: {}",
        return result;
    } catch (Exception e) {
        log.error("Exception in transactional method {}: {}", metho
        throw e;
    }
}

// Before advice with argument binding
@Before("controllerMethodsWithValidatedArgs() && @annotation(valid)
public void beforeValidatedMethod(JoinPoint joinPoint, Valid valid)
    log.info("Validating method {} with annotation: {}",
            joinPoint.getSignature().getName(), valid.message());
}

// After returning advice with result binding
@AfterReturning(
    pointcut = "execution(* com.example.repository.*.findById(..))"
    returning = "result"
)
public void afterFindById(JoinPoint joinPoint, Object result) {
    if (result != null) {
        log.info("Found entity: {} with method: {}",
                result, joinPoint.getSignature().getName());
    } else {
        log.warn("No entity found with method: {}", joinPoint.getSi
    }
}
```

```
    // After throwing advice with exception binding
    @AfterThrowing(
        pointcut = "execution(* com.example.service.*.*(..))",
        throwing = "exception"
    )
    public void afterServiceException(JoinPoint joinPoint, Exception ex
        log.error("Exception in service method {}: {}",
                joinPoint.getSignature().getName(), exception.getMessag

        // Send error notification
        notificationService.sendErrorNotification(exception);
    }
}
```

AspectJ vs Spring AOP Comparison

```
// Spring AOP (Proxy-based, limited to method execution)
@Aspect
@Component
public class SpringAopAspect {

    @Before("execution(* com.example.service.*.*(..))")
    public void beforeService(JoinPoint joinPoint) {
        // Only works on public methods of Spring beans
        log.info("Spring AOP: Before {}", joinPoint.getSignature());
    }
}

// AspectJ (Full weaving, all join points)
public aspect AspectJAspect {

    before(): execution(* com.example.service.*.*(..)) {
        // Works on all methods, private, protected, etc.
        System.out.println("AspectJ: Before " + thisJoinPoint.getSignat
    }

    // AspectJ-specific join points not available in Spring AOP
    before(): get(* com.example.service.*.*) {
        // Field access join point
        System.out.println("Field access: " + thisJoinPoint);
    }

    before(): handler(java.lang.Exception+) {
        // Exception handler join point
        System.out.println("Exception handler: " + thisJoinPoint);
    }

    before(): initialization(com.example.service.*.new(..)) {
        // Constructor initialization join point
        System.out.println("Constructor initialization: " + thisJoinPoi
    }
}
```

AspectJ Performance Considerations

```
@Aspect
@Component
public class PerformanceAspect {

    // Use static final pointcuts for better performance
    private static final Pointcut SERVICE_POINTCUT =
        PointcutUtils.parsePointcutExpression("execution(* com.example.

    // Cache frequently used join point information
    private final ConcurrentHashMap<String, Long> methodTimings = new C

    @Around("execution(* com.example.service.*.*(..))")
    public Object measurePerformance(ProceedingJoinPoint joinPoint) thr
        String methodSignature = joinPoint.getSignature().toShortString

        // Use System.nanoTime() for more precise timing
        long startTime = System.nanoTime();

        try {
            return joinPoint.proceed();
        } finally {
            long duration = System.nanoTime() - startTime;

            // Update timing statistics
            methodTimings.merge(methodSignature, duration, Long::sum);

            // Log only if performance threshold exceeded
            if (duration > 1_000_000) { // 1ms
                log.warn("Slow method detected: {} took {} ns", methodS
            }
        }
    }

    // Use @Transactional for transaction management instead of custom
    @Transactional
    public void transactionalMethod() {
        // Business logic
    }
}
```

AspectJ Best Practices

- **Choose the right weaving approach** - Compile-time for performance, load-time for flexibility
- **Use specific pointcuts** - Avoid overly broad pointcuts that impact performance
- **Cache pointcut expressions** - Reuse static final pointcuts for better performance
- **Minimize aspect complexity** - Keep aspects focused and simple
- **Test aspects thoroughly** - Use integration tests to verify aspect behavior
- **Consider Spring AOP first** - Use Spring AOP for simple cases, AspectJ for complex requirements
- **Monitor performance** - Track aspect overhead in production

- **Use appropriate join points** - Choose the most specific join point type for your needs

Aspect Definition

```
@Aspect                 // Define aspect class
@Component              // Make it a Spring bean
@Pointcut              // Define pointcut expression
@Before                // Before advice
@After                 // After advice (finally)
@AfterReturning        // After successful return
@AfterThrowing         // After exception thrown
@Around                // Around advice
```

```
@Aspect
@Component
@Slf4j
public class LoggingAspect {

    @Pointcut("execution(* com.example.service.*.*(..))")
    public void serviceLayer() {}

    @Before("serviceLayer()")
    public void logBefore(JoinPoint joinPoint) {
        log.info("Executing: {}", joinPoint.getSignature().toShortStrin
    }

    @AfterReturning(pointcut = "serviceLayer()", returning = "result")
    public void logAfterReturning(JoinPoint joinPoint, Object result) {
        log.info("Method {} returned: {}", joinPoint.getSignature().toS
    }

    @Around("serviceLayer()")
    public Object logExecutionTime(ProceedingJoinPoint joinPoint) throw
        long start = System.currentTimeMillis();
        Object result = joinPoint.proceed();
        long duration = System.currentTimeMillis() - start;
        log.info("Method {} executed in {} ms", joinPoint.getSignature(
        return result;
    }
}
```

---

8. Spring Security Annotations

Spring Security Core Configuration

```
@EnableWebSecurity              // Enable Spring Security web security
@EnableMethodSecurity            // Enable method-level security (Sprin
@EnableGlobalMethodSecurity     // Legacy method security (pre-Spring S
@Configuration                   // Security configuration class
@EnableWebMvcSecurity           // Enable MVC security integration
```

Authentication & Authorization

```
@PreAuthorize                    // Pre-invocation authorization check
@PostAuthorize                   // Post-invocation authorization check
@Secured                         // Role-based security (legacy)
@RolesAllowed                    // JSR-250 role-based security
@PreFilter                       // Filter collection before method exec
@PostFilter                      // Filter collection after method execu
@AuthenticationPrincipal         // Inject current authenticated user
@CurrentSecurityContext          // Inject security context
@WithMockUser                    // Test security context
@WithUserDetails                 // Test with custom UserDetails
```

Method Security Examples

```
@RestController
@RequestMapping("/api/v1")
public class UserController {

    @GetMapping("/public/users")
    public List<User> getPublicUsers() {
        // Public endpoint - no security annotation needed
        return userService.findPublicUsers();
    }

    @GetMapping("/users")
    @PreAuthorize("hasRole('USER')")
    public List<User> getAllUsers() {
        return userService.findAll();
    }

    @GetMapping("/admin/users")
    @PreAuthorize("hasRole('ADMIN') and hasAuthority('USER_READ')")
    public List<User> getAdminUsers() {
        return userService.findAllWithDetails();
    }

    @GetMapping("/users/{id}")
    @PreAuthorize("hasRole('USER') and @userService.isOwner")
    public ResponseEntity<User> getUser(@PathVariable Long id) {
        return userService.findById(id)
            .map(ResponseEntity::ok)
            .orElse(ResponseEntity.notFound().build());
    }

    @PostMapping("/users")
    @PreAuthorize("hasAuthority('USER_CREATE')")
    public User createUser(@Valid @RequestBody CreateUserRequest reques
        return userService.create(request);
    }

    @PutMapping("/users/{id}")
    @PreAuthorize("hasRole('USER') and (hasAuthority('USER_UPDATE') or
```

```
    public User updateUser(@PathVariable Long id, @Valid @RequestBody U
        return userService.update(id, request);
    }

    @DeleteMapping("/users/{id}")
    @PreAuthorize("hasRole('ADMIN') and hasAuthority('USER_DELETE')")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    public void deleteUser(@PathVariable Long id) {
        userService.deleteById(id);
    }

    @GetMapping("/users/search")
    @PreFilter("filterObject.department == authentication.user.departme
    public List<User> searchUsers(@RequestBody List<User> users) {
        // Filter users based on department before processing
        return userService.search(users);
    }

    @GetMapping("/users/results")
    @PostFilter("filterObject.active == true")
    public List<User> getActiveUsers() {
        // Filter results to only active users after method execution
        return userService.findAll();
    }

    @GetMapping("/profile")
    public User getCurrentUserProfile(@AuthenticationPrincipal UserDeta
        return userService.findByUsername(userDetails.getUsername());
    }

    @GetMapping("/admin/dashboard")
    @PostAuthorize("returnObject.owner == authentication.name")
    public Dashboard getAdminDashboard() {
        return dashboardService.getAdminDashboard();
    }
}
```

Custom Security Annotations

```
@Target(ElementType.METHOD)
@Retention(RetentionPolicy.RUNTIME)
@PreAuthorize("hasPermission")
public @interface CanReadUser {
}

@Target(ElementType.METHOD)
@Retention(RetentionPolicy.RUNTIME)
@PreAuthorize("hasPermission")
public @interface CanWriteUser {
}

@Target(ElementType.METHOD)
@Retention(RetentionPolicy.RUNTIME)
```

```
@PreAuthorize("@securityService.hasAccessToResource")
public @interface ResourceAccess {
    String resourceId() default "";
}

// Usage
@RestController
public class SecureUserController {

    @GetMapping("/users/{id}")
    @CanReadUser
    public User getUser(@PathVariable Long id) {
        return userService.findById(id);
    }

    @PutMapping("/users/{id}")
    @CanWriteUser
    public User updateUser(@PathVariable Long id, @RequestBody User use
        return userService.update(id, user);
    }
}
```

Security Configuration (Spring Security 6+)

```
@Configuration
@EnableWebSecurity
@EnableMethodSecurity(prePostEnabled = true, securedEnabled = true, jsr
public class SecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Ex
        http
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/public/**", "/actuator/health").
                .requestMatchers("/api/admin/**").hasRole("ADMIN")
                .requestMatchers("/api/user/**").hasAnyRole("USER", "AD
                .requestMatchers(HttpMethod.GET, "/api/users/**").hasAu
                .requestMatchers(HttpMethod.POST, "/api/users/**").hasA
                .requestMatchers(HttpMethod.PUT, "/api/users/**").hasAu
                .requestMatchers(HttpMethod.DELETE, "/api/users/**").ha
                .anyRequest().authenticated()
            )
            .sessionManagement(session -> session
                .sessionCreationPolicy(SessionCreationPolicy.IF_REQUIRE
                .maximumSessions(1)
                .maxSessionsPreventsLogin(false)
            )
            .formLogin(form -> form
                .loginPage("/login")
                .defaultSuccessUrl("/dashboard")
                .failureUrl("/login?error=true")
                .permitAll()
            )
```

```
            .logout(logout -> logout
                .logoutUrl("/logout")
                .logoutSuccessUrl("/login")
                .invalidateHttpSession(true)
                .deleteCookies("JSESSIONID")
                .permitAll()
            )
            .rememberMe(remember -> remember
                .key("uniqueAndSecret")
                .tokenValiditySeconds(86400)
                .rememberMeCookieName("remember-me")
            )
            .csrf(csrf -> csrf
                .csrfTokenRepository(CookieCsrfTokenRepository.withHttp
                .ignoringRequestMatchers("/api/public/**")
            )
            .headers(headers -> headers
                .frameOptions().deny()
                .contentTypeOptions().and()
                .httpStrictTransportSecurity(hstsConfig -> hstsConfig
                    .includeSubDomains(true)
                    .maxAgeInSeconds(31536000)
                )
            );

        return http.build();
    }

    @Bean
    public UserDetailsService userDetailsService(UserRepository userRep
        return username -> {
            User user = userRepository.findByUsername(username)
                .orElseThrow(() -> new UsernameNotFoundException("User

            return org.springframework.security.core.userdetails.User.b
                .username(user.getUsername())
                .password(user.getPassword())
                .roles(user.getRoles().toArray(new String[0]))
                .build();
        };
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
    public AuthenticationManager authenticationManager(
            AuthenticationConfiguration authConfig) throws Exception {
        return authConfig.getAuthenticationManager();
    }
}
```

OAuth2 & JWT Configuration

```
@Configuration
@EnableWebSecurity
@EnableMethodSecurity(prePostEnabled = true)
public class OAuth2SecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Ex
        http
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/public/**").permitAll()
                .requestMatchers("/api/admin/**").hasRole("ADMIN")
                .anyRequest().authenticated()
            )
            .oauth2Login(oauth2 -> oauth2
                .loginPage("/oauth2/authorization/google")
                .defaultSuccessUrl("/dashboard")
                .failureUrl("/login?error=true")
            )
            .oauth2ResourceServer(oauth2 -> oauth2
                .jwt(jwt -> jwt.jwtDecoder(jwtDecoder()))
            )
            .sessionManagement(session -> session
                .sessionCreationPolicy(SessionCreationPolicy.STATELESS)
            );

        return http.build();
    }

    @Bean
    public JwtDecoder jwtDecoder() {
        return NimbusJwtDecoder.withJwkSetUri(jwkSetUri()).build();
    }

    @Value("${spring.security.oauth2.resourceserver.jwt.jwk-set-uri}")
    private String jwkSetUri;
}
```

Custom Permission Evaluation

```
@Component
public class CustomPermissionEvaluator implements PermissionEvaluator {

    @Override
    public boolean hasPermission(Authentication authentication, Object
        if (authentication == null || !authentication.isAuthenticated()
            return false;
        }

        String username = authentication.getName();
        String permissionStr = permission.toString();
```

```
        if (targetDomainObject instanceof User) {
            User user = (User) targetDomainObject;
            return checkUserPermission(username, user, permissionStr);
        }

        return false;
    }

    @Override
    public boolean hasPermission(Authentication authentication, Seriali
        if (authentication == null || !authentication.isAuthenticated()
            return false;
        }

        String username = authentication.getName();
        String permissionStr = permission.toString();

        if ("USER".equals(targetType)) {
            return checkUserPermissionById(username, (Long) targetId, p
        }

        return false;
    }

    private boolean checkUserPermission(String username, User user, Str
        // Custom permission logic
        if ("READ".equals(permission)) {
            return user.getUsername().equals(username) ||
                   authentication.getAuthorities().contains(new SimpleG
        }

        if ("WRITE".equals(permission)) {
            return user.getUsername().equals(username);
        }

        return false;
    }

    private boolean checkUserPermissionById(String username, Long userI
        // Load user and check permission
        User user = userService.findById(userId).orElse(null);
        return user != null && checkUserPermission(username, user, perm
    }
}
```

Role Hierarchy Configuration

```
@Configuration
@EnableWebSecurity
public class RoleHierarchyConfig {

    @Bean
    public RoleHierarchy roleHierarchy() {
```

```
        RoleHierarchyImpl roleHierarchy = new RoleHierarchyImpl();
        String hierarchy = "ROLE_ADMIN > ROLE_MANAGER > ROLE_USER > ROL
        roleHierarchy.setHierarchy(hierarchy);
        return roleHierarchy;
    }

    @Bean
    public MethodSecurityExpressionHandler methodSecurityExpressionHand
        DefaultMethodSecurityExpressionHandler handler = new DefaultMet
        handler.setRoleHierarchy(roleHierarchy);
        return handler;
    }
}
```

Security Context Management

```
@Service
public class SecurityContextService {

    public String getCurrentUsername() {
        Authentication authentication = SecurityContextHolder.getContex
        return authentication != null ? authentication.getName() : null
    }

    public User getCurrentUser() {
        Authentication authentication = SecurityContextHolder.getContex
        if (authentication != null && authentication.isAuthenticated())
            return userService.findByUsername(authentication.getName())
        }
        return null;
    }

    public boolean hasRole(String role) {
        Authentication authentication = SecurityContextHolder.getContex
        return authentication != null &&
               authentication.getAuthorities().stream()
                   .anyMatch(auth -> auth.getAuthority().equals("ROLE_"
    }

    public boolean hasAuthority(String authority) {
        Authentication authentication = SecurityContextHolder.getContex
        return authentication != null &&
               authentication.getAuthorities().stream()
                   .anyMatch(auth -> auth.getAuthority().equals(authori
    }

    @PreAuthorize("#username == authentication.name")
    public User getUserProfile(String username) {
        return userService.findByUsername(username);
    }

    @PreAuthorize("hasRole('ADMIN') or #username == authentication.name
    public void updateUserProfile(String username, UpdateProfileRequest
```

```
        userService.updateProfile(username, request);
    }
}
```

Test Security Annotations

```
@SpringBootTest
@AutoConfigureMockMvc
class UserControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @Test
    @WithMockUser(username = "user", roles = {"USER"})
    void testGetUserWithUserRole() throws Exception {
        mockMvc.perform(get("/api/users/1"))
            .andExpect(status().isOk());
    }

    @Test
    @WithMockUser(username = "admin", roles = {"ADMIN"})
    void testGetAllUsersWithAdminRole() throws Exception {
        mockMvc.perform(get("/api/admin/users"))
            .andExpect(status().isOk());
    }

    @Test
    @WithUserDetails("testuser")
    void testGetCurrentUser() throws Exception {
        mockMvc.perform(get("/api/profile"))
            .andExpect(status().isOk());
    }

    @Test
    @WithMockUser(authorities = {"USER_READ"})
    void testUserWithSpecificAuthority() throws Exception {
        mockMvc.perform(get("/api/users/1"))
            .andExpect(status().isOk());
    }
}
```

Security Best Practices

- **Principle of Least Privilege** - Grant minimum necessary permissions
- **Defense in Depth** - Apply security at multiple layers
- **Secure by Default** - Deny by default, allow explicitly
- **Input Validation** - Validate all inputs at security boundaries
- **Secure Password Storage** - Use strong hashing (BCrypt, Argon2)
- **HTTPS Everywhere** - Enforce HTTPS for all communications
- **CSRF Protection** - Enable CSRF protection for state-changing operations
- **Session Management** - Configure secure session settings
- **Security Headers** - Implement proper security headers

- **Regular Updates** - Keep dependencies updated for security patches

---

16. Spring Core, Web, Boot, Scheduling, Data, Bean Annotations

Bean Definition & Configuration

```
@Component              // Generic Spring-managed bean
@Service               // Service layer component
@Repository            // DAO layer component (enables exception transl
@Controller             // MVC controller component
@Configuration          // Configuration class
@Bean                  // Bean definition method
@Import                // Import configuration classes
@ImportResource         // Import XML configuration
@ComponentScan         // Component scanning
@Lazy                  // Lazy bean initialization
@Scope                 // Bean scope definition
@Primary               // Primary bean candidate
@Qualifier             // Bean qualifier
@DependsOn             // Bean dependency declaration
@Order                 // Bean ordering
@Profile               // Profile-specific bean
```

Dependency Injection

```
@Autowired             // Auto-wire dependency
@Inject                // JSR-330 injection
@Resource              // JSR-250 injection (by name)
@Value                 # Inject property values
@Nullable              // Nullable dependency
@NonNull               // Non-null dependency
```

Core Examples

```
@Configuration
@Import({DatabaseConfig.class, SecurityConfig.class})
@ComponentScan(basePackages = "com.example")
public class AppConfig {

    @Bean
    @Primary
    @Scope("prototype")
    public UserService primaryUserService() {
        return new UserServiceImpl();
    }

    @Bean
    @Lazy
    @Qualifier("cacheService")
    public CacheService cacheService() {
```

```
        return new RedisCacheService();
    }

    @Bean
    @DependsOn("databaseInitializer")
    public DataInitializer dataInitializer() {
        return new DataInitializer();
    }

    @Bean
    @Order(1)
    public CommandLineRunner initializerRunner() {
        return args -> System.out.println("Initializing application..."
    }
}

@Service
@Profile("production")
public class ProductionUserService implements UserService {

    @Autowired
    @Qualifier("userRepository")
    private UserRepository userRepository;

    @Value("${app.user.cache.enabled}")
    private boolean cacheEnabled;

    @Inject
    private CacheManager cacheManager;

    @Resource(name = "notificationService")
    private NotificationService notificationService;
}
```

Lifecycle & Callbacks

```
@PostConstruct          // Post-construction callback
@PreDestroy           // Pre-destruction callback
@EventListener        // Application event listener
@Async                // Async method execution
@Scheduled            // Scheduled task execution
@Timed                // Timed execution (Micrometer)
```

Event Handling

```
@Component
public class ApplicationEventHandler {

    @EventListener
    @Async
    public void handleUserCreated(UserCreatedEvent event) {
        notificationService.sendWelcomeEmail(event.getUser());
    }
```

```
    @EventListener(condition = "#event.success")
    public void handleSuccessfulLogin(LoginEvent event) {
        auditService.logSuccessfulLogin(event.getUser());
    }

    @EventListener
    public void handleContextRefreshed(ContextRefreshedEvent event) {
        log.info("Application context refreshed");
    }
}
```

---

16. Spring Core, Web, Boot, Scheduling, Data, Bean Annotations

HTTP Request Mapping

```
@RequestMapping          // Base URL mapping
@GetMapping             // HTTP GET mapping
@PostMapping            // HTTP POST mapping
@PutMapping             // HTTP PUT mapping
@DeleteMapping          // HTTP DELETE mapping
@PatchMapping           // HTTP PATCH mapping
@RequestHeader          // Bind HTTP header
@CookieValue            # Bind HTTP cookie
@PathVariable           # Bind URL path variable
@RequestParam           # Bind request parameter
@RequestBody            # Bind request body
@RequestPart           # Bind multipart part
@MatrixVariable        # Bind matrix variable
```

Response Handling

```
@ResponseBody          // Return response body
@ResponseStatus        # Set HTTP status
ResponseEntity          # Full response control
@CrossOrigin            # Enable CORS
@ExceptionHandler       # Exception handler
@ControllerAdvice       # Global exception handling
@RestControllerAdvice  # Global REST exception handling
```

Web Configuration

```
@EnableWebMvc           # Enable Spring MVC
@Configuration          # Web configuration class
@WebAppConfiguration    # Web application context
@SessionAttributes      # Session attributes
@ModelAttribute         # Model attribute
@InitBinder            # Data binding initializer
@SessionAttribute      # Session attribute binding
```

Web Examples

```
@RestController
@RequestMapping("/api/v1/users")
@CrossOrigin(origins = "http://localhost:3000")
public class UserController {

    @GetMapping("/{id}")
    public ResponseEntity<User> getUser(
            @PathVariable @Min(1) Long id,
            @RequestHeader("Accept") String acceptHeader,
            @CookieValue(value = "session", required = false) String se
        return userService.findById(id)
            .map(ResponseEntity::ok)
            .orElse(ResponseEntity.notFound().build());
    }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public User createUser(@Valid @RequestBody CreateUserRequest reques
        return userService.create(request);
    }

    @PutMapping("/{id}")
    public User updateUser(
            @PathVariable Long id,
            @RequestPart("user") @Valid User user,
            @RequestPart("file") MultipartFile file) {
        return userService.updateWithFile(id, user, file);
    }

    @GetMapping("/search")
    public Page<User> searchUsers(
            @RequestParam(required = false) String name,
            @RequestParam(defaultValue = "0") int page,
            @RequestParam(defaultValue = "10") int size,
            @MatrixVariable(pathVar = "filters") Map<String, String> fi
        return userService.search(name, page, size, filters);
    }
}

@ControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    @ResponseStatus(HttpStatus.NOT_FOUND)
    public ResponseEntity<ErrorResponse> handleNotFound(ResourceNotFoun
        ErrorResponse error = new ErrorResponse("NOT_FOUND", ex.getMess
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(error);
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    @ResponseStatus(HttpStatus.BAD_REQUEST)
```

```
    public ResponseEntity<ErrorResponse> handleValidation(MethodArgumen
        ErrorResponse error = new ErrorResponse("VALIDATION_ERROR",
            ex.getBindingResult().getFieldErrors().stream()
                .map(FieldError::getDefaultMessage)
                .collect(Collectors.joining(", ")));
        return ResponseEntity.badRequest().body(error);
    }
}
```

MVC Configuration

```
@Configuration
@EnableWebMvc
@ComponentScan(basePackages = "com.example.web")
public class WebConfig implements WebMvcConfigurer {

    @Override
    public void addInterceptors(InterceptorRegistry registry) {
        registry.addInterceptor(new LoggingInterceptor());
    }

    @Override
    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/api/**")
            .allowedOrigins("http://localhost:3000")
            .allowedMethods("GET", "POST", "PUT", "DELETE");
    }

    @Bean
    public InternalResourceViewResolver viewResolver() {
        InternalResourceViewResolver resolver = new InternalResourceVie
        resolver.setPrefix("/WEB-INF/views/");
        resolver.setSuffix(".jsp");
        return resolver;
    }
}
```

---

16. Spring Core, Web, Boot, Scheduling, Data, Bean Annotations

Application Configuration

```
@SpringBootApplication        // Main application class
@EnableAutoConfiguration    // Enable auto-configuration
@SpringBootConfiguration    // Boot-specific configuration
@BootstrapContext           // Bootstrap context
@ImportAutoConfiguration    // Import auto-configuration
@AutoConfigureBefore       // Auto-configuration ordering
@AutoConfigureAfter        // Auto-configuration ordering
```

Properties & Configuration

```
@ConfigurationProperties   // Bind external properties
@EnableConfigurationProperties // Enable configuration properties
@ConstructorBinding        // Constructor-based property binding
@NestedConfigurationProperty // Nested property binding
```

Conditional Configuration

```
@ConditionalOnClass        // Conditional on class presence
@ConditionalOnMissingClass // Conditional on class absence
@ConditionalOnBean         // Conditional on bean presence
@ConditionalOnMissingBean  // Conditional on bean absence
@ConditionalOnProperty     // Conditional on property
@ConditionalOnResource     // Conditional on resource
@ConditionalOnWebApplication // Conditional on web app
@ConditionalOnNotWebApplication // Conditional on non-web app
@ConditionalOnJava         // Conditional on Java version
@ConditionalOnJndi        // Conditional on JNDI
@ConditionalOnCloudPlatform // Conditional on cloud platform
```

Actuator & Monitoring

```
@Endpoint                 // Custom actuator endpoint
@ReadOperation           // Read operation for endpoint
@WriteOperation          // Write operation for endpoint
@DeleteOperation         // Delete operation for endpoint
@Selector                // Endpoint selector
```

Boot Examples

```
@SpringBootApplication
@EnableAutoConfiguration
@ComponentScan(basePackages = "com.example")
public class Application {
    public static void main(String[] args) {
        SpringApplication.run(Application.class, args);
    }
}

@Configuration
@ConfigurationProperties(prefix = "app.datasource")
@ConstructorBinding
public record DataSourceProperties(
    @NotBlank String url,
    @Min(1) @Max(100) int maxConnections,
    @Valid ConnectionPool pool
) {
    public record ConnectionPool(
        @Min(1) @Max(50) int initialSize = 5,
        @Min(1) @Max(100) int maxSize = 20,
        long timeoutMs = 30000
    ) {}
}
```

```
@Endpoint(id = "custom")
@Component
public class CustomEndpoint {

    @ReadOperation
    public Map<String, Object> customInfo() {
        return Map.of(
            "status", "UP",
            "timestamp", Instant.now(),
            "version", "1.0.0"
        );
    }

    @WriteOperation
    public void updateConfig(@Selector String key, String value) {
        configService.updateProperty(key, value);
    }
}
```

---

16. Spring Core, Web, Boot, Scheduling, Data, Bean Annotations

Scheduling Configuration

```
@EnableScheduling        // Enable scheduled tasks
@EnableAsync             // Enable async support
@Scheduled              // Schedule method execution
@Async                  // Async method execution
@Timed                  // Timed execution
@EventListener          // Event listener
@AsyncEventListener      // Async event listener
```

Scheduled Tasks

```
@Service
@EnableScheduling
public class ScheduledTasks {

    @Scheduled(fixedRate = 5000) // Every 5 seconds
    public void reportCurrentTime() {
        log.info("Current time: {}", LocalDateTime.now());
    }

    @Scheduled(fixedDelay = 10000, initialDelay = 5000) // 5s initial,
    public void processBatch() {
        // Batch processing logic
    }

    @Scheduled(cron = "0 0 12 * * MON-FRI") // Weekdays at noon
    public void weekdayTask() {
        // Weekday task
```

```
    }

    @Scheduled(cron = "${app.backup.cron:0 0 2 * * *}") // Configurable
    public void backupDatabase() {
        // Backup logic
    }

    @Scheduled(timeZone = "UTC", cron = "0 0 * * * *") // Every hour in
    public void hourlyTask() {
        // Hourly task
    }
}
```

Async Processing

```
@Service
@EnableAsync
public class EmailService {

    @Async
    public CompletableFuture<Void> sendWelcomeEmail(User user) {
        emailClient.sendWelcome(user);
        return CompletableFuture.completedFuture(null);
    }

    @Async("taskExecutor") // Use specific executor
    public CompletableFuture<String> generateReport(ReportRequest reque
        String report = reportGenerator.generate(request);
        return CompletableFuture.completedFuture(report);
    }

    @Async
    @Timed(name = "email.send", description = "Time taken to send email
    public void sendNotification(Notification notification) {
        notificationService.send(notification);
    }
}

@Configuration
@EnableAsync
public class AsyncConfig {

    @Bean(name = "taskExecutor")
    public Executor taskExecutor() {
        ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
        executor.setCorePoolSize(5);
        executor.setMaxPoolSize(10);
        executor.setQueueCapacity(25);
        executor.setThreadNamePrefix("Async-");
        executor.initialize();
        return executor;
    }
}
```

Event-Based Scheduling

```
@Component
public class EventBasedScheduler {

    @EventListener
    @Async
    public void handleUserRegistration(UserRegisteredEvent event) {
        // Send welcome email asynchronously
        emailService.sendWelcomeEmail(event.getUser());

        // Initialize user preferences
        preferenceService.initializeDefaults(event.getUser());

        // Add to mailing list
        mailingListService.addUser(event.getUser());
    }

    @EventListener(condition = "#event.type == 'PAYMENT'")
    @Scheduled(fixedRate = 60000) // Check every minute
    public void processPaymentEvents() {
        List<PaymentEvent> events = eventRepository.findUnprocessedPaym
        events.forEach(this::processPayment);
    }
}
```

---

16. Spring Core, Web, Boot, Scheduling, Data, Bean Annotations

JPA & Entity Mapping

```
@Entity                  // JPA entity
@Table                  // Table specification
@Id                     // Primary key
@GeneratedValue         // Auto-generated primary key
@Column                 // Column specification
@Embedded               // Embedded object
@Enumerated             // Enum mapping
@Lob                    // Large object
@Temporal               // Date/time mapping
@Version                // Optimistic locking
@Transient              // Non-persistent field
@Basic                  // Basic property mapping
@Access                 // Access type
```

Relationship Mapping

```
@OneToMany              // One-to-many relationship
@ManyToOne              // Many-to-one relationship
@OneToOne               // One-to-one relationship
@ManyToMany             // Many-to-many relationship
```

```
@JoinTable              // Join table specification
@JoinColumn             // Join column specification
@JoinColumns            // Multiple join columns
@PrimaryKeyJoinColumn   // Primary key join column
@MapsId                 // Maps ID from relationship
```

Query & Repository

```
@Query                  // Custom query
@Param                  // Named parameter
@Modifying              // Modifying query
@Lock                   // Locking
@QueryHints             // Query hints
@Procedure              // Stored procedure
@NamedQueries           // Named queries
@NamedQuery             // Named query
```

Transaction & Caching

```
@Transactional          // Transaction boundary
@Cacheable              // Cache result
@CachePut               // Update cache
@CacheEvict             // Remove from cache
@CacheConfig            // Cache configuration
```

Data Examples

```
@Entity
@Table(name = "orders", indexes = @Index(name = "idx_order_date", colum
public class Order {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "order_number", unique = true, nullable = false)
    private String orderNumber;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "customer_id", nullable = false)
    private Customer customer;

    @OneToMany(mappedBy = "order", cascade = CascadeType.ALL, fetch = F
    private List<OrderItem> items = new ArrayList<>();

    @Enumerated(EnumType.STRING)
    @Column(name = "status", nullable = false)
    private OrderStatus status;

    @Temporal(TemporalType.TIMESTAMP)
    @Column(name = "order_date", nullable = false)
    private Date orderDate;
```

```
    @Version
    private Long version;
}

@Repository
public interface OrderRepository extends JpaRepository<Order, Long> {

    @Query("SELECT o FROM Order o WHERE o.customer.id = :customerId AND
    List<Order> findByCustomerAndStatus(@Param("customerId") Long custo

    @Modifying
    @Transactional
    @Query("UPDATE Order o SET o.status = :newStatus WHERE o.id = :orde
    int updateOrderStatus(@Param("orderId") Long orderId, @Param("newSt

    @Lock(LockModeType.PESSIMISTIC_WRITE)
    @Query("SELECT o FROM Order o WHERE o.id = :orderId")
    Optional<Order> findByIdForUpdate(@Param("orderId") Long orderId);

    @QueryHints(@QueryHint(name = org.hibernate.jpa.QueryHints.HINT_CAC
    @Query("SELECT o FROM Order o WHERE o.orderDate BETWEEN :startDate
    List<Order> findByDateRange(@Param("startDate") Date startDate, @Pa
}
```

---

16. Spring Core, Web, Boot, Scheduling, Data, Bean Annotations

Bean Definition

```
@Component              // Generic component
@Service               // Service component
@Repository            // Repository component
@Controller             // Controller component
@Configuration          // Configuration class
@Bean                  // Bean definition
@Lazy                  // Lazy initialization
@Scope                 // Bean scope
@Primary               // Primary bean
@Qualifier             // Bean qualifier
```

Bean Lifecycle

```
@PostConstruct          // After dependency injection
@PreDestroy           // Before bean destruction
@DependsOn             // Bean dependency
@Order                 // Bean ordering
@Role                  // Bean role
```

Bean Processing

```
@ComponentScan         // Component scanning
```

```
@Import                // Import configuration
@ImportResource         // Import XML
@Profile               // Profile-specific
@Conditional           // Conditional bean
@AutoWired             // Auto-wiring
@Inject                // JSR-330 injection
@Resource              // JSR-250 injection
@Value                 # Property injection
```

Bean Examples

```
@Configuration
@ComponentScan(basePackages = "com.example")
@Import({DatabaseConfig.class, SecurityConfig.class})
public class BeanConfig {

    @Bean
    @Primary
    @Scope("singleton")
    public UserService userService(UserRepository userRepository) {
        return new UserServiceImpl(userRepository);
    }

    @Bean
    @Lazy
    @Qualifier("cacheService")
    public CacheService redisCacheService() {
        return new RedisCacheService();
    }

    @Bean
    @DependsOn("dataSource")
    public DatabaseInitializer databaseInitializer() {
        return new DatabaseInitializer();
    }

    @Bean
    @Order(1)
    public CommandLineRunner dataInitializerRunner() {
        return args -> {
            log.info("Initializing data...");
            dataInitializerService.initialize();
        };
    }

    @Bean
    @Profile("production")
    public DataSource productionDataSource() {
        return new HikariDataSource();
    }

    @Bean
    @Profile("development")
```

```
    public DataSource developmentDataSource() {
        return new EmbeddedDatabaseBuilder()
            .setType(EmbeddedDatabaseType.H2)
            .build();
    }
}

@Service
@Order(2)
public class UserServiceImpl implements UserService {

    @Autowired
    @Qualifier("userRepository")
    private UserRepository userRepository;

    @Inject
    private PasswordEncoder passwordEncoder;

    @Value("${app.user.default.role:USER}")
    private String defaultRole;

    @Resource(name = "notificationService")
    private NotificationService notificationService;

    @PostConstruct
    public void init() {
        log.info("UserService initialized with default role: {}", defau
    }

    @PreDestroy
    public void cleanup() {
        log.info("UserService cleanup");
    }
}
```

---

18. JMS, Redis, RocksDB, and Kafka Annotations

JMS Core Annotations

```
@EnableJms                 // Enable JMS support
@JmsListener              // JMS message listener
@JmsListenerDestination  // Configure JMS destination
@SendTo                   // Send message to destination
@JmsHeaders               # JMS headers binding
@Validated                # Message validation
```

JMS Configuration

```
@Configuration
@EnableJms
public class JmsConfig {
```

```
    @Bean
    public DefaultJmsListenerContainerFactory jmsListenerContainerFacto
            ConnectionFactory connectionFactory,
            DefaultMessageListenerContainerFactoryConfigurer configurer

        DefaultJmsListenerContainerFactory factory = new DefaultJmsList
        factory.setConnectionFactory(connectionFactory);
        factory.setConcurrency("3-10");
        factory.setMessageConverter(jacksonJmsMessageConverter());
        factory.setErrorHandler(errorHandler());
        factory.setSessionTransacted(true);
        factory.setSessionAcknowledgeMode(Session.AUTO_ACKNOWLEDGE);

        configurer.configure(factory, connectionFactory);
        return factory;
    }

    @Bean
    public MessageConverter jacksonJmsMessageConverter() {
        MappingJackson2MessageConverter converter = new MappingJackson2
        converter.setTargetType(MessageType.TEXT);
        converter.setTypeIdPropertyName("_type");
        return converter;
    }

    @Bean
    public ErrorHandler errorHandler() {
        return new ErrorHandler() {
            @Override
            public void handleError(Throwable t) {
                log.error("JMS error occurred", t);
            }
        };
    }

    @Bean
    public JmsTemplate jmsTemplate(ConnectionFactory connectionFactory)
        JmsTemplate template = new JmsTemplate(connectionFactory);
        template.setMessageConverter(jacksonJmsMessageConverter());
        template.setSessionTransacted(true);
        template.setDeliveryPersistent(true);
        template.setTimeToLive(3600000); // 1 hour
        return template;
    }
}
```

JMS Message Listeners

```
@Component
public class JmsMessageListeners {

    @JmsListener(destination = "user.created.queue")
```

```
    public void handleUserCreated(UserCreatedMessage message) {
        log.info("Received user created message: {}", message);
        userService.processUserCreated(message);
    }

    @JmsListener(
        destination = "order.processed.topic",
        containerFactory = "jmsListenerContainerFactory",
        subscription = "order-subscription"
    )
    public void handleOrderProcessed(OrderProcessedMessage message) {
        log.info("Received order processed message: {}", message);
        orderService.processOrderProcessed(message);
    }

    @JmsListener(
        destination = "payment.notification.queue",
        selector = "priority = 'HIGH'",
        concurrency = "1-3"
    )
    public void handleHighPriorityPayment(PaymentMessage message) {
        log.info("Processing high priority payment: {}", message);
        paymentService.processHighPriorityPayment(message);
    }

    @JmsListener(
        destination = "error.queue",
        errorHandler = "jmsErrorHandler"
    )
    public void handleErrorMessages(ErrorMessage message) {
        log.error("Received error message: {}", message);
        errorService.handleError(message);
    }

    @JmsListener(destination = "request.reply.queue")
    @SendTo("response.queue")
    public ResponseMessage handleRequest(RequestMessage request) {
        log.info("Processing request: {}", request);
        return requestService.processRequest(request);
    }
}

@Component
public class JmsMessageSender {

    private final JmsTemplate jmsTemplate;

    @Autowired
    public JmsMessageSender(JmsTemplate jmsTemplate) {
        this.jmsTemplate = jmsTemplate;
    }

    public void sendUserCreated(UserCreatedMessage message) {
```

```
        jmsTemplate.convertAndSend("user.created.queue", message);
    }

    public void sendOrderProcessed(OrderProcessedMessage message) {
        jmsTemplate.convertAndSend("order.processed.topic", message);
    }

    @Scheduled(fixedRate = 60000)
    public void sendPeriodicHealthCheck() {
        HealthCheckMessage message = new HealthCheckMessage(Instant.now
        jmsTemplate.convertAndSend("health.check.queue", message);
    }
}
```

---

18. JMS, Redis, RocksDB, and Kafka Annotations

Redis Core Annotations

```
@EnableRedisRepositories   // Enable Redis repositories
@RedisHash                // Redis hash mapping
@TimeToLive              // TTL for Redis keys
@Indexed                // Create secondary index
@Reference              // Reference to another Redis hash
```

Redis Configuration

```
@Configuration
@EnableRedisRepositories(basePackages = "com.example.redis.repository")
public class RedisConfig {

    @Bean
    public RedisConnectionFactory redisConnectionFactory() {
        LettuceConnectionFactory factory = new LettuceConnectionFactory
            new RedisStandaloneConfiguration("localhost", 6379));
        factory.setShareNativeConnection(false);
        return factory;
    }

    @Bean
    public RedisTemplate<String, Object> redisTemplate(RedisConnectionF
        RedisTemplate<String, Object> template = new RedisTemplate<>();
        template.setConnectionFactory(connectionFactory);
        template.setKeySerializer(new StringRedisSerializer());
        template.setValueSerializer(new GenericJackson2JsonRedisSeriali
        template.setHashKeySerializer(new StringRedisSerializer());
        template.setHashValueSerializer(new GenericJackson2JsonRedisSer
        return template;
    }

    @Bean
    public RedisCacheManager redisCacheManager(RedisConnectionFactory c
```

```
        RedisCacheConfiguration config = RedisCacheConfiguration.defaul
            .entryTtl(Duration.ofMinutes(30))
            .disableCachingNullValues()
            .serializeKeysWith(RedisSerializationContext.SerializationP
                .fromSerializer(new StringRedisSerializer()))
            .serializeValuesWith(RedisSerializationContext.Serializatio
                .fromSerializer(new GenericJackson2JsonRedisSerializer(

        return RedisCacheManager.builder(connectionFactory)
            .cacheDefaults(config)
            .build();
    }

    @Bean
    public StringRedisTemplate stringRedisTemplate(RedisConnectionFacto
        StringRedisTemplate template = new StringRedisTemplate();
        template.setConnectionFactory(connectionFactory);
        return template;
    }
}
```

Redis Repositories

```
@RedisHash("users")
public class UserRedis {

    @Id
    private String id;

    @Indexed
    private String username;

    @Indexed
    private String email;

    @TimeToLive(unit = TimeUnit.HOURS)
    private long ttl = 24; // 24 hours

    private String firstName;
    private String lastName;

    @Reference
    private UserProfileRedis profile;

    // Getters and setters
}

@RedisHash("user_profiles")
public class UserProfileRedis {

    @Id
    private String userId;
```

```
    private String bio;
    private String avatar;

    @Indexed
    private String status;

    // Getters and setters
}

public interface UserRedisRepository extends RedisRepository<UserRedis,

    List<UserRedis> findByUsername(String username);

    List<UserRedis> findByEmail(String email);

    Page<UserRedis> findByUsernameContaining(String username, Pageable

    @Query("(@username:?0)")
    List<UserRedis> findByUsernameQuery(String username);
}
```

Redis Operations

```
@Service
public class RedisUserService {

    private final UserRedisRepository userRedisRepository;
    private final RedisTemplate<String, Object> redisTemplate;
    private final StringRedisTemplate stringRedisTemplate;

    public RedisUserService(
            UserRedisRepository userRedisRepository,
            RedisTemplate<String, Object> redisTemplate,
            StringRedisTemplate stringRedisTemplate) {
        this.userRedisRepository = userRedisRepository;
        this.redisTemplate = redisTemplate;
        this.stringRedisTemplate = stringRedisTemplate;
    }

    @Cacheable(value = "users", key = "#id")
    public UserRedis findById(String id) {
        return userRedisRepository.findById(id).orElse(null);
    }

    @CachePut(value = "users", key = "#user.id")
    public UserRedis save(UserRedis user) {
        return userRedisRepository.save(user);
    }

    @CacheEvict(value = "users", key = "#id")
    public void deleteById(String id) {
        userRedisRepository.deleteById(id);
    }
```

```
    public void publishUserEvent(UserEvent event) {
        redisTemplate.convertAndSend("user.events", event);
    }

    @EventListener
    public void handleUserEvent(UserEvent event) {
        log.info("Received user event: {}", event);
        // Process event
    }

    public void incrementLoginCount(String userId) {
        stringRedisTemplate.opsForValue().increment("login:count:" + us
    }

    public void addToSet(String setName, String value) {
        stringRedisTemplate.opsForSet().add(setName, value);
    }

    public void addToZSet(String setName, String value, double score) {
        stringRedisTemplate.opsForZSet().add(setName, value, score);
    }
}
```

---

18. JMS, Redis, RocksDB, and Kafka Annotations

RocksDB Configuration

```
@Configuration
public class RocksDBConfig {

    @Bean
    public RocksDB rocksDB() throws RocksDBException {
        RocksDBOptions options = new RocksDBOptions();
        options.setCreateIfMissing(true);
        options.setCreateMissingColumnFamilies(true);

        List<ColumnFamilyDescriptor> columnFamilyDescriptors = Arrays.a
            new ColumnFamilyDescriptor(RocksDB.DEFAULT_COLUMN_FAMILY),
            new ColumnFamilyDescriptor("users".getBytes()),
            new ColumnFamilyDescriptor("sessions".getBytes()),
            new ColumnFamilyDescriptor("cache".getBytes())
        );

        List<ColumnFamilyHandle> columnFamilyHandles = new ArrayList<>(

        RocksDB db = RocksDB.open(options, "data/rocksdb",
            columnFamilyDescriptors, columnFamilyHandles);

        return new RocksDBWrapper(db, columnFamilyHandles);
    }
```

```
    @Bean
    public RocksDBTemplate rocksDBTemplate(RocksDB rocksDB) {
        return new RocksDBTemplate(rocksDB);
    }

    @Bean
    public RocksDBUserRepository userRocksDBRepository(RocksDBTemplate
        return new RocksDBUserRepository(template);
    }
}
```

RocksDB Operations

```
@Repository
public class RocksDBUserRepository {

    private final RocksDBTemplate template;
    private final ColumnFamilyHandle userColumnFamily;

    public RocksDBUserRepository(RocksDBTemplate template) {
        this.template = template;
        this.userColumnFamily = template.getColumnFamily("users");
    }

    public void save(User user) {
        byte[] key = user.getId().getBytes();
        byte[] value = serialize(user);
        template.put(userColumnFamily, key, value);
    }

    public User findById(Long id) {
        byte[] key = id.toString().getBytes();
        byte[] value = template.get(userColumnFamily, key);
        return value != null ? deserialize(value) : null;
    }

    public void deleteById(Long id) {
        byte[] key = id.toString().getBytes();
        template.delete(userColumnFamily, key);
    }

    public List<User> findAll() {
        List<User> users = new ArrayList<>();
        try (RocksIterator iterator = template.newIterator(userColumnFa
            iterator.seekToFirst();
            while (iterator.isValid()) {
                User user = deserialize(iterator.value());
                users.add(user);
                iterator.next();
            }
        }
        return users;
```

```
    }

    private byte[] serialize(User user) {
        try {
            return objectMapper.writeValueAsBytes(user);
        } catch (Exception e) {
            throw new RuntimeException("Serialization failed", e);
        }
    }

    private User deserialize(byte[] data) {
        try {
            return objectMapper.readValue(data, User.class);
        } catch (Exception e) {
            throw new RuntimeException("Deserialization failed", e);
        }
    }
}
```

---

18. JMS, Redis, RocksDB, and Kafka Annotations

Kafka Core Annotations

```
@EnableKafka               // Enable Kafka support
@KafkaListener            // Kafka message listener
@KafkaHandler             // Kafka message handler
@SendTo                   // Send message to topic
@TopicPartition           // Topic partition assignment
@Header                   // Message header binding
@Payload                  # Message payload binding
```

Kafka Configuration

```
@Configuration
@EnableKafka
public class KafkaConfig {

    @Bean
    public ConsumerFactory<String, Object> consumerFactory() {
        Map<String, Object> props = new HashMap<>();
        props.put(ConsumerConfig.BOOTSTRAP_SERVERS_CONFIG, "localhost:9
        props.put(ConsumerConfig.GROUP_ID_CONFIG, "user-group");
        props.put(ConsumerConfig.KEY_DESERIALIZER_CLASS_CONFIG, StringD
        props.put(ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG, JsonD
        props.put(ConsumerConfig.AUTO_OFFSET_RESET_CONFIG, "earliest");
        props.put(ConsumerConfig.ENABLE_AUTO_COMMIT_CONFIG, false);
        props.put(JsonDeserializer.TRUSTED_PACKAGES, "com.example");

        return new DefaultKafkaConsumerFactory<>(props, new StringDeser
            new JsonDeserializer<>(Object.class));
    }
```

```
    @Bean
    public ProducerFactory<String, Object> producerFactory() {
        Map<String, Object> props = new HashMap<>();
        props.put(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG, "localhost:9
        props.put(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG, StringSer
        props.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG, JsonSer

        return new DefaultKafkaProducerFactory<>(props);
    }

    @Bean
    public KafkaTemplate<String, Object> kafkaTemplate() {
        return new KafkaTemplate<>(producerFactory());
    }

    @Bean
    public ConcurrentKafkaListenerContainerFactory<String, Object> kafk
            ConsumerFactory<String, Object> consumerFactory) {

        ConcurrentKafkaListenerContainerFactory<String, Object> factory
            new ConcurrentKafkaListenerContainerFactory<>();
        factory.setConsumerFactory(consumerFactory);
        factory.setConcurrency(3);
        factory.getContainerProperties().setAckMode(AckMode.MANUAL_IMME
        factory.getContainerProperties().setPollTimeout(3000);
        factory.setErrorHandler(new SeekToCurrentErrorHandler());

        return factory;
    }
}
```

Kafka Message Listeners

```
@Component
public class KafkaMessageListeners {

    @KafkaListener(
        topics = "user.created",
        groupId = "user-group",
        containerFactory = "kafkaListenerContainerFactory"
    )
    public void handleUserCreated(UserCreatedMessage message) {
        log.info("Received user created message: {}", message);
        userService.processUserCreated(message);
    }

    @KafkaListener(
        topics = "order.processed",
        groupId = "order-group",
        containerFactory = "kafkaListenerContainerFactory"
    )
    @SendTo("order.notification")
```

```
public OrderNotificationMessage handleOrderProcessed(OrderProcessed
    log.info("Processing order: {}", message);
    return orderService.createNotification(message);
}

@KafkaListener(
    topics = "payment.events",
    groupId = "payment-group",
    containerFactory = "kafkaListenerContainerFactory"
)
public void handlePaymentEvent(
        @Payload PaymentEvent event,
        @Header(KafkaHeaders.RECEIVED_TOPIC) String topic,
        @Header(KafkaHeaders.RECEIVED_PARTITION_ID) int partition,
        @Header(KafkaHeaders.OFFSET) long offset) {

    log.info("Received payment event from topic: {}, partition: {},
            topic, partition, offset);
    paymentService.processPaymentEvent(event);
}

@KafkaListener(
    topics = "batch.events",
    groupId = "batch-group",
    containerFactory = "kafkaListenerContainerFactory",
    batch = "true"
)
public void handleBatchEvents(List<BatchEvent> events) {
    log.info("Received batch of {} events", events.size());
    events.forEach(this::processBatchEvent);
}

@KafkaListener(
    topics = "error.events",
    groupId = "error-group",
    containerFactory = "kafkaListenerContainerFactory",
    errorHandler = "kafkaErrorHandler"
)
public void handleErrorEvents(ErrorMessage message) {
    log.error("Processing error message: {}", message);
    errorService.handleError(message);
}

@KafkaListener(
    topics = "retry.events",
    groupId = "retry-group",
    containerFactory = "kafkaListenerContainerFactory"
)
@Retryable(value = {Exception.class}, maxAttempts = 3, backoff = @B
public void handleRetryEvents(RetryEvent message) {
    log.info("Processing retry event: {}", message);
    retryService.processRetry(message);
}
```

```
}

@Component
public class KafkaMessageSender {

    private final KafkaTemplate<String, Object> kafkaTemplate;

    @Autowired
    public KafkaMessageSender(KafkaTemplate<String, Object> kafkaTempla
        this.kafkaTemplate = kafkaTemplate;
    }

    public void sendUserCreated(UserCreatedMessage message) {
        kafkaTemplate.send("user.created", message);
    }

    public void sendOrderProcessed(OrderProcessedMessage message) {
        kafkaTemplate.send("order.processed", message);
    }

    @Scheduled(fixedRate = 60000)
    public void sendPeriodicHealthCheck() {
        HealthCheckMessage message = new HealthCheckMessage(Instant.now
        kafkaTemplate.send("health.check", message);
    }

    public void sendWithHeaders(String topic, Object message, Map<Strin
        ProducerRecord<String, Object> record = new ProducerRecord<>(to
        headers.forEach((key, value) -> record.headers().add(key, value
        kafkaTemplate.send(record);
    }

    @EventListener
    public void handleApplicationEvent(ApplicationEvent event) {
        if (event instanceof UserCreatedEvent) {
            UserCreatedEvent userEvent = (UserCreatedEvent) event;
            sendUserCreated(new UserCreatedMessage(userEvent.getUser())
        }
    }
}
```

Kafka Streams Integration

```
@Configuration
public class KafkaStreamsConfig {

    @Bean
    public KStream<String, UserEvent> userEventStream(StreamsBuilder bu
        KStream<String, UserEvent> userEvents = builder.stream("user.ev

        userEvents
            .filter((key, event) -> event.getType().equals("CREATED"))
            .mapValues(event -> transformUserEvent(event))
```

```
            .to("user.processed.events");

        return userEvents;
    }

    @Bean
    public KStream<String, OrderEvent> orderEventStream(StreamsBuilder
        KStream<String, OrderEvent> orderEvents = builder.stream("order

        orderEvents
            .branch(
                (key, event) -> event.getAmount() > 1000,
                (key, event) -> event.getAmount() <= 1000
            )
            .get(0)
            .to("high.value.orders");

        return orderEvents;
    }

    private UserEvent transformUserEvent(UserEvent event) {
        // Transform logic
        return event;
    }
}
```

Multi-Message Type Handling

```
@Component
public class MultiTypeKafkaHandler {

    @KafkaListener(
        topics = "multi.type.events",
        groupId = "multi-type-group",
        containerFactory = "kafkaListenerContainerFactory"
    )
    public void handleMultiTypeEvents(Object message) {
        if (message instanceof UserEvent) {
            handleUserEvent((UserEvent) message);
        } else if (message instanceof OrderEvent) {
            handleOrderEvent((OrderEvent) message);
        } else if (message instanceof PaymentEvent) {
            handlePaymentEvent((PaymentEvent) message);
        } else {
            log.warn("Unknown message type: {}", message.getClass());
        }
    }

    @KafkaHandler
    public void handleUserEvent(UserEvent event) {
        log.info("Handling user event: {}", event);
        userService.processUserEvent(event);
    }
```

```
    @KafkaHandler
    public void handleOrderEvent(OrderEvent event) {
        log.info("Handling order event: {}", event);
        orderService.processOrderEvent(event);
    }

    @KafkaHandler
    public void handlePaymentEvent(PaymentEvent event) {
        log.info("Handling payment event: {}", event);
        paymentService.processPaymentEvent(event);
    }
}
```

---

19. Spring Cloud Annotations

Service Discovery & Registration

```
@EnableDiscoveryClient      // Enable service discovery
@EnableEurekaClient         // Enable Eureka client
@EnableEurekaServer        // Enable Eureka server
@RefreshScope             // Refresh configuration on change
@LoadBalanced             // Enable client-side load balancing
```

Configuration Management

```
@EnableConfigServer        // Enable configuration server
@EnableConfigClient        // Enable configuration client
@ConfigurationProperties   // External configuration binding
@RefreshScope             // Dynamic configuration refresh
```

Circuit Breaker & Resilience

```
@EnableCircuitBreaker      // Enable circuit breaker
@CircuitBreaker           // Circuit breaker on method
@Retry                    // Retry mechanism
@TimeLimiter              // Timeout mechanism
@Bulkhead                 // Bulkhead pattern
@Fallback                 // Fallback method
```

API Gateway

```
@EnableZuulProxy          // Enable Zuul proxy
@EnableZuulServer         // Enable Zuul server
@ZuulFilter              // Custom Zuul filter
@Route                    // Route configuration
```

Distributed Tracing

```
@EnableZipkinServer       // Enable Zipkin server
@EnableZipkinClient       // Enable Zipkin client
```

```
@NewSpan                    // Create new span
@SpanTag                    # Add span tag
```

Spring Cloud Examples

```
// Configuration Server
@SpringBootApplication
@EnableConfigServer
@EnableDiscoveryClient
public class ConfigServerApplication {
    public static void main(String[] args) {
        SpringApplication.run(ConfigServerApplication.class, args);
    }
}

// Service with Discovery and Config
@RestController
@RefreshScope
public class UserController {

    @Value("${app.user.default.role:USER}")
    private String defaultRole;

    @Autowired
    @LoadBalanced
    private RestTemplate restTemplate;

    @GetMapping("/users/{id}")
    public User getUser(@PathVariable Long id) {
        return userService.findById(id);
    }

    @CircuitBreaker(fallbackMethod = "getUserFallback")
    @Retry(maxAttempts = 3)
    public User getUserWithCircuitBreaker(Long id) {
        return restTemplate.getForObject("http://user-service/users/" +
    }

    public User getUserFallback(Long id, Exception ex) {
        return new User(id, "Fallback User", defaultRole);
    }
}

// Circuit Breaker Configuration
@Configuration
@EnableCircuitBreaker
public class CircuitBreakerConfig {

    @Bean
    public CircuitBreaker circuitBreaker() {
        return CircuitBreaker.ofDefaults("userService");
    }
```

```
    @Bean
    public TimeLimiter timeLimiter() {
        return TimeLimiter.of(Duration.ofSeconds(5));
    }

    @Bean
    public Bulkhead bulkhead() {
        return Bulkhead.ofDefaults("userService");
    }
}

// Custom Zuul Filter
@Component
public class CustomZuulFilter extends ZuulFilter {

    @Override
    public String filterType() {
        return "pre";
    }

    @Override
    public int filterOrder() {
        return 1;
    }

    @Override
    public boolean shouldFilter() {
        return true;
    }

    @Override
    public Object run() throws ZuulException {
        RequestContext ctx = RequestContext.getCurrentContext();
        ctx.addZuulRequestHeader("X-Custom-Header", "CustomValue");
        return null;
    }
}

// Distributed Tracing
@RestController
public class TracingController {

    @NewSpan("process-user")
    @SpanTag("userId")
    public User processUser(@SpanTag("user.id") Long userId) {
        // Business logic with tracing
        return userService.process(userId);
    }
}
```

Spring Cloud Config

```
@Configuration
```

```
@EnableConfigServer
public class ConfigServerConfig {

    @Bean
    public EnvironmentRepository environmentRepository() {
        return new GitEnvironmentRepository()
            .setUri("https://github.com/config-repo")
            .setSearchPaths("config")
            .setCloneOnStart(true);
    }
}

// Client Configuration
@Configuration
@EnableConfigClient
public class ConfigClientConfig {

    @Bean
    @RefreshScope
    public AppConfig appConfig() {
        return new AppConfig();
    }
}

@RefreshScope
@ConfigurationProperties(prefix = "app")
public class AppConfig {
    private String name;
    private String version;
    private Map<String, String> features;

    // Getters and setters
}
```

---

20. Advanced Spring Data Annotations

MongoDB Annotations

```
@Document                // MongoDB document mapping
@Id                      // Document ID
@Field                   // Field mapping
@Indexed                 // Create index
@TextIndexed             // Text index
@GeoSpatialIndexed       // Geospatial index
@DBRef                   // Document reference
@CompoundIndex           // Compound index
@Transient               // Non-persistent field
```

MongoDB Examples

```
@Document(collection = "users")
```

```
@CompoundIndex(name = "user_email_idx", def = "{'username': 1, 'email':
public class UserDocument {

    @Id
    private String id;

    @Field("username")
    @Indexed(unique = true)
    private String username;

    @Field("email")
    @Indexed(unique = true)
    private String email;

    @Field("profile")
    @DBRef
    private UserProfileDocument profile;

    @Field("bio")
    @TextIndexed
    private String bio;

    @Field("location")
    @GeoSpatialIndexed
    private GeoJsonPoint location;

    @Transient
    private String computedField;

    // Getters and setters
}

@Repository
public interface UserRepository extends MongoRepository<UserDocument, S

    List<UserDocument> findByUsername(String username);

    List<UserDocument> findByEmail(String email);

    @Query("{ 'username' : ?0, 'active' : true }")
    List<UserDocument> findByUsernameAndActive(String username);

    @Query("{ '$text' : { '$search' : ?0 } }")
    List<UserDocument> searchByText(String searchTerm);

    @Query("{ 'location' : { '$near' : { '$geometry' : ?0, '$maxDistanc
    List<UserDocument> findByLocationNear(GeoJsonPoint point, double ma
}
```

Elasticsearch Annotations

```
@Document               // Elasticsearch document
@Field                  // Field mapping
```

```
@Id                     // Document ID
@Setting                // Index settings
@Mapping                // Type mapping
```

Elasticsearch Examples

```
@Document(indexName = "products")
@Setting(settingPath = "elasticsearch/product-settings.json")
public class ProductDocument {

    @Id
    private String id;

    @Field(type = FieldType.Text, analyzer = "standard")
    private String name;

    @Field(type = FieldType.Keyword)
    private String category;

    @Field(type = FieldType.Double)
    private double price;

    @Field(type = FieldType.Date, format = DateFormat.date_time)
    private LocalDateTime createdAt;

    @Field(type = FieldType.Nested)
    private List<AttributeDocument> attributes;

    // Getters and setters
}

@Repository
public interface ProductRepository extends ElasticsearchRepository<Prod

    List<ProductDocument> findByName(String name);

    List<ProductDocument> findByCategory(String category);

    List<ProductDocument> findByPriceBetween(double minPrice, double ma

    @Query("{\"match\": {\"name\": \"?0\"}}")
    List<ProductDocument> searchByName(String searchTerm);

    @Query("{\"bool\": {\"must\": [{\"match\": {\"name\": \"?0\"}}, {\"
    List<ProductDocument> searchByNameAndCategory(String name, String c
}
```

Cassandra Annotations

```
@Table                  // Cassandra table mapping
@PrimaryKey             // Primary key
@PartitionKey           // Partition key
@ClusteringKey          // Clustering key
```

```
@Column                 // Column mapping
@Indexed                // Create index
@Transient              // Non-persistent field
```

Cassandra Examples

```
@Table(name = "users")
public class UserEntity {

    @PrimaryKeyClass
    public static class Key {
        @PartitionKey
        private UUID userId;

        @ClusteringKey
        private LocalDateTime createdAt;

        // Getters and setters
    }

    @PrimaryKey
    private Key key;

    @Column("username")
    private String username;

    @Column("email")
    @Indexed
    private String email;

    @Transient
    private String computedField;

    // Getters and setters
}

@Repository
public interface UserRepository extends CassandraRepository<UserEntity,

    List<UserEntity> findByKeyUserId(UUID userId);

    List<UserEntity> findByEmail(String email);

    @Query("SELECT * FROM users WHERE user_id = ?0 AND created_at > ?1"
    List<UserEntity> findByUserIdAndCreatedAtAfter(UUID userId, LocalDa
}
```

Neo4j Annotations

```
@Node                   // Neo4j node mapping
@Id                     // Node ID
@Property               // Property mapping
@Relationship          // Relationship mapping
```

```
@RelationshipId        // Relationship ID
@Query                  // Custom query
```

Neo4j Examples

```
@Node
public class UserNode {

    @Id
    @GeneratedValue
    private Long id;

    @Property("username")
    private String username;

    @Property("email")
    private String email;

    @Relationship(type = "FRIEND_OF", direction = Relationship.Directio
    private Set<UserNode> friends = new HashSet<>();

    @Relationship(type = "WORKS_AT", direction = Relationship.Direction
    private CompanyNode company;

    // Getters and setters
}

@Node
public class CompanyNode {

    @Id
    @GeneratedValue
    private Long id;

    @Property("name")
    private String name;

    @Property("industry")
    private String industry;

    // Getters and setters
}

@Repository
public interface UserRepository extends Neo4jRepository<UserNode, Long>

    Optional<UserNode> findByUsername(String username);

    Optional<UserNode> findByEmail(String email);

    @Query("MATCH (u:User)-[:FRIEND_OF]->(f:User) WHERE u.username = $u
    List<UserNode> findFriendsByUsername(String username);
```

```
    @Query("MATCH (u:User)-[:WORKS_AT]->(c:Company) WHERE c.industry =
    List<UserNode> findUsersByCompanyIndustry(String industry);

    @Query("MATCH (u1:User), (u2:User) WHERE u1.username = $username1 A
           "CREATE (u1)-[:FRIEND_OF]->(u2)")
    void createFriendship(String username1, String username2);
}
```

Multi-Database Configuration

```
@Configuration
@EnableJpaRepositories(
    basePackages = "com.example.jpa.repository",
    entityManagerFactoryRef = "jpaEntityManagerFactory",
    transactionManagerRef = "jpaTransactionManager"
)
@EnableMongoRepositories(
    basePackages = "com.example.mongo.repository",
    mongoTemplateRef = "mongoTemplate"
)
public class MultiDatabaseConfig {

    // JPA Configuration
    @Bean
    @Primary
    public LocalContainerEntityManagerFactoryBean jpaEntityManagerFacto
            DataSource dataSource) {
        LocalContainerEntityManagerFactoryBean factory = new LocalConta
        factory.setDataSource(dataSource);
        factory.setPackagesToScan("com.example.jpa.entity");
        factory.setJpaVendorAdapter(new HibernateJpaVendorAdapter());
        factory.setJpaProperties(jpaProperties());
        return factory;
    }

    @Bean
    @Primary
    public PlatformTransactionManager jpaTransactionManager(
            LocalContainerEntityManagerFactoryBean jpaEntityManagerFact
        return new JpaTransactionManager(jpaEntityManagerFactory.getObj
    }

    // MongoDB Configuration
    @Bean
    public MongoTemplate mongoTemplate() {
        return new MongoTemplate(mongoClient(), "testdb");
    }

    @Bean
    public MongoClient mongoClient() {
        return MongoClients.create("mongodb://localhost:27017");
    }
```

```
    // Elasticsearch Configuration
    @Bean
    public ElasticsearchOperations elasticsearchTemplate() {
        return new ElasticsearchRestTemplate(elasticsearchClient());
    }

    @Bean
    public ElasticsearchClient elasticsearchClient() {
        return ElasticsearchClients.createSimple(HttpHost.create("local
    }

    private Properties jpaProperties() {
        Properties properties = new Properties();
        properties.put("hibernate.dialect", "org.hibernate.dialect.H2Di
        properties.put("hibernate.hbm2ddl.auto", "update");
        properties.put("hibernate.show_sql", "true");
        return properties;
    }
}
```

Data Validation & Auditing

```
@Document
public class AuditableDocument {

    @Id
    private String id;

    @CreatedDate
    private LocalDateTime createdAt;

    @LastModifiedDate
    private LocalDateTime updatedAt;

    @CreatedBy
    private String createdBy;

    @LastModifiedBy
    private String updatedBy;

    @Version
    private Long version;

    // Getters and setters
}

@Configuration
@EnableMongoAuditing(auditorAwareRef = "auditorProvider")
public class MongoAuditingConfig {

    @Bean
    public AuditorAware<String> auditorProvider() {
        return () -> {
```

```
            Authentication authentication = SecurityContextHolder.getCo
            return authentication != null ? Optional.of(authentication.
        };
    }
}

@Repository
public interface AuditableRepository extends MongoRepository<AuditableD

    @Lock(LockModeType.PESSIMISTIC_WRITE)
    @Query("{ '_id' : ?0 }")
    AuditableDocument findByIdForUpdate(String id);

    @Modifying
    @Query("{ '_id' : ?0 }")
    void updateDocument(String id, Update update);
}
```

---

21. Spring WebFlux & Reactive Annotations

WebFlux Core Annotations

```
@RestController            // Reactive REST controller
@RequestMapping           // Request mapping
@GetMapping              // GET mapping
@PostMapping             // POST mapping
@PutMapping              // PUT mapping
@DeleteMapping           // DELETE mapping
@PatchMapping            // PATCH mapping
```

Reactive Controllers

```
@RestController
@RequestMapping("/api/reactive/users")
public class ReactiveUserController {

    @GetMapping("/{id}")
    public Mono<User> getUser(@PathVariable Long id) {
        return userService.findById(id);
    }

    @GetMapping
    public Flux<User> getAllUsers() {
        return userService.findAll();
    }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public Mono<User> createUser(@Valid @RequestBody Mono<CreateUserReq
        return requestMono.flatMap(userService::create);
    }
```

```
    @PutMapping("/{id}")
    public Mono<User> updateUser(@PathVariable Long id, @Valid @Request
        return requestMono.flatMap(request -> userService.update(id, re
    }

    @DeleteMapping("/{id}")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    public Mono<Void> deleteUser(@PathVariable Long id) {
        return userService.deleteById(id);
    }

    @GetMapping("/stream")
    public Flux<ServerSentEvent<User>> streamUsers() {
        return userService.findAll()
            .delayElements(Duration.ofSeconds(1))
            .map(user -> ServerSentEvent.builder(user).build());
    }

    @GetMapping("/search")
    public Flux<User> searchUsers(@RequestParam(required = false) Strin
        return userService.search(name);
    }
}
```

Functional Endpoints

```
@Configuration
public class UserRouter {

    @Bean
    public RouterFunction<ServerResponse> userRoutes(UserHandler userHa
        return RouterFunctions.route()
            .path("/api/functional/users", builder -> builder
                .GET("/{id}", userHandler::getUser)
                .GET(userHandler::getAllUsers)
                .POST(userHandler::createUser)
                .PUT("/{id}", userHandler::updateUser)
                .DELETE("/{id}", userHandler::deleteUser)
                .GET("/stream", userHandler::streamUsers)
            )
            .build();
    }
}

@Component
public class UserHandler {

    private final UserService userService;

    public UserHandler(UserService userService) {
        this.userService = userService;
    }
```

```
    public Mono<ServerResponse> getUser(ServerRequest request) {
        Long id = Long.valueOf(request.pathVariable("id"));
        return userService.findById(id)
            .flatMap(user -> ServerResponse.ok().bodyValue(user))
            .switchIfEmpty(ServerResponse.notFound().build());
    }

    public Mono<ServerResponse> getAllUsers(ServerRequest request) {
        return ServerResponse.ok().body(userService.findAll());
    }

    public Mono<ServerResponse> createUser(ServerRequest request) {
        return request.bodyToMono(CreateUserRequest.class)
            .flatMap(userService::create)
            .flatMap(user -> ServerResponse.status(HttpStatus.CREATED).
    }

    public Mono<ServerResponse> updateUser(ServerRequest request) {
        Long id = Long.valueOf(request.pathVariable("id"));
        return request.bodyToMono(UpdateUserRequest.class)
            .flatMap(updateRequest -> userService.update(id, updateRequ
            .flatMap(user -> ServerResponse.ok().bodyValue(user))
            .switchIfEmpty(ServerResponse.notFound().build());
    }

    public Mono<ServerResponse> deleteUser(ServerRequest request) {
        Long id = Long.valueOf(request.pathVariable("id"));
        return userService.deleteById(id)
            .then(ServerResponse.noContent().build());
    }

    public Mono<ServerResponse> streamUsers(ServerRequest request) {
        return ServerResponse.ok()
            .contentType(MediaType.TEXT_EVENT_STREAM)
            .body(userService.findAll()
                .delayElements(Duration.ofSeconds(1))
                .map(user -> ServerSentEvent.builder(user).build()));
    }
}
```

Reactive Security

```
@Configuration
@EnableWebFluxSecurity
public class ReactiveSecurityConfig {

    @Bean
    public SecurityWebFilterChain securityWebFilterChain(ServerHttpSecu
        http
            .authorizeExchange(exchanges -> exchanges
                .pathMatchers("/api/public/**").permitAll()
                .pathMatchers("/api/reactive/admin/**").hasRole("ADMIN"
```

```
                .pathMatchers(HttpMethod.GET, "/api/reactive/users/**")
                .pathMatchers(HttpMethod.POST, "/api/reactive/users/**"
                .anyExchange().authenticated()
            )
            .oauth2Login(withDefaults())
            .csrf(csrf -> csrf.disable())
            .httpBasic(withDefaults());

        return http.build();
    }

    @Bean
    public ReactiveUserDetailsService userDetailsService(UserRepository
        return username -> userRepository.findByUsername(username)
            .map(user -> User.withUsername(user.getUsername())
                .password(user.getPassword())
                .roles(user.getRoles().toArray(new String[0]))
                .build());
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }
}

// Reactive Controller with Security
@RestController
@RequestMapping("/api/reactive/secure")
public class SecureReactiveController {

    @GetMapping("/profile")
    @PreAuthorize("hasRole('USER')")
    public Mono<User> getCurrentProfile(Authentication authentication)
        return userService.findByUsername(authentication.getName());
    }

    @GetMapping("/admin/users")
    @PreAuthorize("hasRole('ADMIN')")
    public Flux<User> getAllUsers() {
        return userService.findAll();
    }

    @PostMapping("/admin/users")
    @PreAuthorize("hasRole('ADMIN')")
    @ResponseStatus(HttpStatus.CREATED)
    public Mono<User> createUser(@Valid @RequestBody Mono<CreateUserReq
        return requestMono.flatMap(userService::create);
    }
}
```

Reactive Data Access

```
@Repository
public interface ReactiveUserRepository extends ReactiveMongoRepository

    Mono<User> findByUsername(String username);

    Flux<User> findByEmail(String email);

    Flux<User> findByActiveTrue();

    @Query("{ 'username' : ?0, 'active' : true }")
    Mono<User> findByUsernameAndActive(String username);

    @Query("{ '$text' : { '$search' : ?0 } }")
    Flux<User> searchByText(String searchTerm);

    @Query("{ 'createdAt' : { '$gte' : ?0, '$lte' : ?1 } }")
    Flux<User> findByCreatedAtBetween(LocalDateTime start, LocalDateTim
}

@Service
public class ReactiveUserService {

    private final ReactiveUserRepository userRepository;
    private final ReactiveRedisTemplate<String, User> redisTemplate;

    public ReactiveUserService(
            ReactiveUserRepository userRepository,
            ReactiveRedisTemplate<String, User> redisTemplate) {
        this.userRepository = userRepository;
        this.redisTemplate = redisTemplate;
    }

    public Mono<User> findById(Long id) {
        return redisTemplate.opsForValue().get("user:" + id)
            .switchIfEmpty(userRepository.findById(id)
                .flatMap(user -> redisTemplate.opsForValue().set("user:
                    .thenReturn(user)));
    }

    public Flux<User> findAll() {
        return userRepository.findAll()
            .delayElements(Duration.ofMillis(100));
    }

    public Mono<User> create(CreateUserRequest request) {
        return userRepository.existsByUsername(request.getUsername())
            .flatMap(exists -> exists ?
                Mono.error(new UserAlreadyExistsException("Username alr
                Mono.fromCallable(() -> convertToUser(request))
                    .flatMap(userRepository::save)
                    .flatMap(user -> redisTemplate.opsForValue().set("u
                        .thenReturn(user))
            );
```

```
    }

    public Mono<User> update(Long id, UpdateUserRequest request) {
        return userRepository.findById(id)
            .switchIfEmpty(Mono.error(new UserNotFoundException("User n
            .flatMap(user -> {
                updateUserData(user, request);
                return userRepository.save(user);
            })
            .flatMap(user -> redisTemplate.opsForValue().set("user:" +
                .thenReturn(user));
    }

    public Mono<Void> deleteById(Long id) {
        return userRepository.findById(id)
            .switchIfEmpty(Mono.error(new UserNotFoundException("User n
            .flatMap(user -> userRepository.deleteById(id)
                .then(redisTemplate.delete("user:" + id))
                .then());
    }

    public Flux<User> search(String searchTerm) {
        return userRepository.searchByText(searchTerm)
            .distinct()
            .take(50);
    }

    public Flux<User> findByActiveTrue() {
        return userRepository.findByActiveTrue()
            .sort(Comparator.comparing(User::getCreatedAt).reversed());
    }

    private User convertToUser(CreateUserRequest request) {
        User user = new User();
        user.setUsername(request.getUsername());
        user.setEmail(request.getEmail());
        user.setFirstName(request.getFirstName());
        user.setLastName(request.getLastName());
        user.setActive(true);
        user.setCreatedAt(LocalDateTime.now());
        return user;
    }

    private void updateUserData(User user, UpdateUserRequest request) {
        user.setFirstName(request.getFirstName());
        user.setLastName(request.getLastName());
        user.setEmail(request.getEmail());
        user.setUpdatedAt(LocalDateTime.now());
    }
}
```

Reactive Validation

```
@Component
public class ReactiveUserValidator {

    public Mono<CreateUserRequest> validateCreateRequest(CreateUserRequ
        return Mono.just(request)
            .flatMap(this::validateUsername)
            .flatMap(this::validateEmail)
            .flatMap(this::validateNames);
    }

    private Mono<CreateUserRequest> validateUsername(CreateUserRequest
        if (StringUtils.isBlank(request.getUsername()) || request.getUs
            return Mono.error(new ValidationException("Username must be
        }
        return Mono.just(request);
    }

    private Mono<CreateUserRequest> validateEmail(CreateUserRequest req
        if (!isValidEmail(request.getEmail())) {
            return Mono.error(new ValidationException("Invalid email fo
        }
        return Mono.just(request);
    }

    private Mono<CreateUserRequest> validateNames(CreateUserRequest req
        if (StringUtils.isBlank(request.getFirstName()) || StringUtils.
            return Mono.error(new ValidationException("First name and l
        }
        return Mono.just(request);
    }

    private boolean isValidEmail(String email) {
        return email != null && email.matches("^[A-Za-z0-9+_.-]+@(.+)$"
    }
}

// Global Reactive Exception Handler
@ControllerAdvice
public class ReactiveExceptionHandler {

    @ExceptionHandler(ValidationException.class)
    public ResponseEntity<ErrorResponse> handleValidation(ValidationExc
        ErrorResponse error = new ErrorResponse("VALIDATION_ERROR", ex.
        return ResponseEntity.badRequest().body(error);
    }

    @ExceptionHandler(UserNotFoundException.class)
    public ResponseEntity<ErrorResponse> handleUserNotFound(UserNotFoun
        ErrorResponse error = new ErrorResponse("USER_NOT_FOUND", ex.ge
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(error);
    }

    @ExceptionHandler(UserAlreadyExistsException.class)
```

```
    public ResponseEntity<ErrorResponse> handleUserAlreadyExists(UserAl
        ErrorResponse error = new ErrorResponse("USER_ALREADY_EXISTS",
        return ResponseEntity.status(HttpStatus.CONFLICT).body(error);
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<ErrorResponse> handleGeneric(Exception ex) {
        ErrorResponse error = new ErrorResponse("INTERNAL_ERROR", "An u
        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).
    }
}
```

Reactive WebClient

```
@Component
public class ReactiveUserClient {

    private final WebClient webClient;

    public ReactiveUserClient(WebClient.Builder webClientBuilder) {
        this.webClient = webClientBuilder
            .baseUrl("http://user-service")
            .build();
    }

    public Mono<User> getUserById(Long id) {
        return webClient.get()
            .uri("/users/{id}", id)
            .retrieve()
            .bodyToMono(User.class);
    }

    public Flux<User> getAllUsers() {
        return webClient.get()
            .uri("/users")
            .retrieve()
            .bodyToFlux(User.class);
    }

    public Mono<User> createUser(CreateUserRequest request) {
        return webClient.post()
            .uri("/users")
            .bodyValue(request)
            .retrieve()
            .bodyToMono(User.class);
    }

    public Mono<User> updateUser(Long id, UpdateUserRequest request) {
        return webClient.put()
            .uri("/users/{id}", id)
            .bodyValue(request)
            .retrieve()
            .bodyToMono(User.class);
```

```
    }

    public Mono<Void> deleteUser(Long id) {
        return webClient.delete()
            .uri("/users/{id}", id)
            .retrieve()
            .bodyToMono(Void.class);
    }

    public Flux<User> searchUsers(String searchTerm) {
        return webClient.get()
            .uri(uriBuilder -> uriBuilder
                .path("/users/search")
                .queryParam("term", searchTerm)
                .build())
            .retrieve()
            .bodyToFlux(User.class);
    }

    public Flux<User> streamUsers() {
        return webClient.get()
            .uri("/users/stream")
            .accept(MediaType.TEXT_EVENT_STREAM)
            .retrieve()
            .bodyToFlux(User.class);
    }
}
```

Reactive Testing

```
@SpringBootTest
@AutoConfigureWebTestClient
class ReactiveUserControllerTest {

    @Autowired
    private WebTestClient webTestClient;

    @Test
    void shouldGetUserById() {
        webTestClient.get()
            .uri("/api/reactive/users/1")
            .exchange()
            .expectStatus().isOk()
            .expectBody(User.class)
            .value(user -> assertEquals("testuser", user.getUsername()))
    }

    @Test
    void shouldGetAllUsers() {
        webTestClient.get()
            .uri("/api/reactive/users")
            .exchange()
            .expectStatus().isOk()
```

```
            .expectBodyList(User.class)
            .hasSize(2);
    }

    @Test
    void shouldCreateUser() {
        CreateUserRequest request = new CreateUserRequest("newuser", "n

        webTestClient.post()
            .uri("/api/reactive/users")
            .bodyValue(request)
            .exchange()
            .expectStatus().isCreated()
            .expectBody(User.class)
            .value(user -> assertEquals("newuser", user.getUsername()))
    }

    @Test
    void shouldStreamUsers() {
        webTestClient.get()
            .uri("/api/reactive/users/stream")
            .accept(MediaType.TEXT_EVENT_STREAM)
            .exchange()
            .expectStatus().isOk()
            .expectHeader().contentType(MediaType.TEXT_EVENT_STREAM)
            .expectBodyList(User.class)
            .hasSize(5);
    }

    @Test
    void shouldReturnNotFoundForNonExistentUser() {
        webTestClient.get()
            .uri("/api/reactive/users/999")
            .exchange()
            .expectStatus().isNotFound();
    }
}
```

Reactive Configuration

```
@Configuration
@EnableWebFlux
public class WebFluxConfig {

    @Bean
    public WebFluxConfigurer webFluxConfigurer() {
        return new WebFluxConfigurer() {
            @Override
            public void addCorsMappings(CorsRegistry registry) {
                registry.addMapping("/api/**")
                    .allowedOrigins("http://localhost:3000")
                    .allowedMethods("GET", "POST", "PUT", "DELETE")
                    .maxAge(Duration.ofHours(1));
```

```
            }

            @Override
            public void configureHttpMessageCodecs(ServerCodecConfigure
                configurer.defaultCodecs().jackson2JsonEncoder(new Jack
                configurer.defaultCodecs().jackson2JsonDecoder(new Jack
            }
        };
    }

    @Bean
    public WebClient.Builder webClientBuilder() {
        return WebClient.builder()
            .codecs(configurer -> configurer.defaultCodecs().maxInMemor
    }

    @Bean
    public ReactiveRedisTemplate<String, Object> reactiveRedisTemplate(
            ReactiveRedisConnectionFactory connectionFactory) {
        Jackson2JsonRedisSerializer<Object> serializer = new Jackson2Js
        StringRedisSerializer stringSerializer = new StringRedisSeriali

        RedisSerializationContext.RedisSerializationContextBuilder<Obje
            RedisSerializationContext.newSerializationContext(stringSer

        builder.key(stringSerializer).value(serializer);
        builder.hashKey(stringSerializer).hashValue(serializer);

        return new ReactiveRedisTemplate<>(connectionFactory, builder.b
    }
}
```

Reactive Best Practices

- **Use non-blocking operations** - Avoid blocking calls in reactive chains
- **Prefer flatMap over map** - For asynchronous operations
- **Handle backpressure** - Use onBackpressureBuffer, onBackpressureDrop, etc.
- **Limit reactive chains** - Keep chains readable and maintainable
- **Use proper error handling** - Handle exceptions with onError, onErrorResume
- **Test reactive streams** - Use StepVerifier for testing
- **Consider thread models** - Understand Schedulers for thread management
- **Use reactive repositories** - Leverage reactive data access patterns

---

22. GraphQL Annotations

GraphQL Core Annotations

```
@Schema                    // GraphQL schema definition
@Type                      // GraphQL type
@Input                     // GraphQL input type
@Query                     // GraphQL query
```

```
@Mutation                // GraphQL mutation
@Subscription            // GraphQL subscription
@Argument                // GraphQL argument
@Field                   // GraphQL field
@NonNull                 # Non-null field
@Deprecated              # Deprecated field
@Description             # Field description
```

GraphQL Schema Definition

```
@Schema(types = {UserResolver.class, PostResolver.class})
public class GraphQLSchemaConfig {

    @Bean
    public GraphQLCustomType dateTimeType() {
        return GraphQLScalarType.newScalar()
            .name("DateTime")
            .description("Java 8 DateTime")
            .coercing(new Coercing<LocalDateTime, String>() {
                @Override
                public String serialize(Object dataFetcherResult) {
                    return dataFetcherResult.toString();
                }

                @Override
                public LocalDateTime parseValue(Object input) {
                    return LocalDateTime.parse(input.toString());
                }

                @Override
                public LocalDateTime parseLiteral(Object input) {
                    return LocalDateTime.parse(((StringValue) input).ge
                }
            })
            .build();
    }
}

@Type("User")
@Description("User entity")
public class UserType {

    @NonNull
    @Field("id")
    @Description("User ID")
    private Long id;

    @NonNull
    @Field("username")
    @Description("Username")
    private String username;

    @Field("email")
```

```
    @Description("Email address")
    private String email;

    @Field("posts")
    @Description("User's posts")
    private List<PostType> posts;

    @Deprecated
    @Field("legacyField")
    @Description("Legacy field - deprecated")
    private String legacyField;

    // Getters and setters
}

@Input("UserInput")
@Description("User input for mutations")
public class UserInputType {

    @NonNull
    @Field("username")
    private String username;

    @NonNull
    @Field("email")
    private String email;

    @Field("firstName")
    private String firstName;

    @Field("lastName")
    private String lastName;

    // Getters and setters
}
```

GraphQL Resolvers

```
@Controller
public class UserResolver {

    private final UserService userService;

    public UserResolver(UserService userService) {
        this.userService = userService;
    }

    @Query("user")
    @Description("Get user by ID")
    public User getUser(@Argument("id") Long id) {
        return userService.findById(id);
    }
```

```
    @Query("users")
    @Description("Get all users")
    public List<User> getUsers() {
        return userService.findAll();
    }

    @Query("searchUsers")
    @Description("Search users by term")
    public List<User> searchUsers(@Argument("term") String term) {
        return userService.search(term);
    }

    @Mutation("createUser")
    @Description("Create a new user")
    public User createUser(@Argument("input") UserInput input) {
        return userService.create(input);
    }

    @Mutation("updateUser")
    @Description("Update an existing user")
    public User updateUser(@Argument("id") Long id, @Argument("input")
        return userService.update(id, input);
    }

    @Mutation("deleteUser")
    @Description("Delete a user")
    public Boolean deleteUser(@Argument("id") Long id) {
        return userService.deleteById(id);
    }

    @Subscription("userCreated")
    @Description("Subscribe to user creation events")
    public Publisher<User> userCreated() {
        return userService.getUserCreatedEvents();
    }

    @Subscription("userUpdated")
    @Description("Subscribe to user update events")
    public Publisher<User> userUpdated(@Argument("userId") Long userId)
        return userService.getUserUpdatedEvents(userId);
    }
}

@Controller
public class PostResolver {

    private final PostService postService;

    public PostResolver(PostService postService) {
        this.postService = postService;
    }

    @Query("post")
```

```
    public Post getPost(@Argument("id") Long id) {
        return postService.findById(id);
    }

    @Query("posts")
    public List<Post> getPosts(@Argument("userId") Long userId) {
        return postService.findByUserId(userId);
    }

    @Mutation("createPost")
    public Post createPost(@Argument("input") PostInput input) {
        return postService.create(input);
    }

    @Subscription("postCreated")
    public Publisher<Post> postCreated() {
        return postService.getPostCreatedEvents();
    }
}
```

GraphQL Configuration

```
@Configuration
@EnableGraphQL
public class GraphQLConfig {

    @Bean
    public GraphQLExecutionStrategy executionStrategy() {
        return new AsyncExecutionStrategy();
    }

    @Bean
    public GraphQLSource graphQLSource(GraphQLResourceResolver resource
        return new SchemaResourceBuilder()
            .schemaResources("schema.graphqls")
            .resolvers(userResolver(), postResolver())
            .build();
    }

    @Bean
    public GraphQLResourceResolver resourceResolver() {
        return new DefaultResourceResolver();
    }

    @Bean
    public DataLoaderOptions dataLoaderOptions() {
        return DataLoaderOptions.newOptions()
            .setMaxBatchSize(100)
            .setBatchingEnabled(true)
            .setCachingEnabled(true);
    }

    @Bean
```

```
    public DataLoader<Long, User> userDataLoader(UserService userServic
        DataLoader<Long, User> dataLoader = DataLoaderFactory.newDataLo
            Flux.fromIterable(userIds)
                .flatMap(userService::findById)
                .collectList()
                .map(users -> {
                    Map<Long, User> userMap = new HashMap<>();
                    users.forEach(user -> userMap.put(user.getId(), use
                    return userIds.stream()
                        .map(userMap::get)
                        .collect(Collectors.toList());
                })
        );

        return dataLoader;
    }
}
```

---

23. REST Microservices Annotations

Microservice Core Annotations

```
@SpringBootApplication          // Spring Boot application
@EnableDiscoveryClient     // Service discovery
@EnableConfigClient       // Configuration client
@RefreshScope             // Dynamic configuration refresh
@LoadBalanced             // Client-side load balancing
@CircuitBreaker           // Circuit breaker
@Retry                    // Retry mechanism
@TimeLimiter              // Timeout mechanism
```

Service Discovery & Registration

```
@SpringBootApplication
@EnableDiscoveryClient
@EnableConfigClient
public class UserServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(UserServiceApplication.class, args);
    }
}

@RestController
@RequestMapping("/api/users")
@RefreshScope
public class UserController {

    @Value("${app.user.default.role:USER}")
    private String defaultRole;

    @Autowired
```

```
    @LoadBalanced
    private RestTemplate restTemplate;

    @GetMapping("/{id}")
    public User getUser(@PathVariable Long id) {
        return userService.findById(id);
    }

    @PostMapping
    @CircuitBreaker(fallbackMethod = "createUserFallback")
    @Retry(maxAttempts = 3)
    public User createUser(@Valid @RequestBody CreateUserRequest reques
        return userService.create(request);
    }

    public User createUserFallback(CreateUserRequest request, Exception
        log.error("Fallback for user creation", ex);
        return new User(null, request.getUsername(), defaultRole);
    }
}
```

API Gateway Configuration

```
@SpringBootApplication
@EnableZuulProxy
@EnableDiscoveryClient
public class ApiGatewayApplication {
    public static void main(String[] args) {
        SpringApplication.run(ApiGatewayApplication.class, args);
    }
}

@Component
public class CustomZuulFilter extends ZuulFilter {

    @Override
    public String filterType() {
        return "pre";
    }

    @Override
    public int filterOrder() {
        return 1;
    }

    @Override
    public boolean shouldFilter() {
        return true;
    }

    @Override
    public Object run() throws ZuulException {
        RequestContext ctx = RequestContext.getCurrentContext();
```

```
        ctx.addZuulRequestHeader("X-Request-ID", UUID.randomUUID().toSt
        return null;
    }
}
```

Circuit Breaker Configuration

```
@Configuration
@EnableCircuitBreaker
public class CircuitBreakerConfig {

    @Bean
    public CircuitBreaker circuitBreaker() {
        return CircuitBreaker.ofDefaults("userService");
    }

    @Bean
    public TimeLimiter timeLimiter() {
        return TimeLimiter.of(Duration.ofSeconds(5));
    }

    @Bean
    public Bulkhead bulkhead() {
        return Bulkhead.ofDefaults("userService");
    }
}

@Service
public class ExternalServiceClient {

    @CircuitBreaker(fallbackMethod = "fallbackMethod")
    @Retry(maxAttempts = 3, backoff = @Backoff(delay = 1000))
    @TimeLimiter(name = "externalService")
    public String callExternalService(String request) {
        return restTemplate.postForObject(
            "http://external-service/api/process",
            request,
            String.class
        );
    }

    public String fallbackMethod(String request, Exception ex) {
        log.error("Fallback triggered for external service call", ex);
        return "Default response";
    }
}
```

Distributed Configuration

```
@Configuration
@EnableConfigServer
public class ConfigServerApplication {
    public static void main(String[] args) {
```

```
        SpringApplication.run(ConfigServerApplication.class, args);
    }
}

@RefreshScope
@ConfigurationProperties(prefix = "app")
public class AppConfig {
    private String name;
    private String version;
    private Map<String, String> features;

    // Getters and setters
}

@RestController
@RequestMapping("/api/config")
@RefreshScope
public class ConfigController {

    @Autowired
    private AppConfig appConfig;

    @GetMapping
    public Map<String, Object> getConfig() {
        return Map.of(
            "name", appConfig.getName(),
            "version", appConfig.getVersion(),
            "features", appConfig.getFeatures()
        );
    }
}
```

---

24. gRPC Annotations

gRPC Core Annotations

```
@GrpcService             // gRPC service annotation
@GrpcClient              // gRPC client annotation
@GrpcAdvice              // gRPC advice
@GrpcExceptionHandler   // gRPC exception handler
```

gRPC Service Definition

```
// Proto file: user.proto
syntax = "proto3";

package user;

service UserService {
    rpc GetUser(GetUserRequest) returns (GetUserResponse);
    rpc ListUsers(ListUsersRequest) returns (stream GetUsersResponse);
```

```
    rpc CreateUser(CreateUserRequest) returns (CreateUserResponse);
    rpc UpdateUser(stream UpdateUserRequest) returns (UpdateUserRespons
}

message GetUserRequest {
    int64 user_id = 1;
}

message GetUserResponse {
    User user = 1;
}

message User {
    int64 id = 1;
    string username = 2;
    string email = 3;
    string first_name = 4;
    string last_name = 5;
}
```

gRPC Service Implementation

```
@GrpcService
public class UserGrpcService extends UserServiceGrpc.UserServiceImplBas

    private final UserService userService;

    public UserGrpcService(UserService userService) {
        this.userService = userService;
    }

    @Override
    public void getUser(GetUserRequest request, StreamObserver<GetUserR
        try {
            User user = userService.findById(request.getUserId());
            GetUserResponse response = GetUserResponse.newBuilder()
                .setUser(convertToGrpcUser(user))
                .build();

            responseObserver.onNext(response);
            responseObserver.onCompleted();
        } catch (Exception e) {
            responseObserver.onError(Status.INTERNAL
                .withDescription("Error getting user: " + e.getMessage(
                .asRuntimeException());
        }
    }

    @Override
    public void listUsers(ListUsersRequest request, StreamObserver<GetU
        try {
            List<User> users = userService.findAll();
            users.stream()
```

```
            .map(user -> GetUsersResponse.newBuilder()
                .setUser(convertToGrpcUser(user))
                .build())
            .forEach(responseObserver::onNext);

        responseObserver.onCompleted();
    } catch (Exception e) {
        responseObserver.onError(Status.INTERNAL
            .withDescription("Error listing users: " + e.getMessage
            .asRuntimeException());
    }
}

@Override
public StreamObserver<UpdateUserRequest> updateUser(StreamObserver<
    return new StreamObserver<UpdateUserRequest>() {
        private List<User> users = new ArrayList<>();

        @Override
        public void onNext(UpdateUserRequest request) {
            User user = userService.update(request.getUser().getId(
            users.add(user);
        }

        @Override
        public void onError(Throwable t) {
            log.error("Error in updateUser stream", t);
        }

        @Override
        public void onCompleted() {
            UpdateUserResponse response = UpdateUserResponse.newBui
                .setSuccess(true)
                .setUpdatedCount(users.size())
                .build();
            responseObserver.onNext(response);
            responseObserver.onCompleted();
        }
    };
}

private com.example.grpc.User convertToGrpcUser(User user) {
    return com.example.grpc.User.newBuilder()
        .setId(user.getId())
        .setUsername(user.getUsername())
        .setEmail(user.getEmail())
        .setFirstName(user.getFirstName())
        .setLastName(user.getLastName())
        .build();
}

private User convertFromGrpcUser(com.example.grpc.User grpcUser) {
    User user = new User();
```

```
        user.setId(grpcUser.getId());
        user.setUsername(grpcUser.getUsername());
        user.setEmail(grpcUser.getEmail());
        user.setFirstName(grpcUser.getFirstName());
        user.setLastName(grpcUser.getLastName());
        return user;
    }
}
```

gRPC Client Configuration

```
@Configuration
public class GrpcClientConfig {

    @Bean
    public ManagedChannel userGrpcChannel() {
        return ManagedChannelBuilder.forAddress("localhost", 9090)
            .usePlaintext()
            .build();
    }

    @Bean
    @GrpcClient("userGrpcChannel")
    public UserServiceGrpc.UserServiceBlockingStub userServiceBlockingS
            ManagedChannel channel) {
        return UserServiceGrpc.newBlockingStub(channel);
    }

    @Bean
    @GrpcClient("userGrpcChannel")
    public UserServiceGrpc.UserServiceStub userServiceStub(ManagedChann
        return UserServiceGrpc.newStub(channel);
    }
}

@Service
public class GrpcUserClient {

    private final UserServiceGrpc.UserServiceBlockingStub blockingStub;
    private final UserServiceGrpc.UserServiceStub asyncStub;

    public GrpcUserClient(
            UserServiceGrpc.UserServiceBlockingStub blockingStub,
            UserServiceGrpc.UserServiceStub asyncStub) {
        this.blockingStub = blockingStub;
        this.asyncStub = asyncStub;
    }

    public com.example.grpc.User getUser(Long userId) {
        GetUserRequest request = GetUserRequest.newBuilder()
            .setUserId(userId)
            .build();
```

```
        GetUserResponse response = blockingStub.getUser(request);
        return response.getUser();
    }

    public List<com.example.grpc.User> listUsers() {
        ListUsersRequest request = ListUsersRequest.newBuilder().build(
        List<com.example.grpc.User> users = new ArrayList<>();

        Iterator<GetUsersResponse> iterator = blockingStub.listUsers(re
        while (iterator.hasNext()) {
            users.add(iterator.next().getUser());
        }

        return users;
    }

    public CompletableFuture<com.example.grpc.User> getUserAsync(Long u
        GetUserRequest request = GetUserRequest.newBuilder()
            .setUserId(userId)
            .build();

        CompletableFuture<com.example.grpc.User> future = new Completab

        asyncStub.getUser(request, new StreamObserver<GetUserResponse>(
            @Override
            public void onNext(GetUserResponse response) {
                future.complete(response.getUser());
            }

            @Override
            public void onError(Throwable t) {
                future.completeExceptionally(t);
            }

            @Override
            public void onCompleted() {
                if (!future.isDone()) {
                    future.completeExceptionally(new RuntimeException("
                }
            }
        });

        return future;
    }
}
```

gRPC Interceptors

```
@Component
public class GrpcLoggingInterceptor implements ServerInterceptor {

    private static final Logger log = LoggerFactory.getLogger(GrpcLoggi
```

```
    @Override
    public <ReqT, RespT> ServerCall.Listener<ReqT> interceptCall(
            ServerCall<ReqT, RespT> call,
            Metadata headers,
            ServerCallHandler<ReqT, RespT> next) {

        log.info("gRPC call: {} {}", call.getMethodDescriptor().getFull
                call.getAttributes().get(Grpc.TRANSPORT_ATTR_REMOTE_ADD

        return new ForwardingServerCallListener.SimpleForwardingServerC
                next.startCall(call, headers)) {

            @Override
            public void onMessage(ReqT message) {
                log.debug("gRPC request: {}", message);
                super.onMessage(message);
            }

            @Override
            public void onComplete() {
                log.info("gRPC call completed: {}", call.getMethodDescr
                super.onComplete();
            }
        };
    }
}

@Component
public class GrpcAuthInterceptor implements ServerInterceptor {

    @Override
    public <ReqT, RespT> ServerCall.Listener<ReqT> interceptCall(
            ServerCall<ReqT, RespT> call,
            Metadata headers,
            ServerCallHandler<ReqT, RespT> next) {

        String token = headers.get(Metadata.Key.of("authorization", Met

        if (!isValidToken(token)) {
            call.close(Status.UNAUTHENTICATED.withDescription("Invalid
            return new ServerCall.Listener<ReqT>() {};
        }

        return next.startCall(call, headers);
    }

    private boolean isValidToken(String token) {
        // Token validation logic
        return token != null && token.startsWith("Bearer ");
    }
}
```

gRPC Configuration

```
@Configuration
public class GrpcServerConfig {

    @Bean
    public Server server(UserGrpcService userGrpcService,
                         GrpcLoggingInterceptor loggingInterceptor,
                         GrpcAuthInterceptor authInterceptor) {

        return ServerBuilder.forPort(9090)
            .addService(ServerInterceptors.intercept(userGrpcService, l
            .build();
    }

    @Bean
    public GrpcServerProperties grpcServerProperties() {
        GrpcServerProperties properties = new GrpcServerProperties();
        properties.setPort(9090);
        properties.setAddress("0.0.0.0");
        return properties;
    }

    @PreDestroy
    public void destroyServer(Server server) throws InterruptedExceptio
        server.shutdown().awaitTermination(5, TimeUnit.SECONDS);
    }
}
```

REST to gRPC Bridge

```
@RestController
@RequestMapping("/api/grpc/users")
public class GrpcBridgeController {

    private final GrpcUserClient grpcUserClient;

    public GrpcBridgeController(GrpcUserClient grpcUserClient) {
        this.grpcUserClient = grpcUserClient;
    }

    @GetMapping("/{id}")
    public ResponseEntity<User> getUser(@PathVariable Long id) {
        try {
            com.example.grpc.User grpcUser = grpcUserClient.getUser(id)
            User user = convertFromGrpcUser(grpcUser);
            return ResponseEntity.ok(user);
        } catch (Exception e) {
            return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERR
        }
    }

    @GetMapping
    public ResponseEntity<List<User>> getAllUsers() {
        try {
```

```
            List<com.example.grpc.User> grpcUsers = grpcUserClient.list
            List<User> users = grpcUsers.stream()
                .map(this::convertFromGrpcUser)
                .collect(Collectors.toList());
            return ResponseEntity.ok(users);
        } catch (Exception e) {
            return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERR
        }
    }

    @GetMapping("/{id}/async")
    public CompletableFuture<ResponseEntity<User>> getUserAsync(@PathVa
        return grpcUserClient.getUserAsync(id)
            .thenApply(grpcUser -> ResponseEntity.ok(convertFromGrpcUse
            .exceptionally(ex -> ResponseEntity.status(HttpStatus.INTER
    }

    private User convertFromGrpcUser(com.example.grpc.User grpcUser) {
        User user = new User();
        user.setId(grpcUser.getId());
        user.setUsername(grpcUser.getUsername());
        user.setEmail(grpcUser.getEmail());
        user.setFirstName(grpcUser.getFirstName());
        user.setLastName(grpcUser.getLastName());
        return user;
    }
}
```

---

25. Resilience4j Annotations

Resilience4j Core Annotations

```
@CircuitBreaker         // Circuit breaker pattern
@Retry                  // Retry mechanism
@RateLimiter            // Rate limiting
@Bulkhead               // Bulkhead pattern (concurrency control)
@TimeLimiter            // Timeout control
@Fallback               // Fallback method
```

Circuit Breaker Configuration

```
@Configuration
@EnableCircuitBreaker
public class Resilience4jConfig {

    @Bean
    public CircuitBreakerRegistry circuitBreakerRegistry() {
        CircuitBreakerConfig config = CircuitBreakerConfig.custom()
            .failureRateThreshold(50)
            .waitDurationInOpenState(Duration.ofSeconds(30))
            .slidingWindowSize(10)
```

```
        .minimumNumberOfCalls(5)
        .permittedNumberOfCallsInHalfOpenState(3)
        .automaticTransitionFromOpenToHalfOpenEnabled(true)
        .recordExceptions(IOException.class, TimeoutException.class
        .ignoreExceptions(BusinessException.class)
        .build();

    return CircuitBreakerRegistry.of(config);
}

@Bean
public RetryRegistry retryRegistry() {
    RetryConfig config = RetryConfig.custom()
        .maxAttempts(3)
        .waitDuration(Duration.ofMillis(1000))
        .retryExceptions(IOException.class, TimeoutException.class)
        .ignoreExceptions(BusinessException.class)
        .retryOnException(ex -> ex instanceof IOException)
        .build();

    return RetryRegistry.of(config);
}

@Bean
public RateLimiterRegistry rateLimiterRegistry() {
    RateLimiterConfig config = RateLimiterConfig.custom()
        .limitForPeriod(10)
        .limitRefreshPeriod(Duration.ofSeconds(1))
        .timeoutDuration(Duration.ofSeconds(5))
        .build();

    return RateLimiterRegistry.of(config);
}

@Bean
public BulkheadRegistry bulkheadRegistry() {
    BulkheadConfig config = BulkheadConfig.custom()
        .maxConcurrentCalls(10)
        .maxWaitDuration(Duration.ofSeconds(5))
        .build();

    return BulkheadRegistry.of(config);
}

@Bean
public TimeLimiterRegistry timeLimiterRegistry() {
    TimeLimiterConfig config = TimeLimiterConfig.custom()
        .timeoutDuration(Duration.ofSeconds(3))
        .cancelRunningFuture(true)
        .build();

    return TimeLimiterRegistry.of(config);
}
```

```
}
```

Resilience4j Service Implementation

```
@Service
public class ResilientUserService {

    private final UserServiceClient userServiceClient;
    private final CircuitBreaker circuitBreaker;
    private final Retry retry;
    private final RateLimiter rateLimiter;
    private final Bulkhead bulkhead;
    private final TimeLimiter timeLimiter;

    public ResilientUserService(UserServiceClient userServiceClient,
                                CircuitBreakerRegistry circuitBreakerReg
                                RetryRegistry retryRegistry,
                                RateLimiterRegistry rateLimiterRegistry,
                                BulkheadRegistry bulkheadRegistry,
                                TimeLimiterRegistry timeLimiterRegistry)
        this.userServiceClient = userServiceClient;
        this.circuitBreaker = circuitBreakerRegistry.circuitBreaker("us
        this.retry = retryRegistry.retry("userService");
        this.rateLimiter = rateLimiterRegistry.rateLimiter("userService
        this.bulkhead = bulkheadRegistry.bulkhead("userService");
        this.timeLimiter = timeLimiterRegistry.timeLimiter("userService
    }

    @CircuitBreaker(name = "userService", fallbackMethod = "getUserFall
    @Retry(name = "userService")
    @RateLimiter(name = "userService")
    @Bulkhead(name = "userService")
    @TimeLimiter(name = "userService")
    public CompletableFuture<User> getUserAsync(Long userId) {
        return CompletableFuture.supplyAsync(() -> {
            return userServiceClient.getUser(userId);
        });
    }

    @CircuitBreaker(name = "userService", fallbackMethod = "createUserF
    @Retry(name = "userService")
    @RateLimiter(name = "userService")
    public User createUser(CreateUserRequest request) {
        return userServiceClient.createUser(request);
    }

    public CompletableFuture<User> getUserFallback(Long userId, Excepti
        log.error("Fallback triggered for getUser: {}", userId, ex);
        return CompletableFuture.completedAsync(getDefaultUser(userId))
    }

    public User createUserFallback(CreateUserRequest request, Exception
        log.error("Fallback triggered for createUser: {}", request.getU
```

```
        return getDefaultUser(request.getUsername());
    }

    private User getDefaultUser(Long userId) {
        User user = new User();
        user.setId(userId);
        user.setUsername("default");
        user.setEmail("default@example.com");
        return user;
    }

    private User getDefaultUser(String username) {
        User user = new User();
        user.setUsername(username);
        user.setEmail(username + "@default.com");
        return user;
    }
}

// Annotation-based approach
@Service
public class AnnotationBasedUserService {

    private final UserServiceClient userServiceClient;

    public AnnotationBasedUserService(UserServiceClient userServiceClie
        this.userServiceClient = userServiceClient;
    }

    @CircuitBreaker(name = "userService", fallbackMethod = "getUserFall
    @Retry(name = "userService")
    @RateLimiter(name = "userService")
    public User getUser(Long userId) {
        return userServiceClient.getUser(userId);
    }

    @CircuitBreaker(name = "userService", fallbackMethod = "createUserF
    @Retry(name = "userService")
    @Bulkhead(name = "userService")
    public User createUser(CreateUserRequest request) {
        return userServiceClient.createUser(request);
    }

    public User getUserFallback(Long userId, Exception ex) {
        log.error("Fallback for getUser: {}", userId, ex);
        return getDefaultUser(userId);
    }

    public User createUserFallback(CreateUserRequest request, Exception
        log.error("Fallback for createUser: {}", request.getUsername(),
        return getDefaultUser(request.getUsername());
    }
```

```
    private User getDefaultUser(Long userId) {
        User user = new User();
        user.setId(userId);
        user.setUsername("default");
        user.setEmail("default@example.com");
        return user;
    }

    private User getDefaultUser(String username) {
        User user = new User();
        user.setUsername(username);
        user.setEmail(username + "@default.com");
        return user;
    }
}
```

Resilience4j Configuration Properties

```
# application.yml
resilience4j:
  circuitbreaker:
    instances:
      userService:
        failureRateThreshold: 50
        waitDurationInOpenState: 30s
        slidingWindowSize: 10
        minimumNumberOfCalls: 5
        permittedNumberOfCallsInHalfOpenState: 3
        automaticTransitionFromOpenToHalfOpenEnabled: true
        recordExceptions:
          - java.io.IOException
          - java.util.concurrent.TimeoutException
        ignoreExceptions:
          - com.example.BusinessException

  retry:
    instances:
      userService:
        maxAttempts: 3
        waitDuration: 1s
        retryExceptions:
          - java.io.IOException
          - java.util.concurrent.TimeoutException
        ignoreExceptions:
          - com.example.BusinessException

  ratelimiter:
    instances:
      userService:
        limitForPeriod: 10
        limitRefreshPeriod: 1s
        timeoutDuration: 5s
```

```
  bulkhead:
    instances:
      userService:
        maxConcurrentCalls: 10
        maxWaitDuration: 5s

  timelimiter:
    instances:
      userService:
        timeoutDuration: 3s
        cancelRunningFuture: true
```

---

26. Microservices Design Patterns

API Gateway Pattern

```
@SpringBootApplication
@EnableZuulProxy
@EnableDiscoveryClient
public class ApiGatewayApplication {
    public static void main(String[] args) {
        SpringApplication.run(ApiGatewayApplication.class, args);
    }
}

@Component
public class GatewayFilter extends ZuulFilter {

    @Override
    public String filterType() {
        return "pre";
    }

    @Override
    public int filterOrder() {
        return 1;
    }

    @Override
    public boolean shouldFilter() {
        return true;
    }

    @Override
    public Object run() throws ZuulException {
        RequestContext ctx = RequestContext.getCurrentContext();
        ctx.addZuulRequestHeader("X-Request-ID", UUID.randomUUID().toSt
        ctx.addZuulRequestHeader("X-Source", "api-gateway");
        return null;
    }
}
```

```
// Spring Cloud Gateway
@SpringBootApplication
public class SpringCloudGatewayApplication {
    public static void main(String[] args) {
        SpringApplication.run(SpringCloudGatewayApplication.class, args
    }
}

@Configuration
public class GatewayConfig {

    @Bean
    public RouteLocator customRouteLocator(RouteLocatorBuilder builder)
        return builder.routes()
            .route("user-service", r -> r.path("/api/users/**")
                .filters(f -> f.filter(filter))
                .uri("lb://user-service"))
            .route("order-service", r -> r.path("/api/orders/**")
                .filters(f -> f.filter(filter))
                .uri("lb://order-service"))
            .build();
    }

    @Bean
    public GlobalFilter customGlobalFilter() {
        return (exchange, chain) -> {
            exchange.getRequest().getHeaders().add("X-Request-ID", UUID
            return chain.filter(exchange);
        };
    }
}
```

Service Registry Pattern

```
// Eureka Server
@SpringBootApplication
@EnableEurekaServer
public class EurekaServerApplication {
    public static void main(String[] args) {
        SpringApplication.run(EurekaServerApplication.class, args);
    }
}

// Eureka Client
@SpringBootApplication
@EnableEurekaClient
public class UserServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(UserServiceApplication.class, args);
    }
}
```

```
@Service
public class ServiceDiscoveryClient {

    @Autowired
    private DiscoveryClient discoveryClient;

    public List<ServiceInstance> getInstances(String serviceName) {
        return discoveryClient.getInstances(serviceName);
    }

    public List<String> getAllServices() {
        return discoveryClient.getServices();
    }

    @LoadBalanced
    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}
```

Circuit Breaker Pattern

```
@Service
public class CircuitBreakerService {

    private final CircuitBreaker circuitBreaker;

    public CircuitBreakerService(CircuitBreakerRegistry registry) {
        this.circuitBreaker = registry.circuitBreaker("externalService"
    }

    public String callExternalService(String request) {
        Supplier<String> supplier = CircuitBreaker
            .decorateSupplier(circuitBreaker, () -> externalServiceCall

        try {
            return supplier.get();
        } catch (Exception e) {
            return "Fallback response";
        }
    }

    private String externalServiceCall(String request) {
        // External service call
        return "Response from external service";
    }
}

// Hystrix alternative
@Component
public class HystrixService {
```

```
    @HystrixCommand(fallbackMethod = "fallbackMethod",
        commandProperties = {
            @HystrixProperty(name = "execution.isolation.thread.timeout
            @HystrixProperty(name = "circuitBreaker.requestVolumeThresh
            @HystrixProperty(name = "circuitBreaker.sleepWindowInMillis
            @HystrixProperty(name = "circuitBreaker.errorThresholdPerce
        })
    public String callService(String request) {
        return externalServiceCall(request);
    }

    public String fallbackMethod(String request) {
        return "Fallback response";
    }

    private String externalServiceCall(String request) {
        // External service call
        return "Response";
    }
}
```

Saga Pattern

```
@Service
public class OrderSagaOrchestrator {

    @Autowired
    private UserServiceClient userServiceClient;

    @Autowired
    private PaymentServiceClient paymentServiceClient;

    @Autowired
    private InventoryServiceClient inventoryServiceClient;

    @Autowired
    private NotificationServiceClient notificationServiceClient;

    @Transactional
    public OrderResponse processOrder(OrderRequest request) {
        try {
            // Step 1: Validate user
            User user = userServiceClient.getUser(request.getUserId());

            // Step 2: Reserve inventory
            InventoryReservation reservation = inventoryServiceClient.r

            // Step 3: Process payment
            Payment payment = paymentServiceClient.processPayment(reque

            // Step 4: Confirm inventory
            inventoryServiceClient.confirmReservation(reservation.getId
```

```
            // Step 5: Create order
            Order order = createOrder(request, user, payment);

            // Step 6: Send notification
            notificationServiceClient.sendOrderConfirmation(user.getEma

            return OrderResponse.success(order);

        } catch (Exception e) {
            // Compensating transactions
            compensateOrder(request);
            throw new OrderProcessingException("Order processing failed
        }
    }

    private void compensateOrder(OrderRequest request) {
        try {
            // Compensate payment
            paymentServiceClient.refundPayment(request.getPaymentInfo()

            // Release inventory
            inventoryServiceClient.releaseReservation(request.getItems(

            // Send failure notification
            User user = userServiceClient.getUser(request.getUserId());
            notificationServiceClient.sendOrderFailure(user.getEmail(),

        } catch (Exception e) {
            log.error("Compensation failed", e);
        }
    }
}
```

Event-Driven Architecture

```
// Event Publisher
@Service
public class EventPublisherService {

    @Autowired
    private KafkaTemplate<String, Object> kafkaTemplate;

    @EventListener
    public void handleUserCreated(UserCreatedEvent event) {
        UserEvent userEvent = new UserEvent();
        userEvent.setEventType("USER_CREATED");
        userEvent.setUserId(event.getUser().getId());
        userEvent.setTimestamp(Instant.now());

        kafkaTemplate.send("user-events", userEvent);
    }

    @EventListener
```

```
    public void handleOrderCreated(OrderCreatedEvent event) {
        OrderEvent orderEvent = new OrderEvent();
        orderEvent.setEventType("ORDER_CREATED");
        orderEvent.setOrderId(event.getOrder().getId());
        orderEvent.setUserId(event.getOrder().getUserId());
        orderEvent.setTimestamp(Instant.now());

        kafkaTemplate.send("order-events", orderEvent);
    }
}

// Event Consumer
@Service
public class EventConsumerService {

    @KafkaListener(topics = "user-events")
    public void handleUserEvent(UserEvent event) {
        switch (event.getEventType()) {
            case "USER_CREATED":
                handleUserCreated(event);
                break;
            case "USER_UPDATED":
                handleUserUpdated(event);
                break;
            case "USER_DELETED":
                handleUserDeleted(event);
                break;
        }
    }

    @KafkaListener(topics = "order-events")
    public void handleOrderEvent(OrderEvent event) {
        switch (event.getEventType()) {
            case "ORDER_CREATED":
                handleOrderCreated(event);
                break;
            case "ORDER_PAID":
                handleOrderPaid(event);
                break;
            case "ORDER_SHIPPED":
                handleOrderShipped(event);
                break;
        }
    }
}
```

CQRS Pattern

```
// Command Side
@Service
public class UserCommandService {

    @Autowired
```

```
    private UserEventStore eventStore;

    @Autowired
    private ApplicationEventPublisher eventPublisher;

    public void createUser(CreateUserCommand command) {
        User user = new User();
        user.setId(UUID.randomUUID());
        user.setUsername(command.getUsername());
        user.setEmail(command.getEmail());

        UserCreatedEvent event = new UserCreatedEvent(user);
        eventStore.saveEvent(event);
        eventPublisher.publishEvent(event);
    }

    public void updateUser(UpdateUserCommand command) {
        User user = getUserById(command.getId());
        user.setUsername(command.getUsername());
        user.setEmail(command.getEmail());

        UserUpdatedEvent event = new UserUpdatedEvent(user);
        eventStore.saveEvent(event);
        eventPublisher.publishEvent(event);
    }
}

// Query Side
@Service
public class UserQueryService {

    @Autowired
    private UserReadRepository readRepository;

    public User getUserById(UUID id) {
        return readRepository.findById(id);
    }

    public List<User> getAllUsers() {
        return readRepository.findAll();
    }

    @EventListener
    public void handleUserCreated(UserCreatedEvent event) {
        User user = new User();
        user.setId(event.getUser().getId());
        user.setUsername(event.getUser().getUsername());
        user.setEmail(event.getUser().getEmail());
        readRepository.save(user);
    }

    @EventListener
    public void handleUserUpdated(UserUpdatedEvent event) {
```

```
        User user = readRepository.findById(event.getUser().getId());
        user.setUsername(event.getUser().getUsername());
        user.setEmail(event.getUser().getEmail());
        readRepository.save(user);
    }
}
```

Event Sourcing Pattern

```
@Service
public class EventSourcingService {

    @Autowired
    private EventStore eventStore;

    public void saveEvent(UUID aggregateId, Object event) {
        EventRecord record = new EventRecord();
        record.setAggregateId(aggregateId);
        record.setEventData(event);
        record.setTimestamp(Instant.now());
        record.setVersion(getNextVersion(aggregateId));

        eventStore.save(record);
    }

    public List<Object> getEvents(UUID aggregateId) {
        return eventStore.findByAggregateId(aggregateId)
            .stream()
            .map(EventRecord::getEventData)
            .collect(Collectors.toList());
    }

    public User rebuildUser(UUID userId) {
        List<Object> events = getEvents(userId);
        User user = new User();

        for (Object event : events) {
            if (event instanceof UserCreatedEvent) {
                UserCreatedEvent createdEvent = (UserCreatedEvent) even
                user.setId(createdEvent.getUser().getId());
                user.setUsername(createdEvent.getUser().getUsername());
                user.setEmail(createdEvent.getUser().getEmail());
            } else if (event instanceof UserUpdatedEvent) {
                UserUpdatedEvent updatedEvent = (UserUpdatedEvent) even
                user.setUsername(updatedEvent.getUser().getUsername());
                user.setEmail(updatedEvent.getUser().getEmail());
            }
        }

        return user;
    }
}
```

Distributed Tracing Pattern

```
@Configuration
@EnableZipkinServer
public class ZipkinServerConfig {
    // Zipkin server configuration
}

@SpringBootApplication
@EnableZipkinClient
public class UserServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(UserServiceApplication.class, args);
    }
}

@RestController
public class TracingController {

    @NewSpan("process-user")
    @SpanTag("userId")
    public User processUser(@SpanTag("user.id") Long userId) {
        log.info("Processing user: {}", userId);
        return userService.process(userId);
    }

    @NewSpan("create-user")
    public User createUser(@SpanTag("user.username") String username) {
        log.info("Creating user: {}", username);
        return userService.create(username);
    }
}

@Component
public class TracingInterceptor implements HandlerInterceptor {

    @Autowired
    private Tracer tracer;

    @Override
    public boolean preHandle(HttpServletRequest request, HttpServletRes
        Span span = tracer.nextSpan()
            .name(request.getMethod() + " " + request.getRequestURI())
            .tag("http.method", request.getMethod())
            .tag("http.url", request.getRequestURI())
            .start();

        try (Tracer.SpanInScope ws = tracer.withSpanInScope(span)) {
            return true;
        } finally {
            span.end();
        }
    }
```

```
}
```

Configuration Management Pattern

```
// Config Server
@SpringBootApplication
@EnableConfigServer
public class ConfigServerApplication {
    public static void main(String[] args) {
        SpringApplication.run(ConfigServerApplication.class, args);
    }
}

// Config Client
@SpringBootApplication
@EnableConfigClient
public class UserServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(UserServiceApplication.class, args);
    }
}

@RestController
@RefreshScope
public class ConfigController {

    @Value("${app.user.default.role:USER}")
    private String defaultRole;

    @Value("${app.user.max.login.attempts:3}")
    private int maxLoginAttempts;

    @GetMapping("/config")
    public Map<String, Object> getConfig() {
        return Map.of(
            "defaultRole", defaultRole,
            "maxLoginAttempts", maxLoginAttempts
        );
    }
}
```

---

27. System Design Patterns & Principles

SOLID Principles Implementation

```
// Single Responsibility Principle (SRP)
@Service
public class UserValidator {
    public void validate(CreateUserRequest request) {
        if (request.getUsername() == null || request.getUsername().leng
            throw new ValidationException("Username must be at least 3
```

```
        }
        if (!isValidEmail(request.getEmail())) {
            throw new ValidationException("Invalid email format");
        }
    }

    private boolean isValidEmail(String email) {
        return email != null && email.matches("^[A-Za-z0-9+_.-]+@(.+)$"
    }
}

@Service
public class UserPersistenceService {
    private final UserRepository userRepository;

    public User save(User user) {
        return userRepository.save(user);
    }

    public User findById(Long id) {
        return userRepository.findById(id);
    }
}

// Open/Closed Principle (OCP)
public interface PaymentProcessor {
    boolean process(Payment payment);
}

@Component
public class CreditCardProcessor implements PaymentProcessor {
    @Override
    public boolean process(Payment payment) {
        // Credit card processing logic
        return true;
    }
}

@Component
public class PayPalProcessor implements PaymentProcessor {
    @Override
    public boolean process(Payment payment) {
        // PayPal processing logic
        return true;
    }
}

@Service
public class PaymentService {
    private final List<PaymentProcessor> processors;

    public PaymentService(List<PaymentProcessor> processors) {
        this.processors = processors;
```

```
    }

    public boolean processPayment(Payment payment) {
        return processors.stream()
            .filter(processor -> processor.supports(payment.getType()))
            .findFirst()
            .map(processor -> processor.process(payment))
            .orElse(false);
    }
}

// Liskov Substitution Principle (LSP)
public abstract class Bird {
    public abstract void fly();
    public abstract void makeSound();
}

@Component
public class Sparrow extends Bird {
    @Override
    public void fly() {
        System.out.println("Sparrow flying");
    }

    @Override
    public void makeSound() {
        System.out.println("Sparrow chirping");
    }
}

// Interface Segregation Principle (ISP)
public interface Reader {
    String read(String filePath);
}

public interface Writer {
    void write(String filePath, String content);
}

@Component
public class FileReader implements Reader {
    @Override
    public String read(String filePath) {
        // File reading logic
        return "File content";
    }
}

@Component
public class FileWriter implements Writer {
    @Override
    public void write(String filePath, String content) {
        // File writing logic
```

```
    }
}

// Dependency Inversion Principle (DIP)
public interface MessageService {
    void sendMessage(String message, String recipient);
}

@Component
public class EmailService implements MessageService {
    @Override
    public void sendMessage(String message, String recipient) {
        // Email sending logic
    }
}

@Service
public class NotificationService {
    private final MessageService messageService;

    public NotificationService(MessageService messageService) {
        this.messageService = messageService;
    }

    public void sendNotification(String message, String recipient) {
        messageService.sendMessage(message, recipient);
    }
}
```

Design Patterns Implementation

```
// Factory Pattern
public abstract class Vehicle {
    public abstract void drive();
}

@Component
public class Car extends Vehicle {
    @Override
    public void drive() {
        System.out.println("Driving a car");
    }
}

@Component
public class Motorcycle extends Vehicle {
    @Override
    public void drive() {
        System.out.println("Riding a motorcycle");
    }
}

@Component
```

```
public class VehicleFactory {
    private final ApplicationContext context;

    public VehicleFactory(ApplicationContext context) {
        this.context = context;
    }

    public Vehicle createVehicle(String type) {
        switch (type.toUpperCase()) {
            case "CAR":
                return context.getBean(Car.class);
            case "MOTORCYCLE":
                return context.getBean(Motorcycle.class);
            default:
                throw new IllegalArgumentException("Unknown vehicle typ
        }
    }
}

// Abstract Factory Pattern
public interface GUIFactory {
    Button createButton();
    Checkbox createCheckbox();
}

@Component
public class WindowsFactory implements GUIFactory {
    @Override
    public Button createButton() {
        return new WindowsButton();
    }

    @Override
    public Checkbox createCheckbox() {
        return new WindowsCheckbox();
    }
}

@Component
public class MacFactory implements GUIFactory {
    @Override
    public Button createButton() {
        return new MacButton();
    }

    @Override
    public Checkbox createCheckbox() {
        return new MacCheckbox();
    }
}

// Singleton Pattern
@Component
```

```
@Scope("singleton")
public class LoggerService {
    private static final Logger log = LoggerFactory.getLogger(LoggerSer

    public void log(String message) {
        log.info(message);
    }
}

// Observer Pattern
public interface EventListener {
    void onEvent(String event);
}

@Component
public class EmailNotificationListener implements EventListener {
    @Override
    public void onEvent(String event) {
        System.out.println("Email notification: " + event);
    }
}

@Component
public class SMSServiceListener implements EventListener {
    @Override
    public void onEvent(String event) {
        System.out.println("SMS notification: " + event);
    }
}

@Service
public class EventService {
    private final List<EventListener> listeners;

    public EventService(List<EventListener> listeners) {
        this.listeners = listeners;
    }

    public void fireEvent(String event) {
        listeners.forEach(listener -> listener.onEvent(event));
    }
}

// Strategy Pattern
public interface SortingStrategy {
    int[] sort(int[] array);
}

@Component
public class BubbleSortStrategy implements SortingStrategy {
    @Override
    public int[] sort(int[] array) {
        // Bubble sort implementation
```

```
        return array;
    }
}

@Component
public class QuickSortStrategy implements SortingStrategy {
    @Override
    public int[] sort(int[] array) {
        // Quick sort implementation
        return array;
    }
}

@Service
public class SortingService {
    private final Map<String, SortingStrategy> strategies;

    public SortingService(List<SortingStrategy> strategies) {
        this.strategies = strategies.stream()
            .collect(Collectors.toMap(
                strategy -> strategy.getClass().getSimpleName().replace
                Function.identity()
            ));
    }

    public int[] sort(int[] array, String strategyName) {
        SortingStrategy strategy = strategies.get(strategyName);
        if (strategy == null) {
            throw new IllegalArgumentException("Unknown strategy: " + s
        }
        return strategy.sort(array);
    }
}
```

---

28. Domain-Driven Design (DDD) Annotations

DDD Core Concepts

```
// Entity
@Entity
@Table(name = "orders")
public class Order {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Embedded
    private OrderId orderId;

    @Embedded
```

```
private CustomerId customerId;

@OneToMany(mappedBy = "order", cascade = CascadeType.ALL, orphanRem
private List<OrderLine> orderLines = new ArrayList<>();

@Enumerated(EnumType.STRING)
private OrderStatus status;

@Embedded
private Money totalAmount;

@CreatedDate
private LocalDateTime createdAt;

@LastModifiedDate
private LocalDateTime updatedAt;

// Domain behavior
public void addOrderLine(Product product, int quantity, Money unitP
    OrderLine line = new OrderLine(product, quantity, unitPrice);
    orderLines.add(line);
    recalculateTotal();
}

public void removeOrderLine(OrderLine line) {
    orderLines.remove(line);
    recalculateTotal();
}

public void confirm() {
    if (status != OrderStatus.PENDING) {
        throw new OrderAlreadyConfirmedException("Order is already
    }
    status = OrderStatus.CONFIRMED;
}

public void cancel() {
    if (status == OrderStatus.SHIPPED) {
        throw new OrderCannotBeCancelledException("Shipped order ca
    }
    status = OrderStatus.CANCELLED;
}

private void recalculateTotal() {
    totalAmount = orderLines.stream()
        .map(OrderLine::getTotalPrice)
        .reduce(Money.ZERO, Money::add);
}

// Getters
public OrderId getOrderId() { return orderId; }
public OrderStatus getStatus() { return status; }
public Money getTotalAmount() { return totalAmount; }
```

```
    public List<OrderLine> getOrderLines() { return Collections.unmodif
}

// Value Object
@Embeddable
public class OrderId implements Serializable {

    @Column(name = "order_id")
    private String value;

    protected OrderId() {}

    public OrderId(String value) {
        if (value == null || value.trim().isEmpty()) {
            throw new IllegalArgumentException("Order ID cannot be null
        }
        this.value = value;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        OrderId orderId = (OrderId) o;
        return Objects.equals(value, orderId.value);
    }

    @Override
    public int hashCode() {
        return Objects.hash(value);
    }

    @Override
    public String toString() {
        return value;
    }
}

@Embeddable
public class Money implements Serializable {

    public static final Money ZERO = new Money(0, "USD");

    @Column(name = "amount")
    private BigDecimal amount;

    @Column(name = "currency")
    private String currency;

    protected Money() {}

    public Money(BigDecimal amount, String currency) {
        if (amount == null) {
```

```
            throw new IllegalArgumentException("Amount cannot be null")
        }
        if (currency == null || currency.trim().isEmpty()) {
            throw new IllegalArgumentException("Currency cannot be null
        }
        this.amount = amount.setScale(2, RoundingMode.HALF_UP);
        this.currency = currency;
    }

    public Money add(Money other) {
        if (!this.currency.equals(other.currency)) {
            throw new CurrencyMismatchException("Cannot add money with
        }
        return new Money(this.amount.add(other.amount), this.currency);
    }

    public Money subtract(Money other) {
        if (!this.currency.equals(other.currency)) {
            throw new CurrencyMismatchException("Cannot subtract money
        }
        return new Money(this.amount.subtract(other.amount), this.curre
    }

    public Money multiply(BigDecimal multiplier) {
        return new Money(this.amount.multiply(multiplier), this.currenc
    }

    // Getters
    public BigDecimal getAmount() { return amount; }
    public String getCurrency() { return currency; }
}

// Aggregate Root
@Repository
public interface OrderRepository extends JpaRepository<Order, Long> {
    Optional<Order> findByOrderId(OrderId orderId);
    List<Order> findByCustomerId(CustomerId customerId);
    List<Order> findByStatus(OrderStatus status);
}

@Service
@Transactional
public class OrderService {

    private final OrderRepository orderRepository;
    private final ProductRepository productRepository;
    private final DomainEventPublisher eventPublisher;

    public OrderService(OrderRepository orderRepository,
                        ProductRepository productRepository,
                        DomainEventPublisher eventPublisher) {
        this.orderRepository = orderRepository;
        this.productRepository = productRepository;
```

```
        this.eventPublisher = eventPublisher;
    }

    public Order createOrder(CreateOrderCommand command) {
        Order order = new Order();
        order.setOrderId(new OrderId(UUID.randomUUID().toString()));
        order.setCustomerId(command.getCustomerId());
        order.setStatus(OrderStatus.PENDING);

        for (CreateOrderLineCommand lineCommand : command.getOrderLines
            Product product = productRepository.findById(lineCommand.ge
                .orElseThrow(() -> new ProductNotFoundException("Produc

            order.addOrderLine(product, lineCommand.getQuantity(),
                new Money(lineCommand.getUnitPrice(), "USD"));
        }

        order = orderRepository.save(order);

        // Publish domain event
        eventPublisher.publish(new OrderCreatedEvent(order.getOrderId()

        return order;
    }

    public void confirmOrder(OrderId orderId) {
        Order order = orderRepository.findByOrderId(orderId)
            .orElseThrow(() -> new OrderNotFoundException("Order not fo

        order.confirm();
        orderRepository.save(order);

        eventPublisher.publish(new OrderConfirmedEvent(orderId));
    }
}
```

Domain Events

```
public abstract class DomainEvent {
    private final Instant occurredOn;
    private final String eventId;

    protected DomainEvent() {
        this.occurredOn = Instant.now();
        this.eventId = UUID.randomUUID().toString();
    }

    public Instant getOccurredOn() { return occurredOn; }
    public String getEventId() { return eventId; }
}

public class OrderCreatedEvent extends DomainEvent {
    private final OrderId orderId;
```

```
    private final CustomerId customerId;

    public OrderCreatedEvent(OrderId orderId, CustomerId customerId) {
        super();
        this.orderId = orderId;
        this.customerId = customerId;
    }

    public OrderId getOrderId() { return orderId; }
    public CustomerId getCustomerId() { return customerId; }
}

public class OrderConfirmedEvent extends DomainEvent {
    private final OrderId orderId;

    public OrderConfirmedEvent(OrderId orderId) {
        super();
        this.orderId = orderId;
    }

    public OrderId getOrderId() { return orderId; }
}

@Component
public class DomainEventPublisher {

    private final ApplicationEventPublisher applicationEventPublisher;

    public DomainEventPublisher(ApplicationEventPublisher applicationEv
        this.applicationEventPublisher = applicationEventPublisher;
    }

    public void publish(DomainEvent event) {
        applicationEventPublisher.publishEvent(event);
    }
}

@EventListener
@Component
public class OrderEventHandler {

    private final NotificationService notificationService;
    private final InventoryService inventoryService;

    public OrderEventHandler(NotificationService notificationService,
                             InventoryService inventoryService) {
        this.notificationService = notificationService;
        this.inventoryService = inventoryService;
    }

    @EventListener
    public void handle(OrderCreatedEvent event) {
        notificationService.sendOrderConfirmation(event.getCustomerId()
```

```
    }

    @EventListener
    public void handle(OrderConfirmedEvent event) {
        inventoryService.reserveItems(event.getOrderId());
    }
}
```

Application Services

```
@Service
@Transactional
public class OrderApplicationService {

    private final OrderService orderService;
    private final OrderRepository orderRepository;
    private final OrderMapper orderMapper;

    public OrderApplicationService(OrderService orderService,
                                   OrderRepository orderRepository,
                                   OrderMapper orderMapper) {
        this.orderService = orderService;
        this.orderRepository = orderRepository;
        this.orderMapper = orderMapper;
    }

    public OrderDto createOrder(CreateOrderRequest request) {
        CreateOrderCommand command = orderMapper.toCommand(request);
        Order order = orderService.createOrder(command);
        return orderMapper.toDto(order);
    }

    public OrderDto getOrder(String orderId) {
        Order order = orderRepository.findByOrderId(new OrderId(orderId
            .orElseThrow(() -> new OrderNotFoundException("Order not fo
        return orderMapper.toDto(order);
    }

    public void confirmOrder(String orderId) {
        orderService.confirmOrder(new OrderId(orderId));
    }
}
```

---

29. Object-Oriented Modeling & Design (OOMD)

Class Design Principles

```
// Encapsulation
public class BankAccount {
    private String accountNumber;
    private BigDecimal balance;
```

```
    private String accountHolder;

    public BankAccount(String accountNumber, String accountHolder, BigD
        this.accountNumber = accountNumber;
        this.accountHolder = accountHolder;
        this.balance = initialBalance;
    }

    public void deposit(BigDecimal amount) {
        if (amount.compareTo(BigDecimal.ZERO) <= 0) {
            throw new IllegalArgumentException("Deposit amount must be
        }
        balance = balance.add(amount);
    }

    public void withdraw(BigDecimal amount) {
        if (amount.compareTo(BigDecimal.ZERO) <= 0) {
            throw new IllegalArgumentException("Withdrawal amount must
        }
        if (balance.compareTo(amount) < 0) {
            throw new InsufficientFundsException("Insufficient funds");
        }
        balance = balance.subtract(amount);
    }

    public BigDecimal getBalance() {
        return balance;
    }

    // No direct setter for balance - controlled through deposit/withdr
}

// Abstraction
public abstract class Shape {
    protected String color;

    public Shape(String color) {
        this.color = color;
    }

    public abstract double calculateArea();
    public abstract double calculatePerimeter();

    public void display() {
        System.out.println("Shape with color: " + color);
    }
}

public class Circle extends Shape {
    private double radius;

    public Circle(String color, double radius) {
        super(color);
```

```
        this.radius = radius;
    }

    @Override
    public double calculateArea() {
        return Math.PI * radius * radius;
    }

    @Override
    public double calculatePerimeter() {
        return 2 * Math.PI * radius;
    }
}

// Polymorphism
public interface Drawable {
    void draw();
}

public class Rectangle extends Shape implements Drawable {
    private double width;
    private double height;

    public Rectangle(String color, double width, double height) {
        super(color);
        this.width = width;
        this.height = height;
    }

    @Override
    public double calculateArea() {
        return width * height;
    }

    @Override
    public double calculatePerimeter() {
        return 2 * (width + height);
    }

    @Override
    public void draw() {
        System.out.println("Drawing rectangle with color: " + color);
    }
}

// Composition over Inheritance
public class Engine {
    private String type;
    private int horsepower;

    public Engine(String type, int horsepower) {
        this.type = type;
        this.horsepower = horsepower;
```

```
    }

    public void start() {
        System.out.println(type + " engine starting");
    }
}

public class Car {
    private Engine engine;
    private String brand;
    private String model;

    public Car(String brand, String model, Engine engine) {
        this.brand = brand;
        this.model = model;
        this.engine = engine;
    }

    public void startCar() {
        System.out.println("Starting " + brand + " " + model);
        engine.start();
    }
}
```

Design Patterns in OOMD

```
// Template Method Pattern
public abstract class DataProcessor {

    public final void processData() {
        loadData();
        validateData();
        transformData();
        saveData();
        cleanup();
    }

    protected abstract void loadData();
    protected abstract void validateData();
    protected abstract void transformData();
    protected abstract void saveData();

    protected void cleanup() {
        System.out.println("Default cleanup");
    }
}

@Component
public class CSVDataProcessor extends DataProcessor {

    @Override
    protected void loadData() {
        System.out.println("Loading CSV data");
```

```
    }

    @Override
    protected void validateData() {
        System.out.println("Validating CSV data");
    }

    @Override
    protected void transformData() {
        System.out.println("Transforming CSV data");
    }

    @Override
    protected void saveData() {
        System.out.println("Saving CSV data");
    }
}

// Decorator Pattern
public interface DataSource {
    void writeData(String data);
    String readData();
}

@Component
public class FileDataSource implements DataSource {
    private String filename;

    public FileDataSource(String filename) {
        this.filename = filename;
    }

    @Override
    public void writeData(String data) {
        System.out.println("Writing to file: " + filename);
    }

    @Override
    public String readData() {
        System.out.println("Reading from file: " + filename);
        return "File data";
    }
}

public class DataSourceDecorator implements DataSource {
    protected DataSource wrappee;

    public DataSourceDecorator(DataSource source) {
        this.wrappee = source;
    }

    @Override
    public void writeData(String data) {
```

```
        wrappee.writeData(data);
    }

    @Override
    public String readData() {
        return wrappee.readData();
    }
}

@Component
public class EncryptionDecorator extends DataSourceDecorator {

    public EncryptionDecorator(DataSource source) {
        super(source);
    }

    @Override
    public void writeData(String data) {
        System.out.println("Encrypting data");
        super.writeData(data);
    }

    @Override
    public String readData() {
        System.out.println("Decrypting data");
        return super.readData();
    }
}

// Command Pattern
public interface Command {
    void execute();
    void undo();
}

@Component
public class Light {
    private boolean isOn = false;

    public void turnOn() {
        isOn = true;
        System.out.println("Light is ON");
    }

    public void turnOff() {
        isOn = false;
        System.out.println("Light is OFF");
    }

    public boolean isOn() {
        return isOn;
    }
}
```

```
public class LightOnCommand implements Command {
    private Light light;

    public LightOnCommand(Light light) {
        this.light = light;
    }

    @Override
    public void execute() {
        light.turnOn();
    }

    @Override
    public void undo() {
        light.turnOff();
    }
}

public class LightOffCommand implements Command {
    private Light light;

    public LightOffCommand(Light light) {
        this.light = light;
    }

    @Override
    public void execute() {
        light.turnOff();
    }

    @Override
    public void undo() {
        light.turnOn();
    }
}

@Component
public class RemoteControl {
    private Command command;

    public void setCommand(Command command) {
        this.command = command;
    }

    public void pressButton() {
        command.execute();
    }

    public void pressUndo() {
        command.undo();
    }
}
```

UML Modeling with Spring

```
// Association
@Entity
public class Student {
    @Id
    private Long id;

    private String name;

    @ManyToOne
    @JoinColumn(name = "course_id")
    private Course course;

    // Getters and setters
}

@Entity
public class Course {
    @Id
    private Long id;

    private String name;

    @OneToMany(mappedBy = "course")
    private List<Student> students;

    // Getters and setters
}

// Aggregation
@Entity
public class Department {
    @Id
    private Long id;

    private String name;

    @OneToMany(mappedBy = "department", cascade = CascadeType.ALL)
    private List<Professor> professors;

    // Getters and setters
}

@Entity
public class Professor {
    @Id
    private Long id;

    private String name;

    @ManyToOne
    @JoinColumn(name = "department_id")
```

```
    private Department department;

    // Getters and setters
}

// Composition
@Entity
public class Car {
    @Id
    private Long id;

    @OneToOne(mappedBy = "car", cascade = CascadeType.ALL, orphanRemova
    private Engine engine;

    @OneToMany(mappedBy = "car", cascade = CascadeType.ALL, orphanRemov
    private List<Wheel> wheels = new ArrayList<>();

    public void addWheel(Wheel wheel) {
        wheels.add(wheel);
        wheel.setCar(this);
    }

    // Getters and setters
}

@Entity
public class Engine {
    @Id
    private Long id;

    private String type;

    @OneToOne
    @JoinColumn(name = "car_id")
    private Car car;

    // Getters and setters
}

@Entity
public class Wheel {
    @Id
    private Long id;

    private int position;

    @ManyToOne
    @JoinColumn(name = "car_id")
    private Car car;

    // Getters and setters
}
```

30. Hibernate Annotations

Hibernate Core Annotations

```
@Entity                  // JPA entity
@Table                  // Table specification
@Column                 // Column specification
@Id                     // Primary key
@GeneratedValue         // Auto-generated primary key
@Embedded               // Embedded object
@Embeddable             // Embeddable class
@Transient              // Non-persistent field
@Version                // Optimistic locking
@Lob                    // Large object
@Enumerated             // Enum mapping
@Temporal               // Date/time mapping
@Basic                  // Basic property mapping
@Access                 // Access type
```

Hibernate-Specific Annotations

```
@Cacheable               // Enable second-level cache
@Cache                  // Cache configuration
@NaturalId              // Natural identifier
@Filter                 // Hibernate filter
@FilterDef              // Filter definition
@Where                  // Where clause
@Index                  // Index definition
@IndexColumn            // Index column
@Formula                // SQL formula
@Type                   // Custom type
@TypeDef                // Type definition
@Any                    // Polymorphic association
@AnyMetaDef             // Any meta definition
@CollectionId           // Collection ID
@CollectionTable        // Collection table
@ElementCollection      // Element collection
@MapKey                 // Map key
@MapKeyColumn           // Map key column
@MapKeyTemporal         // Map key temporal
@MapKeyEnumerated       // Map key enum
@JoinTable              // Join table
@JoinColumn             // Join column
@JoinColumns            // Multiple join columns
@PrimaryKeyJoinColumn   // Primary key join column
@Inheritance            // Inheritance strategy
@DiscriminatorColumn    // Discriminator column
@DiscriminatorValue     // Discriminator value
@MappedSuperclass       // Mapped superclass
@AttributeOverride      // Attribute override
@AttributeOverrides     // Multiple attribute overrides
```

```
@AssociationOverride    // Association override
@AssociationOverrides   // Multiple association overrides
@SQLInsert              // Custom SQL insert
@SQLUpdate              // Custom SQL update
@SQLDelete              // Custom SQL delete
@SQLDeleteAll           // Custom SQL delete all
@Loader                 // Custom loader
@NamedQueries           // Named queries
@NamedQuery             // Named query
@NamedStoredProcedureQuery // Named stored procedure
@StoredProcedureParameter // Stored procedure parameter
@DynamicInsert          // Dynamic insert
@DynamicUpdate          // Dynamic update
@SelectBeforeUpdate     // Select before update
@OptimisticLock         // Optimistic locking
@PessimisticLock        // Pessimistic locking
@BatchSize              // Batch size
@Fetch                  // Fetch strategy
@LazyGroup              // Lazy group
@Proxy                  // Proxy configuration
@Cascade                // Cascade operations
@OnDelete               // On delete action
@Check                  // Check constraint
@ColumnDefault          // Column default value
@ColumnTransform        // Column transformation
@GenericGenerator       // Generic generator
@TableGenerator         // Table generator
@SequenceGenerator      // Sequence generator
```

Hibernate Entity Examples

```
@Entity
@Table(name = "users", indexes = {
    @Index(name = "idx_user_username", columnList = "username"),
    @Index(name = "idx_user_email", columnList = "email"),
    @Index(name = "idx_user_created_at", columnList = "created_at")
})
@Cacheable
@org.hibernate.annotations.Cache(usage = CacheConcurrencyStrategy.READ_
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NaturalId
    @Column(name = "username", unique = true, nullable = false, length
    private String username;

    @Column(name = "email", nullable = false, length = 100)
    private String email;

    @Column(name = "first_name", length = 50)
```

```
    private String firstName;

    @Column(name = "last_name", length = 50)
    private String lastName;

    @Column(name = "password", nullable = false)
    private String password;

    @Enumerated(EnumType.STRING)
    @Column(name = "status", nullable = false)
    private UserStatus status;

    @Temporal(TemporalType.TIMESTAMP)
    @Column(name = "created_at", nullable = false, updatable = false)
    private Date createdAt;

    @Temporal(TemporalType.TIMESTAMP)
    @Column(name = "updated_at")
    private Date updatedAt;

    @Version
    @Column(name = "version")
    private Long version;

    @Embedded
    private UserProfile profile;

    @OneToMany(mappedBy = "user", cascade = CascadeType.ALL, orphanRemo
    @Cache(usage = CacheConcurrencyStrategy.READ_WRITE)
    private List<Order> orders = new ArrayList<>();

    @ManyToMany(fetch = FetchType.LAZY)
    @JoinTable(
        name = "user_roles",
        joinColumns = @JoinColumn(name = "user_id"),
        inverseJoinColumns = @JoinColumn(name = "role_id"),
        indexes = @Index(columnList = "user_id, role_id")
    )
    @Cache(usage = CacheConcurrencyStrategy.READ_WRITE)
    private Set<Role> roles = new HashSet<>();

    @ElementCollection(fetch = FetchType.LAZY)
    @CollectionTable(
        name = "user_addresses",
        joinColumns = @JoinColumn(name = "user_id"),
        indexes = @Index(columnList = "user_id")
    )
    @Cache(usage = CacheConcurrencyStrategy.READ_WRITE)
    private List<Address> addresses = new ArrayList<>();

    // Getters and setters
}
```

```
@Embeddable
public class UserProfile {

    @Column(name = "phone", length = 20)
    private String phone;

    @Column(name = "date_of_birth")
    @Temporal(TemporalType.DATE)
    private Date dateOfBirth;

    @Column(name = "avatar_url", length = 255)
    private String avatarUrl;

    @Column(name = "bio", columnDefinition = "TEXT")
    private String bio;

    // Getters and setters
}

@Entity
@Table(name = "orders")
@org.hibernate.annotations.Cache(usage = CacheConcurrencyStrategy.READ_
public class Order {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "order_number", unique = true, nullable = false, len
    private String orderNumber;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "user_id", nullable = false)
    @org.hibernate.annotations.Cache(usage = CacheConcurrencyStrategy.R
    private User user;

    @OneToMany(mappedBy = "order", cascade = CascadeType.ALL, orphanRem
    @BatchSize(size = 20)
    private List<OrderItem> items = new ArrayList<>();

    @Enumerated(EnumType.STRING)
    @Column(name = "status", nullable = false)
    private OrderStatus status;

    @Column(name = "total_amount", nullable = false, precision = 19, sc
    private BigDecimal totalAmount;

    @Temporal(TemporalType.TIMESTAMP)
    @Column(name = "order_date", nullable = false)
    private Date orderDate;

    @Column(name = "shipping_address", length = 500)
    private String shippingAddress;
```

```
    @Column(name = "notes", columnDefinition = "TEXT")
    private String notes;

    @Version
    @Column(name = "version")
    private Long version;

    // Getters and setters
}

@Entity
@Table(name = "order_items")
public class OrderItem {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "order_id", nullable = false)
    private Order order;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "product_id", nullable = false)
    private Product product;

    @Column(name = "quantity", nullable = false)
    private Integer quantity;

    @Column(name = "unit_price", nullable = false, precision = 19, scal
    private BigDecimal unitPrice;

    @Column(name = "total_price", nullable = false, precision = 19, sca
    private BigDecimal totalPrice;

    // Getters and setters
}
```

Hibernate Configuration

```
@Configuration
@EnableTransactionManagement
@EnableJpaRepositories(basePackages = "com.example.repository")
public class HibernateConfig {

    @Bean
    public LocalContainerEntityManagerFactoryBean entityManagerFactory(
            DataSource dataSource,
            JpaProperties jpaProperties) {

        LocalContainerEntityManagerFactoryBean emf = new LocalContainer
        emf.setDataSource(dataSource);
```

```
        emf.setPackagesToScan("com.example.entity");

        HibernateJpaVendorAdapter vendorAdapter = new HibernateJpaVendo
        emf.setJpaVendorAdapter(vendorAdapter);

        Map<String, Object> properties = new HashMap<>();
        properties.putAll(jpaProperties.getHibernateProperties(dataSour

        // Hibernate-specific properties
        properties.put("hibernate.show_sql", true);
        properties.put("hibernate.format_sql", true);
        properties.put("hibernate.use_sql_comments", true);
        properties.put("hibernate.jdbc.batch_size", 20);
        properties.put("hibernate.order_inserts", true);
        properties.put("hibernate.order_updates", true);
        properties.put("hibernate.jdbc.fetch_size", 100);
        properties.put("hibernate.cache.use_second_level_cache", true);
        properties.put("hibernate.cache.region.factory_class", "org.hib
        properties.put("hibernate.javax.cache.provider", "org.ehcache.j
        properties.put("hibernate.generate_statistics", true);
        properties.put("hibernate.connection.provider_disables_autocomm

        emf.setJpaPropertyMap(properties);

        return emf;
    }

    @Bean
    public PlatformTransactionManager transactionManager(EntityManagerF
        JpaTransactionManager transactionManager = new JpaTransactionMa
        transactionManager.setEntityManagerFactory(entityManagerFactory
        return transactionManager;
    }

    @Bean
    public JpaTransactionManager jpaTransactionManager(EntityManagerFac
        JpaTransactionManager transactionManager = new JpaTransactionMa
        transactionManager.setEntityManagerFactory(entityManagerFactory
        return transactionManager;
    }

    @Bean
    public HibernateTransactionManager hibernateTransactionManager(Sess
        HibernateTransactionManager transactionManager = new HibernateT
        transactionManager.setSessionFactory(sessionFactory);
        return transactionManager;
    }
}

// Application properties
@ConfigurationProperties(prefix = "spring.jpa")
public class JpaProperties {
```

```
private Map<String, String> properties = new HashMap<>();
private Hibernate hibernate = new Hibernate();

public Map<String, String> getProperties() {
    return properties;
}

public void setProperties(Map<String, String> properties) {
    this.properties = properties;
}

public Hibernate getHibernate() {
    return hibernate;
}

public void setHibernate(Hibernate hibernate) {
    this.hibernate = hibernate;
}

public Map<String, Object> getHibernateProperties(DataSource dataSo
    Map<String, Object> hibernateProperties = new HashMap<>();

    if (hibernate.getDdlAuto() != null) {
        hibernateProperties.put("hibernate.hbm2ddl.auto", hibernate
    }

    if (hibernate.getDialect() != null) {
        hibernateProperties.put("hibernate.dialect", hibernate.getD
    }

    if (hibernate.getShowSql() != null) {
        hibernateProperties.put("hibernate.show_sql", hibernate.get
    }

    if (hibernate.getFormatSql() != null) {
        hibernateProperties.put("hibernate.format_sql", hibernate.g
    }

    if (hibernate.getUseSqlComments() != null) {
        hibernateProperties.put("hibernate.use_sql_comments", hiber
    }

    return hibernateProperties;
}

public static class Hibernate {
    private String ddlAuto;
    private String dialect;
    private Boolean showSql;
    private Boolean formatSql;
    private Boolean useSqlComments;

    // Getters and setters
```

```
    }
}
```

Hibernate Query Examples

```
@Repository
public interface UserRepository extends JpaRepository<User, Long> {

    @Query("SELECT u FROM User u WHERE u.username = :username")
    Optional<User> findByUsername(@Param("username") String username);

    @Query("SELECT u FROM User u WHERE u.email = :email")
    Optional<User> findByEmail(@Param("email") String email);

    @Query("SELECT u FROM User u WHERE u.status = :status")
    List<User> findByStatus(@Param("status") UserStatus status);

    @Query("SELECT u FROM User u WHERE u.createdAt BETWEEN :startDate A
    List<User> findByCreatedAtBetween(@Param("startDate") Date startDat

    @Query("SELECT u FROM User u WHERE u.firstName LIKE %:name% OR u.la
    List<User> findByNameContaining(@Param("name") String name);

    @Query(value = "SELECT * FROM users WHERE email = :email", nativeQu
    Optional<User> findByEmailNative(@Param("email") String email);

    @Modifying
    @Query("UPDATE User u SET u.status = :status WHERE u.id = :id")
    int updateStatus(@Param("id") Long id, @Param("status") UserStatus

    @Modifying
    @Query("DELETE FROM User u WHERE u.status = :status")
    int deleteByStatus(@Param("status") UserStatus status);

    @Query("SELECT u FROM User u JOIN FETCH u.orders o WHERE o.status =
    List<User> findUsersWithOrdersByStatus(@Param("orderStatus") OrderS

    @Query("SELECT u FROM User u WHERE u.id IN :ids")
    List<User> findByIds(@Param("ids") List<Long> ids);

    @Query("SELECT COUNT(u) FROM User u WHERE u.status = :status")
    Long countByStatus(@Param("status") UserStatus status);

    @Query("SELECT u FROM User u ORDER BY u.createdAt DESC")
    Page<User> findAllOrderByCreatedAtDesc(Pageable pageable);
}

// Named queries
@Entity
@NamedQueries({
    @NamedQuery(
        name = "User.findByUsername",
        query = "SELECT u FROM User u WHERE u.username = :username"
```

```
    ),
    @NamedQuery(
        name = "User.findByEmail",
        query = "SELECT u FROM User u WHERE u.email = :email"
    ),
    @NamedQuery(
        name = "User.findByStatus",
        query = "SELECT u FROM User u WHERE u.status = :status"
    ),
    @NamedQuery(
        name = "User.countByStatus",
        query = "SELECT COUNT(u) FROM User u WHERE u.status = :status"
    )
})
@Table(name = "users")
public class User {
    // Entity fields and methods
}
```

---

31. MyBatis Annotations

MyBatis Core Annotations

```
@Mapper                  // MyBatis mapper interface
@Select                 // Select query
@Insert                 // Insert query
@Update                 // Update query
@Delete                 // Delete query
@Param                  # Parameter mapping
@Results                # Result mapping
@Result                 # Result column mapping
@One                    # One-to-one mapping
@Many                   # One-to-many mapping
@Options                # Query options
@SelectProvider          # Dynamic SQL provider
@InsertProvider         # Dynamic insert provider
@UpdateProvider         # Dynamic update provider
@DeleteProvider         # Dynamic delete provider
@ConstructorArgs        # Constructor arguments
@Arg                    # Constructor argument
@CacheNamespace         # Cache namespace
@CacheRef               # Cache reference
@Flush                  # Flush cache
@Options                # Options configuration
```

MyBatis Mapper Examples

```
@Mapper
public interface UserMapper {

    @Select("SELECT * FROM users WHERE id = #{id}")
```

```
User findById(@Param("id") Long id);

@Select("SELECT * FROM users WHERE username = #{username}")
User findByUsername(@Param("username") String username);

@Select("SELECT * FROM users WHERE email = #{email}")
User findByEmail(@Param("email") String email);

@Select("SELECT * FROM users WHERE status = #{status}")
List<User> findByStatus(@Param("status") String status);

@Select("SELECT * FROM users WHERE first_name LIKE CONCAT('%', #{na
List<User> findByNameContaining(@Param("name") String name);

@Select("SELECT * FROM users WHERE created_at BETWEEN #{startDate}
List<User> findByCreatedAtBetween(
    @Param("startDate") Date startDate,
    @Param("endDate") Date endDate
);

@Select("SELECT * FROM users ORDER BY created_at DESC LIMIT #{limit
List<User> findAllWithPagination(@Param("limit") int limit, @Param(

@Select("SELECT COUNT(*) FROM users")
Long countAll();

@Select("SELECT COUNT(*) FROM users WHERE status = #{status}")
Long countByStatus(@Param("status") String status);

@Insert("INSERT INTO users (username, email, first_name, last_name,
        "VALUES ")
@Options(useGeneratedKeys = true, keyProperty = "id")
int insert(User user);

@Update("UPDATE users SET username = #{username}, email = #{email},
        "last_name = #{lastName}, status = #{status}, updated_at =
int update(User user);

@Delete("DELETE FROM users WHERE id = #{id}")
int deleteById(@Param("id") Long id);

@Delete("DELETE FROM users WHERE status = #{status}")
int deleteByStatus(@Param("status") String status);

@Select("SELECT u.*, o.* FROM users u LEFT JOIN orders o ON u.id =
@Results(id = "userWithOrders", value = {
    @Result(property = "id", column = "id"),
    @Result(property = "username", column = "username"),
    @Result(property = "email", column = "email"),
    @Result(property = "firstName", column = "first_name"),
    @Result(property = "lastName", column = "last_name"),
    @Result(property = "status", column = "status"),
    @Result(property = "createdAt", column = "created_at"),
```

```
        @Result(property = "orders", column = "user_id", javaType = Lis
            many = @Many(select = "findOrdersByUserId"))
    })
    UserWithOrders findUserWithOrders(@Param("id") Long id);

    @Select("SELECT * FROM orders WHERE user_id = #{userId}")
    List<Order> findOrdersByUserId(@Param("userId") Long userId);

    @Select("SELECT u.*, p.* FROM users u JOIN user_profiles p ON u.id
    @Results(id = "userWithProfile", value = {
        @Result(property = "id", column = "id"),
        @Result(property = "username", column = "username"),
        @Result(property = "email", column = "email"),
        @Result(property = "profile", column = "user_id", javaType = Us
            one = @One(select = "findProfileByUserId"))
    })
    UserWithProfile findUserWithProfile(@Param("id") Long id);

    @Select("SELECT * FROM user_profiles WHERE user_id = #{userId}")
    UserProfile findProfileByUserId(@Param("userId") Long userId);
}

@Mapper
public interface OrderMapper {

    @Select("SELECT * FROM orders WHERE id = #{id}")
    Order findById(@Param("id") Long id);

    @Select("SELECT * FROM orders WHERE user_id = #{userId}")
    List<Order> findByUserId(@Param("userId") Long userId);

    @Select("SELECT * FROM orders WHERE status = #{status}")
    List<Order> findByStatus(@Param("status") String status);

    @Select("SELECT * FROM orders WHERE order_date BETWEEN #{startDate}
    List<Order> findByOrderDateBetween(
        @Param("startDate") Date startDate,
        @Param("endDate") Date endDate
    );

    @Insert("INSERT INTO orders (order_number, user_id, status, total_a
            "VALUES ")
    @Options(useGeneratedKeys = true, keyProperty = "id")
    int insert(Order order);

    @Update("UPDATE orders SET status = #{status}, total_amount = #{tot
            "shipping_address = #{shippingAddress} WHERE id = #{id}")
    int update(Order order);

    @Delete("DELETE FROM orders WHERE id = #{id}")
    int deleteById(@Param("id") Long id);

    @Select("SELECT o.*, i.* FROM orders o LEFT JOIN order_items i ON o
```

```
    @Results(id = "orderWithItems", value = {
        @Result(property = "id", column = "id"),
        @Result(property = "orderNumber", column = "order_number"),
        @Result(property = "userId", column = "user_id"),
        @Result(property = "status", column = "status"),
        @Result(property = "totalAmount", column = "total_amount"),
        @Result(property = "orderDate", column = "order_date"),
        @Result(property = "items", column = "id", javaType = List.clas
            many = @Many(select = "findItemsByOrderId"))
    })
    OrderWithItems findOrderWithItems(@Param("id") Long id);

    @Select("SELECT * FROM order_items WHERE order_id = #{orderId}")
    List<OrderItem> findItemsByOrderId(@Param("orderId") Long orderId);
}
```

MyBatis Dynamic SQL

```
@Mapper
public interface UserDynamicMapper {

    @SelectProvider(type = UserSqlProvider.class, method = "findUsersBy
    List<User> findUsersByCriteria(@Param("criteria") UserSearchCriteri

    @InsertProvider(type = UserSqlProvider.class, method = "insertUser"
    int insertUser(@Param("user") User user);

    @UpdateProvider(type = UserSqlProvider.class, method = "updateUser"
    int updateUser(@Param("user") User user);

    @DeleteProvider(type = UserSqlProvider.class, method = "deleteUser"
    int deleteUser(@Param("id") Long id);
}

public class UserSqlProvider {

    public String findUsersByCriteria(UserSearchCriteria criteria) {
        return new SQL() {{
            SELECT("*");
            FROM("users");
            WHERE("1=1");

            if (StringUtils.isNotBlank(criteria.getUsername())) {
                AND("username LIKE CONCAT('%', #{criteria.username}, '%
            }

            if (StringUtils.isNotBlank(criteria.getEmail())) {
                AND("email LIKE CONCAT('%', #{criteria.email}, '%')");
            }

            if (StringUtils.isNotBlank(criteria.getStatus())) {
                AND("status = #{criteria.status}");
            }
```

```
        if (criteria.getStartDate() != null) {
            AND("created_at >= #{criteria.startDate}");
        }

        if (criteria.getEndDate() != null) {
            AND("created_at <= #{criteria.endDate}");
        }

        if (StringUtils.isNotBlank(criteria.getSortBy())) {
            ORDER_BY("#{criteria.sortBy}");

            if (criteria.getSortDirection() != null) {
                ORDER_BY("#{criteria.sortDirection}");
            }
        }

        if (criteria.getLimit() != null) {
            LIMIT("#{criteria.limit}");
            if (criteria.getOffset() != null) {
                OFFSET("#{criteria.offset}");
            }
        }
    }}.toString();
}

public String insertUser(User user) {
    return new SQL() {{
        INSERT_INTO("users");
        VALUES("username", "#{username}");
        VALUES("email", "#{email}");
        VALUES("first_name", "#{firstName}");
        VALUES("last_name", "#{lastName}");
        VALUES("password", "#{password}");
        VALUES("status", "#{status}");
        VALUES("created_at", "#{createdAt}");
    }}.toString();
}

public String updateUser(User user) {
    return new SQL() {{
        UPDATE("users");

        if (StringUtils.isNotBlank(user.getUsername())) {
            SET("username = #{username}");
        }

        if (StringUtils.isNotBlank(user.getEmail())) {
            SET("email = #{email}");
        }

        if (StringUtils.isNotBlank(user.getFirstName())) {
            SET("first_name = #{firstName}");
```

```
            }

            if (StringUtils.isNotBlank(user.getLastName())) {
                SET("last_name = #{lastName}");
            }

            if (user.getStatus() != null) {
                SET("status = #{status}");
            }

            SET("updated_at = #{updatedAt}");

            WHERE("id = #{id}");
        }}.toString();
    }

    public String deleteUser(Long id) {
        return new SQL() {{
            DELETE_FROM("users");
            WHERE("id = #{id}");
        }}.toString();
    }
}
```

MyBatis Configuration

```
@Configuration
@MapperScan(basePackages = "com.example.mapper")
public class MyBatisConfig {

    @Bean
    public SqlSessionFactory sqlSessionFactory(DataSource dataSource) t
        SqlSessionFactoryBean sessionFactory = new SqlSessionFactoryBea
        sessionFactory.setDataSource(dataSource);
        sessionFactory.setMapperLocations(new PathMatchingResourcePatte
            .getResources("classpath*:mapper/*.xml"));

        org.apache.ibatis.session.Configuration configuration = new org
        configuration.setMapUnderscoreToCamelCase(true);
        configuration.setCacheEnabled(true);
        configuration.setLazyLoadingEnabled(true);
        configuration.setAggressiveLazyLoading(false);
        configuration.setMultipleResultSetsEnabled(true);
        configuration.setUseColumnLabel(true);
        configuration.setUseGeneratedKeys(true);
        configuration.setDefaultExecutorType(ExecutorType.REUSE);
        configuration.setDefaultStatementTimeout(30);
        configuration.setSafeRowBoundsEnabled(true);
        configuration.setLocalCacheScope(LocalCacheScope.SESSION);
        configuration.setJdbcTypeForNull(JdbcType.NULL);
        configuration.setLazyLoadTriggerMethods(new HashSet<>());
        configuration.setDefaultScriptingLanguageDriver(XMLLanguageDriv
        configuration.setCallSettersOnNulls(true);
```

```
        configuration.setReturnInstanceForEmptyRow(true);
        configuration.setLogPrefix("MYBATIS.");
        configuration.setConfigurationFactory(configurationFactory);

        sessionFactory.setConfiguration(configuration);

        return sessionFactory.getObject();
    }

    @Bean
    public SqlSessionTemplate sqlSessionTemplate(SqlSessionFactory sqlS
        return new SqlSessionTemplate(sqlSessionFactory);
    }

    @Bean
    public DataSourceTransactionManager transactionManager(DataSource d
        return new DataSourceTransactionManager(dataSource);
    }

    @Bean
    public ConfigurationFactory configurationFactory() {
        return new ConfigurationFactory() {
            @Override
            public org.apache.ibatis.session.Configuration createConfig
                org.apache.ibatis.session.Configuration configuration =
                configuration.setMapUnderscoreToCamelCase(true);
                configuration.setCacheEnabled(true);
                configuration.setLazyLoadingEnabled(true);
                return configuration;
            }
        };
    }
}

// MyBatis properties
@ConfigurationProperties(prefix = "mybatis")
public class MyBatisProperties {

    private String configLocation;
    private String mapperLocations;
    private String typeAliasesPackage;
    private String typeHandlersPackage;
    private boolean checkConfigLocation = false;
    private ExecutorType executorType;
    private Properties configurationProperties;

    // Getters and setters
}
```

MyBatis Spring Integration

```
@Service
@Transactional
```

```
public class UserService {

    private final UserMapper userMapper;
    private final OrderMapper orderMapper;

    public UserService(UserMapper userMapper, OrderMapper orderMapper)
        this.userMapper = userMapper;
        this.orderMapper = orderMapper;
    }

    public User findById(Long id) {
        return userMapper.findById(id);
    }

    public User findByUsername(String username) {
        return userMapper.findByUsername(username);
    }

    public List<User> findByStatus(String status) {
        return userMapper.findByStatus(status);
    }

    public Page<User> findAll(PageRequest pageRequest) {
        int offset = pageRequest.getOffset();
        int limit = pageRequest.getPageSize();

        List<User> users = userMapper.findAllWithPagination(limit, offs
        Long total = userMapper.countAll();

        return new PageImpl<>(users, pageRequest, total);
    }

    public User create(User user) {
        user.setCreatedAt(new Date());
        user.setStatus("ACTIVE");

        userMapper.insert(user);
        return user;
    }

    public User update(User user) {
        user.setUpdatedAt(new Date());
        userMapper.update(user);
        return user;
    }

    public void deleteById(Long id) {
        userMapper.deleteById(id);
    }

    public UserWithOrders findUserWithOrders(Long id) {
        return userMapper.findUserWithOrders(id);
    }
```

```
    public List<User> searchUsers(UserSearchCriteria criteria) {
        return userMapper.findUsersByCriteria(criteria);
    }
}

// Search criteria class
public class UserSearchCriteria {
    private String username;
    private String email;
    private String status;
    private Date startDate;
    private Date endDate;
    private String sortBy;
    private String sortDirection;
    private Integer limit;
    private Integer offset;

    // Getters and setters
}
```

---

32. Java JSR Annotations (Up to Java 26)

JSR 250 (Common Annotations)

```
@Resource                // Resource injection
@PostConstruct           // Post-construction callback
@PreDestroy              // Pre-destruction callback
@Generated              // Generated code marker
@Resources              // Multiple resources
@DeclareRoles           // Role declarations
@RolesAllowed           // Role-based access control
@PermitAll              // Permit all access
@DenyAll                // Deny all access
@RunAs                  // Run as specific role
```

JSR 269 (Annotation Processing)

```
@SupportedAnnotationTypes    // Supported annotation types
@SupportedSourceVersion       // Supported source version
@Retention                    // Retention policy
@Target                       // Element type target
@Inherited                    // Inherited annotation
@Documented                  // Documented in Javadoc
@Repeatable                  // Repeatable annotation
@Native                      // Native method marker
```

JSR 305 (Annotations for Software Defect Detection)

```
@Nonnull                // Non-null parameter/return
@Nullable               // Nullable parameter/return
```

```
@CheckForNull          // Check for null
@CheckReturnValue      // Check return value
@GuardedBy            // Guarded by lock
@Immutable             // Immutable object
@ThreadSafe            // Thread-safe class
@NotThreadSafe         // Not thread-safe
@Beta                  // Beta API
@VisibleForTesting     // Visible for testing
@Override              // Method override
@Deprecated            // Deprecated method
@SuppressWarnings        // Suppress warnings
```

JSR 330 (Dependency Injection for Java)

```
@Inject                // Constructor/field/method injection
@Named                 // Named qualifier
@Qualifier             // Custom qualifier
@Scope                 // Scope definition
@Singleton             // Singleton scope
```

JSR 250 Examples

```
@Component
public class UserService {

    @Resource(name = "userDataSource")
    private DataSource userDataSource;

    @Resources({
        @Resource(name = "primaryDataSource"),
        @Resource(name = "secondaryDataSource")
    })
    private Map<String, DataSource> dataSources;

    @PostConstruct
    public void initialize() {
        System.out.println("UserService initialized");
        validateDataSources();
    }

    @PreDestroy
    public void cleanup() {
        System.out.println("UserService cleanup");
        closeConnections();
    }

    private void validateDataSources() {
        if (dataSources.isEmpty()) {
            throw new IllegalStateException("No data sources configured
        }
    }

    private void closeConnections() {
```

```
        dataSources.values().forEach(dataSource -> {
            if (dataSource instanceof Closeable) {
                try {
                    ((Closeable) dataSource).close();
                } catch (IOException e) {
                    log.error("Error closing data source", e);
                }
            }
        });
    }
}

// Role-based security
@Path("/api/users")
@DeclareRoles({"ADMIN", "USER", "MANAGER"})
public class UserResource {

    @GET
    @Path("/{id}")
    @RolesAllowed({"ADMIN", "USER"})
    public Response getUser(@PathParam("id") Long id) {
        // Implementation
        return Response.ok().build();
    }

    @POST
    @RolesAllowed({"ADMIN"})
    public Response createUser(User user) {
        // Implementation
        return Response.created().build();
    }

    @DELETE
    @Path("/{id}")
    @RolesAllowed({"ADMIN"})
    public Response deleteUser(@PathParam("id") Long id) {
        // Implementation
        return Response.noContent().build();
    }

    @GET
    @Path("/public")
    @PermitAll
    public Response getPublicInfo() {
        // Implementation
        return Response.ok().build();
    }

    @POST
    @Path("/admin-only")
    @DenyAll
    public Response adminOnlyOperation() {
        // This method denies all access
```

```
        return Response.status(Response.Status.FORBIDDEN).build();
    }
}
```

JSR 269 Annotation Processor

```
@SupportedAnnotationTypes({
    "javax.annotation.Resource",
    "javax.annotation.PostConstruct",
    "javax.annotation.PreDestroy"
})
@SupportedSourceVersion(SourceVersion.RELEASE_21)
public class ResourceProcessor extends AbstractProcessor {

    @Override
    public boolean process(Set<? extends TypeElement> annotations, Roun
        for (TypeElement annotation : annotations) {
            Set<? extends Element> elements = roundEnv.getElementsAnnot

            for (Element element : elements) {
                processElement(element, annotation);
            }
        }
        return true;
    }

    private void processElement(Element element, TypeElement annotation
        if (element.getKind() == ElementKind.FIELD) {
            VariableElement field = (VariableElement) element;
            System.out.println("Processing field: " + field.getSimpleNa
                               " with annotation: " + annotation.getSimpl
        }
    }
}

// Custom annotation
@Target(ElementType.METHOD)
@Retention(RetentionPolicy.RUNTIME)
@Inherited
@Documented
@Repeatable(CustomAnnotations.class)
public @interface CustomAnnotation {
    String value() default "";
    int priority() default 0;
}

@Target(ElementType.METHOD)
@Retention(RetentionPolicy.RUNTIME)
@Inherited
public @interface CustomAnnotations {
    CustomAnnotation[] value();
}
```

JSR 305 Annotations

```
public class SafeStringOperations {

    @Nonnull
    public String concatenate(@Nonnull String str1, @Nullable String st
        if (str2 == null) {
            return str1;
        }
        return str1 + str2;
    }

    @CheckForNull
    public String findSubstring(@Nonnull String text, @Nonnull String p
        int index = text.indexOf(pattern);
        return index >= 0 ? text.substring(index) : null;
    }

    @CheckReturnValue
    @Nonnull
    public String trimAndValidate(@CheckForNull String input) {
        if (input == null) {
            throw new IllegalArgumentException("Input cannot be null");
        }
        return input.trim();
    }

    @GuardedBy("lock")
    private List<String> protectedList = new ArrayList<>();

    private final Object lock = new Object();

    public void addToList(@Nonnull String item) {
        synchronized (lock) {
            protectedList.add(item);
        }
    }

    @Immutable
    public static class ImmutableUser {
        private final String username;
        private final String email;

        public ImmutableUser(@Nonnull String username, @Nonnull String
            this.username = username;
            this.email = email;
        }

        @Nonnull
        public String getUsername() {
            return username;
        }
```

```
        @Nonnull
        public String getEmail() {
            return email;
        }
    }
}

@ThreadSafe
public class ThreadSafeCounter {
    private final AtomicInteger counter = new AtomicInteger(0);

    @Nonnull
    public Integer increment() {
        return counter.incrementAndGet();
    }

    @Nonnull
    public Integer getValue() {
        return counter.get();
    }
}

@Beta
@VisibleForTesting
public class ExperimentalFeature {

    @VisibleForTesting
    void internalMethod() {
        // Implementation for testing purposes
    }
}
```

JSR 330 Dependency Injection

```
@Singleton
public class DatabaseService {

    @Inject
    @Named("primaryDataSource")
    private DataSource primaryDataSource;

    @Inject
    @Named("secondaryDataSource")
    private DataSource secondaryDataSource;

    public Connection getPrimaryConnection() throws SQLException {
        return primaryDataSource.getConnection();
    }

    public Connection getSecondaryConnection() throws SQLException {
        return secondaryDataSource.getConnection();
    }
}
```

```
@Qualifier
@Target({ElementType.FIELD, ElementType.PARAMETER, ElementType.METHOD})
@Retention(RetentionPolicy.RUNTIME)
public @interface DatabaseType {
    String value();
}

public class UserService {

    @Inject
    public UserService(@DatabaseType("primary") DatabaseService dbServi
        this.databaseService = dbService;
    }

    private final DatabaseService databaseService;

    public void saveUser(User user) {
        try (Connection conn = databaseService.getPrimaryConnection())
            // Save user logic
        } catch (SQLException e) {
            throw new RuntimeException("Database error", e);
        }
    }
}

// Custom scope
@Scope
@Target({ElementType.TYPE, ElementType.METHOD})
@Retention(RetentionPolicy.RUNTIME)
public @interface RequestScope {
}
```

---

33. Guava Annotations

Guava Core Annotations

```
@Beta                   // Beta API marker
@VisibleForTesting      // Visible for testing
@DoNotMock              // Do not mock this class
@Immutable              // Immutable object
@CheckReturnValue       // Check return value
@CanIgnoreReturnValue   // Can ignore return value
@GuardedBy             // Guarded by lock
@ThreadSafe             // Thread-safe class
@Weak                   // Weak reference
```

Guava Examples

```
@Beta
@VisibleForTesting
```

```
public class ExperimentalCache {

    private final LoadingCache<String, User> userCache;

    public ExperimentalCache() {
        this.userCache = CacheBuilder.newBuilder()
            .maximumSize(1000)
            .expireAfterWrite(10, TimeUnit.MINUTES)
            .build(new CacheLoader<String, User>() {
                @Override
                public User load(String key) {
                    return loadUserFromDatabase(key);
                }
            });
    }

    @CheckReturnValue
    public Optional<User> getUser(@Nonnull String username) {
        try {
            return Optional.ofNullable(userCache.get(username));
        } catch (ExecutionException e) {
            return Optional.empty();
        }
    }

    @CanIgnoreReturnValue
    public void invalidateUser(@Nonnull String username) {
        userCache.invalidate(username);
    }

    private User loadUserFromDatabase(String username) {
        // Database loading logic
        return new User(username, username + "@example.com");
    }
}

@Immutable
public class ImmutableConfiguration {
    private final String databaseUrl;
    private final int maxConnections;
    private final Duration timeout;

    public ImmutableConfiguration(String databaseUrl, int maxConnection
        this.databaseUrl = databaseUrl;
        this.maxConnections = maxConnections;
        this.timeout = timeout;
    }

    @Nonnull
    public String getDatabaseUrl() {
        return databaseUrl;
    }
```

```
    public int getMaxConnections() {
        return maxConnections;
    }

    @Nonnull
    public Duration getTimeout() {
        return timeout;
    }
}

@ThreadSafe
public class ThreadSafeEventBus {

    private final EventBus eventBus = new EventBus();
    private final ExecutorService executor = Executors.newCachedThreadP

    @GuardedBy("this")
    private final Set<Object> subscribers = new HashSet<>();

    public synchronized void register(@Nonnull Object subscriber) {
        subscribers.add(subscriber);
        eventBus.register(subscriber);
    }

    public synchronized void unregister(@Nonnull Object subscriber) {
        subscribers.remove(subscriber);
        eventBus.unregister(subscriber);
    }

    public void post(@Nonnull Object event) {
        executor.submit(() -> eventBus.post(event));
    }

    @DoNotMock
    public static class CriticalEvent {
        private final String type;
        private final Object data;

        public CriticalEvent(String type, Object data) {
            this.type = type;
            this.data = data;
        }

        @Nonnull
        public String getType() {
            return type;
        }

        @Nonnull
        public Object getData() {
            return data;
        }
    }
```

```
}
```

34. Apache Commons Annotations

Commons Lang Annotations

```
@Beta                   // Beta API marker
@Experimental           // Experimental feature
@Immutable              // Immutable object
@Mutable                // Mutable object
@ThreadSafe             // Thread-safe class
@NotThreadSafe          // Not thread-safe
@Since                  // Available since version
@Deprecated             // Deprecated method
```

Commons IO Annotations

```
@Deprecated             // Deprecated method
@Since                  // Available since version
@Beta                   // Beta API marker
```

Commons Lang Examples

```
@Beta
@Experimental
public class AdvancedStringUtils {

    @ThreadSafe
    public static String safeTrim(@Nullable String str) {
        return str == null ? "" : str.trim();
    }

    @Immutable
    public static class ImmutablePair<L, R> {
        private final L left;
        private final R right;

        public ImmutablePair(L left, R right) {
            this.left = left;
            this.right = right;
        }

        @Nullable
        public L getLeft() {
            return left;
        }

        @Nullable
        public R getRight() {
            return right;
        }
```

```
        @Override
        public boolean equals(Object obj) {
            if (obj == this) return true;
            if (obj instanceof ImmutablePair) {
                ImmutablePair<?, ?> other = (ImmutablePair<?, ?>) obj;
                return Objects.equals(left, other.left) && Objects.equa
            }
            return false;
        }

        @Override
        public int hashCode() {
            return Objects.hash(left, right);
        }
    }

    @NotThreadSafe
    public static class MutableCounter {
        private int count = 0;

        public void increment() {
            count++;
        }

        public void decrement() {
            count--;
        }

        public int getCount() {
            return count;
        }

        public void reset() {
            count = 0;
        }
    }
}

@Since("3.0")
public class VersionedFeature {

    @Deprecated
    @Since("1.0")
    public String oldMethod() {
        return "This method is deprecated";
    }

    @Since("3.0")
    public String newMethod() {
        return "This is the new method";
    }
}
```

Commons IO Examples

```
@Beta
public class AdvancedFileUtils {

    @Since("2.0")
    public static void safeDelete(@Nullable File file) {
        if (file != null && file.exists()) {
            try {
                Files.deleteIfExists(file.toPath());
            } catch (IOException e) {
                throw new RuntimeException("Failed to delete file: " +
            }
        }
    }

    @Deprecated
    @Since("1.0")
    public static void legacyCopy(File source, File destination) {
        // Legacy implementation
        try {
            Files.copy(source.toPath(), destination.toPath());
        } catch (IOException e) {
            throw new RuntimeException("Copy failed", e);
        }
    }

    @Since("2.0")
    public static void enhancedCopy(File source, File destination) {
        // Enhanced implementation with better error handling
        Objects.requireNonNull(source, "Source file cannot be null");
        Objects.requireNonNull(destination, "Destination file cannot be

        try {
            Files.createDirectories(destination.getParentFile().toPath(
            Files.copy(source.toPath(), destination.toPath(),
                StandardCopyOption.REPLACE_EXISTING, StandardCopyOption
        } catch (IOException e) {
            throw new RuntimeException("Copy failed", e);
        }
    }
}
```

---

35. Logging Framework Annotations

SLF4J Annotations

```
@Slf4j                  // Lombok SLF4J logger
@XSlf4j                 // Lombok extended SLF4J logger
```

Logback Annotations

```
@NoAutoStart           // No auto-start for components
@ContextName           // Context name for logger
```

Logging Examples

```
// Lombok SLF4J annotations
@Slf4j
@Service
public class UserService {

    public User createUser(UserRequest request) {
        log.info("Creating user with username: {}", request.getUsername

        try {
            User user = new User(request.getUsername(), request.getEmai
            log.debug("User created successfully: {}", user.getId());
            return user;
        } catch (Exception e) {
            log.error("Failed to create user with username: {}", reques
            throw new UserCreationException("Failed to create user", e)
        }
    }

    public void deleteUser(Long userId) {
        log.warn("Deleting user with ID: {}", userId);

        try {
            // Delete logic
            log.info("User deleted successfully: {}", userId);
        } catch (Exception e) {
            log.error("Failed to delete user: {}", userId, e);
            throw new UserDeletionException("Failed to delete user", e)
        }
    }
}

// Extended SLF4J with custom logger
@XSlf4j(topic = "USER_SERVICE")
@RestController
public class UserController {

    @GetMapping("/users/{id}")
    public ResponseEntity<User> getUser(@PathVariable Long id) {
        log.info("Fetching user with ID: {}", id);

        try {
            User user = userService.findById(id);
            if (user == null) {
                log.warn("User not found: {}", id);
                return ResponseEntity.notFound().build();
            }

            log.debug("User found: {}", user.getUsername());
```

```
            return ResponseEntity.ok(user);
        } catch (Exception e) {
            log.error("Error fetching user: {}", id, e);
            return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERR
        }
    }

    @PostMapping("/users")
    public ResponseEntity<User> createUser(@Valid @RequestBody UserRequ
        log.entry(request.getUsername(), request.getEmail());

        try {
            User user = userService.createUser(request);
            log.exit("User created: {}", user.getId());
            return ResponseEntity.status(HttpStatus.CREATED).body(user)
        } catch (Exception e) {
            log.throwing(e);
            return ResponseEntity.status(HttpStatus.BAD_REQUEST).build(
        }
    }
}

// Custom logging configuration
@Configuration
public class LoggingConfig {

    @Bean
    public Logger customLogger() {
        return LoggerFactory.getLogger("CUSTOM_LOGGER");
    }

    @Bean
    public MDCFilter mdcFilter() {
        return new MDCFilter();
    }
}

@Component
public class MDCFilter implements Filter {

    @Override
    public void doFilter(ServletRequest request, ServletResponse respon
            throws IOException, ServletException {

        try {
            MDC.put("requestId", UUID.randomUUID().toString());
            MDC.put("userAgent", ((HttpServletRequest) request).getHead
            MDC.put("remoteAddr", request.getRemoteAddr());

            chain.doFilter(request, response);
        } finally {
            MDC.clear();
        }
```

```
    }
}

// Structured logging with markers
@Component
public class StructuredLogger {

    private static final Marker SECURITY_MARKER = MarkerFactory.getMark
    private static final Marker PERFORMANCE_MARKER = MarkerFactory.getM
    private static final Marker BUSINESS_MARKER = MarkerFactory.getMark

    public void logSecurityEvent(String event, String user, String acti
        log.info(SECURITY_MARKER, "Security event: {} by user: {} - act
    }

    public void logPerformanceMetric(String operation, long durationMs)
        log.info(PERFORMANCE_MARKER, "Performance: {} took {} ms", oper
    }

    public void logBusinessEvent(String event, Object data) {
        log.info(BUSINESS_MARKER, "Business event: {} - data: {}", even
    }
}

// Conditional logging
@Component
public class ConditionalLogger {

    private final Logger logger = LoggerFactory.getLogger(ConditionalLo

    public void logWithCondition(String message, Object... args) {
        if (logger.isTraceEnabled()) {
            logger.trace("TRACE: " + message, args);
        }

        if (logger.isDebugEnabled()) {
            logger.debug("DEBUG: " + message, args);
        }

        if (logger.isInfoEnabled()) {
            logger.info("INFO: " + message, args);
        }

        if (logger.isWarnEnabled()) {
            logger.warn("WARN: " + message, args);
        }

        if (logger.isErrorEnabled()) {
            logger.error("ERROR: " + message, args);
        }
    }

    public void logLargeData(String data) {
```

```
        if (data != null && data.length() > 1000) {
            logger.info("Large data ({} chars): {}", data.length(), dat
        } else {
            logger.info("Data: {}", data);
        }
    }
}
```

---

36. Testing Annotations

Spring Boot Testing

```
@SpringBootTest         // Full Spring Boot test context
@WebMvcTest            // Web layer test (controllers only)
@DataJpaTest           // JPA repository test
@JdbcTest              // JDBC repository test
@DataMongoTest         // MongoDB test
@RestClientTest        // REST client test
```

Spring Framework Testing

```
@ExtendWith            // JUnit 5 extension
@ContextConfiguration  // Load test context
@ActiveProfiles        // Activate specific profiles
@TestPropertySource    # Override properties for test
@MockBean              # Mock Spring bean
@SpyBean               # Spy on Spring bean
@Import                # Import test configuration
```

```
@SpringBootTest
@ActiveProfiles("test")
@TestPropertySource(properties = {
    "spring.datasource.url=jdbc:h2:mem:testdb",
    "spring.jpa.hibernate.ddl-auto=create-drop"
})
class UserServiceIntegrationTest {

    @Autowired
    private UserService userService;

    @MockBean
    private EmailService emailService;

    @Test
    void shouldCreateUserSuccessfully() {
        // Test implementation
    }
}
```

Web Layer Testing

```
@WebMvcTest(UserController.class)
class UserControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @MockBean
    private UserService userService;

    @Test
    void shouldGetUserById() throws Exception {
        when(userService.findById(1L)).thenReturn(createTestUser());

        mockMvc.perform(get("/api/users/1"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.name").value("John Doe"));
    }
}
```

---

9. Caching Annotations

Cache Abstraction

```
@EnableCaching          // Enable caching support
@Cacheable              // Cache method result
@CachePut               # Update cache
@CacheEvict             # Remove from cache
@Caching                # Multiple cache operations
```

```
@Service
@CacheConfig(cacheNames = "users")
public class UserService {

    @Cacheable(key = "#id")
    public User findById(Long id) {
        return userRepository.findById(id).orElse(null);
    }

    @CachePut(key = "#user.id")
    public User save(User user) {
        return userRepository.save(user);
    }

    @CacheEvict(key = "#id")
    public void deleteById(Long id) {
        userRepository.deleteById(id);
    }

    @Caching(
        cacheable = @Cacheable(key = "#email"),
        evict = @CacheEvict(key = "#user.id")
```

```
    )
    public User updateEmail(User user, String email) {
        user.setEmail(email);
        return userRepository.save(user);
    }
}
```

---

10. Scheduling & Asynchronous Annotations

Scheduling

```
@EnableScheduling      // Enable scheduled tasks
@Scheduled            # Schedule method execution
@Async                # Execute method asynchronously
@EnableAsync          # Enable async support
```

```
@Service
@EnableScheduling
public class ScheduledTasks {

    @Scheduled(fixedRate = 5000) // Every 5 seconds
    public void reportCurrentTime() {
        log.info("Current time: {}", LocalDateTime.now());
    }

    @Scheduled(cron = "0 0 12 * * MON-FRI") // Weekdays at noon
    public void weekdayTask() {
        // Weekday task
    }

    @Scheduled(initialDelay = 1000, fixedDelay = 10000)
    public void delayedTask() {
        // Start after 1s, then every 10s after completion
    }
}

@Service
@EnableAsync
public class EmailService {

    @Async
    public CompletableFuture<Void> sendEmail(String to, String subject,
        // Send email asynchronously
        return CompletableFuture.completedFuture(null);
    }
}
```

---

11. Spring Context Annotations

Application Context Management

```
@ContextConfiguration       // Load context for tests
@ActiveProfiles            // Activate specific profiles
@TestPropertySource         # Override properties
@ContextHierarchy          # Define context hierarchy
```

Environment & Profiles

```
@Profile                   # Bean activation based on profile
@PropertySource           # Load properties files
@Environment              # Inject Environment object
@Value                    # Inject property values
```

```
@Configuration
@Profile("production")
@PropertySource("classpath:application-prod.properties")
public class ProductionConfig {

    @Bean
    @Profile("cache")
    public CacheManager cacheManager() {
        return new ConcurrentMapCacheManager("users", "products");
    }

    @Value("${app.datasource.url}")
    private String datasourceUrl;

    @Value("${app.connection.pool.size:10}")
    private int poolSize;

    @Bean
    public DataSource dataSource(Environment env) {
        return DataSourceBuilder.create()
            .url(env.getProperty("app.datasource.url"))
            .username(env.getProperty("app.datasource.username"))
            .password(env.getProperty("app.datasource.password"))
            .build();
    }
}
```

Bean Lifecycle & Scope

```
@Scope                    # Define bean scope
@Lazy                     # Lazy initialization
@DependsOn               # Specify bean dependencies
@Primary                  # Primary bean candidate
@Order                    # Bean ordering
```

```
@Configuration
public class BeanConfig {

    @Bean
    @Scope("prototype") // New instance each time
    public PrototypeBean prototypeBean() {
```

```
        return new PrototypeBean();
    }

    @Bean
    @Lazy // Initialize only when first requested
    public LazyBean lazyBean() {
        return new LazyBean();
    }

    @Bean
    @DependsOn("databaseInitializer") // Initialize after databaseIniti
    public UserService userService() {
        return new UserService();
    }

    @Bean
    @Primary
    public DataSource primaryDataSource() {
        return new HikariDataSource();
    }

    @Bean
    @Order(1)
    public CommandLineRunner firstRunner() {
        return args -> System.out.println("First runner");
    }

    @Bean
    @Order(2)
    public CommandLineRunner secondRunner() {
        return args -> System.out.println("Second runner");
    }
}
```

Bean Scopes

Scope	Description	Use Case
`singleton`	Single instance per container (default)	Stateless services
`prototype`	New instance each request	Stateful objects
`request`	Single instance per HTTP request	Web request data
`session`	Single instance per HTTP session	User session data
`application`	Single instance per ServletContext	Application-wide data
`websocket`	Single instance per WebSocket	WebSocket session data

Event Handling

```
@EventListener           # Handle application events
@AsyncEventListener      # Async event handling
@TransactionalEventListener # Transaction-aware events

@Component
```

```
public class UserEventHandlers {

    @EventListener
    @Async
    public void handleUserCreated(UserCreatedEvent event) {
        // Send welcome email asynchronously
        emailService.sendWelcomeEmail(event.getUser());
    }

    @TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
    public void handleUserRegistered(UserRegisteredEvent event) {
        // Only execute after successful transaction commit
        auditService.logUserRegistration(event.getUser());
    }

    @EventListener(condition = "#event.success")
    public void handleSuccessfulLogin(LoginEvent event) {
        // Handle only successful logins
        securityService.updateLastLogin(event.getUser());
    }
}
```

Application Events

```
@EventListener
public class ApplicationEventHandler {

    @EventListener
    public void handleContextRefreshed(ContextRefreshedEvent event) {
        // Application context fully refreshed
        log.info("Application context refreshed");
    }

    @EventListener
    public void handleContextClosed(ContextClosedEvent event) {
        // Application context closing
        log.info("Application shutting down");
    }

    @EventListener
    public void handleApplicationReady(ApplicationReadyEvent event) {
        // Application ready to serve requests
        log.info("Application ready");
    }
}
```

Conditional Bean Creation

```
@ConditionalOnClass      # Bean exists if class is available
@ConditionalOnMissingClass # Bean exists if class is missing
@ConditionalOnBean       # Bean exists if another bean exists
@ConditionalOnMissingBean # Bean exists if another bean is missing
@ConditionalOnProperty   # Bean exists based on property
```

```
@ConditionalOnResource   # Bean exists if resource is available
@ConditionalOnWebApplication # Bean exists in web context
@ConditionalOnNotWebApplication # Bean exists in non-web context

@Configuration
public class ConditionalConfig {

    @Bean
    @ConditionalOnClass(name = "com.mysql.cj.jdbc.Driver")
    public DataSource mysqlDataSource() {
        return new HikariDataSource();
    }

    @Bean
    @ConditionalOnProperty(name = "cache.type", havingValue = "redis")
    public RedisCacheManager redisCacheManager() {
        return RedisCacheManager.builder()
            .connectionFactory(redisConnectionFactory())
            .build();
    }

    @Bean
    @ConditionalOnBean(DataSource.class)
    public DatabaseHealthIndicator databaseHealthIndicator(DataSource d
        return new DatabaseHealthIndicator(dataSource);
    }

    @Bean
    @ConditionalOnMissingBean(PasswordEncoder.class)
    public NoOpPasswordEncoder passwordEncoder() {
        return NoOpPasswordEncoder.getInstance();
    }
}
```

Configuration Properties Binding

```
@ConfigurationProperties     # Bind external properties
@Validated                   # Enable validation
@ConstructorBinding         # Constructor-based binding (Java 16+)

@ConfigurationProperties(prefix = "app.mail")
@Validated
@ConstructorBinding
public record MailProperties(
    @NotBlank String host,
    @Min(1) @Max(65535) int port,
    @NotBlank String username,
    String password,
    @Valid SmtpProperties smtp
) {

    public record SmtpProperties(
        boolean auth = true,
```

```
        boolean starttls = true,
        String protocol = "smtp"
    ) {}
}

// Usage in configuration
@Configuration
@EnableConfigurationProperties(MailProperties.class)
public class MailConfig {

    @Bean
    public JavaMailSender mailSender(MailProperties properties) {
        JavaMailSenderImpl sender = new JavaMailSenderImpl();
        sender.setHost(properties.host());
        sender.setPort(properties.port());
        sender.setUsername(properties.username());
        sender.setPassword(properties.password());
        return sender;
    }
}
```

Import & Component Scanning

```
@Import                    # Import configuration classes
@ImportResource            # Import XML configuration
@ComponentScan            # Scan for components
@FilterType               # Filter components during scan

@Configuration
@Import({
    DatabaseConfig.class,
    SecurityConfig.class,
    WebConfig.class
})
@ComponentScan(
    basePackages = "com.example",
    excludeFilters = {
        @Filter(type = FilterType.ASSIGNABLE_TYPE, classes = TestConfig
        @Filter(type = FilterType.REGEX, pattern = "com\\.example\\.tes
    }
)
public class AppConfig {

    @Bean
    public static PropertySourcesPlaceholderConfigurer propertySourcesP
        return new PropertySourcesPlaceholderConfigurer();
    }
}
```

Lazy Loading & Performance

```
@Lazy                      # Lazy bean initialization
@Lazy(value = false)       # Eager initialization (override)
```

```
@Configuration
@Lazy // All beans in this configuration are lazy by default
public class LazyConfiguration {

    @Bean
    @Lazy(false) // Override configuration-level lazy
    public EagerService eagerService() {
        return new EagerService();
    }

    @Bean
    public LazyService lazyService() {
        return new LazyService();
    }
}
```

Application Runner & CommandLineRunner

```
@CommandLineRunner        # Run on application startup
@ApplicationRunner       # Run on application startup (with Application
```

```
@Component
@Order(1)
public class DataInitializer implements CommandLineRunner {

    @Override
    public void run(String... args) throws Exception {
        // Initialize data on startup
        log.info("Initializing application data...");
        initializeDefaultUsers();
        initializeDefaultSettings();
    }
}

@Component
@Order(2)
public class MigrationRunner implements ApplicationRunner {

    @Override
    public void run(ApplicationArguments args) throws Exception {
        // Run database migrations
        if (args.containsOption("run-migrations")) {
            log.info("Running database migrations...");
            migrationService.migrate();
        }
    }
}
```

---

12. Actuator & Monitoring Annotations

Health Indicators

```
@Component
public class DatabaseHealthIndicator implements HealthIndicator {

    @Override
    public Health health() {
        // Check database connectivity
        return Health.up()
            .withDetail("database", "MySQL")
            .withDetail("status", "Connected")
            .build();
    }
}
```

Metrics

```
@RestController
public class MetricsController {

    private final MeterRegistry meterRegistry;

    public MetricsController(MeterRegistry meterRegistry) {
        this.meterRegistry = meterRegistry;
    }

    @GetMapping("/api/process")
    public String process() {
        Timer.Sample sample = Timer.start(meterRegistry);
        try {
            // Process logic
            return "Processed";
        } finally {
            sample.stop(Timer.builder("process.timer").register(meterRe
        }
    }
}
```

---

12. Spring Boot 4 New Features & Annotations

Native Image Support

```
@NativeHint              // GraalVM native image hints
@RegisterReflectionForBinding // Register reflection for JSON binding
```

Observability

```
@Observed                // Observe method execution (Micrometer)
```

Problem Details

```
@ExceptionHandler         // Handle specific exceptions
@ResponseStatus          // Set response status for exceptions
```

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    @ResponseStatus(HttpStatus.NOT_FOUND)
    public ProblemDetail handleNotFound(ResourceNotFoundException ex) {
        ProblemDetail problem = ProblemDetail.forStatusAndDetail(
            HttpStatus.NOT_FOUND, ex.getMessage());
        problem.setProperty("timestamp", Instant.now());
        return problem;
    }
}
```

---

13. Common Annotation Combinations

REST Controller Pattern

```
@RestController
@RequestMapping("/api/v1/resources")
@Validated
public class ResourceController {

    @GetMapping("/{id}")
    public ResponseEntity<Resource> getResource(
            @PathVariable @Positive Long id,
            @RequestHeader("Accept") String acceptHeader) {
        // Implementation
    }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public Resource createResource(@Valid @RequestBody ResourceRequest
        // Implementation
    }
}
```

Service Layer Pattern

```
@Service
@Transactional
@Validated
@Slf4j
public class BusinessService {

    @Cacheable(key = "#id")
    @PreAuthorize("hasRole('USER')")
    public BusinessObject process(@Valid @NotNull Long id) {
        // Implementation
    }
}
```

Repository Pattern

```
@Repository
public interface DataRepository extends JpaRepository<Entity, Long> {

    @Query("SELECT e FROM Entity e WHERE e.status = :status")
    @Lock(LockModeType.PESSIMISTIC_READ)
    List<Entity> findByStatus(@Param("status") Status status);
}
```

---

14. Annotation Best Practices

Configuration Organization

- Use `@Configuration` classes for related beans
- Group beans by functionality (web, data, security)
- Use `@Profile` for environment-specific configurations
- Prefer constructor injection over field injection

Validation Strategy

- Validate at API boundaries (`@Valid` on `@RequestBody`)
- Use method-level validation in service layer
- Implement custom validators for business rules
- Use `@Validated` for method parameter validation

Security Implementation

- Apply security at both endpoint and method levels
- Use expression-based access control for fine-grained permissions
- Implement proper exception handling for security violations
- Test security configurations thoroughly

Testing Approach

- Use appropriate test slice annotations (`@WebMvcTest`, `@DataJpaTest`)
- Mock external dependencies with `@MockBean`
- Use `@TestPropertySource` for test-specific configurations
- Write integration tests with `@SpringBootTest`

---

15. Quick Reference Cheat Sheet

Category	Key Annotations	Primary Use
Core	`@Component`, `@Service`, `@Repository`, `@Controller`	Bean definition
DI	`@Autowired`, `@Qualifier`, `@Primary`, `@Resource`	Dependency injection
Config	`@Configuration`, `@Bean`, `@ComponentScan`	Application setup

Web	`@RestController`, `@GetMapping`, `@PostMapping`, `@RequestBody`	REST APIs
Data	`@Entity`, `@Repository`, `@Transactional`, `@Query`	Database access
Validation	`@Valid`, `@NotNull`, `@Size`, `@Email`	Input validation
Security	`@PreAuthorize`, `@PostAuthorize`, `@EnableWebSecurity`	Access control
Testing	`@SpringBootTest`, `@WebMvcTest`, `@MockBean`	Test setup
Caching	`@Cacheable`, `@CachePut`, `@CacheEvict`	Performance
Async	`@Async`, `@Scheduled`, `@EnableAsync`	Concurrency
AOP	`@Aspect`, `@Before`, `@After`, `@Around`	Cross-cutting concerns

---

16. Migration Notes (Spring Boot 3.x to 4.x)

Key Changes

- **Java 21** minimum requirement
- **Native Image** support improvements
- **Observability** enhancements with Micrometer
- **Problem Details** standardized error responses
- **Virtual Threads** support for concurrent applications

Deprecated Annotations

- Some `@ConditionalOn*` annotations have improved alternatives
- Legacy `@EnableWebMvc` replaced by more declarative approaches
- Security configuration moved to component-based setup

New Recommendations

- Use `@RegisterReflectionForBinding` for native image compatibility
- Adopt `@Observed` for method-level observability
- Leverage virtual threads with `@Async` for I/O-bound operations